ASSIGNMENT 2

QUESTIONS

CO2: Analyze public-key cryptosystem strategies using RSA and key exchange for encryption algorithms to strengthen information security.

CO2 Coverage: Symmetric key cryptosystems; Data Encryption Standard (DES); Triple DES (3DES); Blowfish; RC5; block ciphers; stream ciphers; block cipher modes of operation; public-key cryptosystems; RSA algorithm; key distribution and management; Diffie–Hellman key exchange; elliptic curve cryptography (ECC); introduction to quantum cryptography; number-theory foundations; RSA attacks and padding; digital signatures; hybrid encryption.

Assignment Title: Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network (Total Marks: 50)

Scenario — Secure Healthcare Data Exchange

A healthcare network connects one central hospital with remote clinics that exchange patient records, prescriptions, diagnostic reports and billing data over public networks. The network contains low-resource devices, high-volume file transfers and multiple security requirements. The security team must select appropriate symmetric and public-key techniques, establish keys securely, authenticate exchanges and prepare for long-term cryptographic migration.

The system must:

- Protect stored and transmitted clinical records with confidentiality.
- Establish session keys over an observable public network.
- Provide authentication and integrity for security-sensitive exchanges.
- Support large files without performing expensive public-key encryption on every data block.
- Remain practical for constrained clinics while allowing secure migration as threats and technology change.

Design Challenge

During peak operation the network expands to many remote sites. An attacker can observe traffic, attempt a man-in-the-middle attack during unauthenticated key exchange, exploit weak RSA parameters, and analyse repeated ciphertext patterns. Management also requires a migration plan for future quantum threats. Re-evaluate the original design using your mathematical and experimental evidence.

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)

Problem Context – Protecting Clinical Records

The network must compare classical symmetric choices before using public-key techniques. Evaluate DES, 3DES, Blowfish and RC5, together with the distinction between block and stream ciphers and the principal block-cipher modes.

Your Challenge

- Explain the structure, block size, key characteristics and round operation of DES, 3DES, Blowfish and RC5.

- Implement or simulate at least three of these algorithms on student-created test data and verify encryption/decryption.
- Explain the difference between block and stream ciphers and identify an appropriate use for each.
- Explain ECB, CBC, CFB, OFB and CTR, including IV/nonce requirements, pattern leakage, error propagation and parallelism.
- Demonstrate why ECB can reveal repeated plaintext patterns and why IV/nonce reuse is dangerous.
- Compare operation count and execution time for increasing message sizes.

Evaluation

Rank DES, 3DES, Blowfish and RC5 for the scenario, identify legacy limitations, and recommend where a block or stream construction would be appropriate. Do not treat a larger key space alone as proof of security.

Evidence Expected: Algorithm comparison, mode comparison, test results, pattern/IV demonstration and a justified recommendation.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)

Problem Context – RSA Key Generation

RSA depends on modular arithmetic, modular inverses, modular exponentiation and the generation of suitable primes.

Your Challenge

- Implement GCD, Extended Euclidean Algorithm, modular inverse and fast modular exponentiation.
- Compare trial-search inverse with Extended Euclid and naive repeated multiplication with square-and-multiply.
- Explain Euler's totient function and the condition gcd(e, φ(n)) = 1.
- Compare trial division, Fermat and Miller–Rabin on safe toy candidates, including Carmichael numbers such as 561, 1105 and 1729.
- Record modular multiplications, division/loop steps and false acceptance results.
- Explain how prime density affects the number of candidates tested.
- Re-evaluate the methods when the RSA modulus grows from classroom-sized values toward 1024-, 2048- and 3072-bit engineering requirements.

Evidence Expected: Mathematical derivations, pseudocode, designed test cases, operation counts, primality-test evidence and engineering decision.

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)

Problem Context – Online Banking Message Protection

A bank uses textbook RSA for short tokens. The security team must determine whether correct decryption is sufficient evidence of secure encryption.

Your Challenge

- Generate RSA keys and implement C = M^e mod n and M = C^d mod n.

- Verify one hand-checkable RSA example.
- Test deterministic encryption, repeated-message encryption, small-message/small-exponent behaviour, boundary messages and invalid parameter choices.
- Explain how factoring n permits recovery of φ(n) and d.
- Implement or simulate trial-division and Fermat factorisation on weak toy moduli, including close primes and a small factor.
- Explain why RSA key length alone does not guarantee security and why weak key generation can undermine a large modulus.
- Explain RSA padding, especially OAEP, and how randomisation prevents structural weaknesses of textbook RSA.

Inquiry

Predict each test before execution, compare prediction with evidence, and state the limitations of classroom-scale factoring experiments.

Evidence Expected: Correct RSA implementation, attack traces, factoring evidence, padding analysis and security conclusions.

Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

Problem Context – Secure Link Between Remote Clinics

Two clinics must establish a shared session key over a network that an attacker can fully observe, without having a pre-shared secret.

Your Challenge

- Derive the Diffie–Hellman procedure from (g^a)^b ≡ (g^b)^a (mod p).
- Develop pseudocode showing private values, public values and the shared-key computation.
- Test a small hand-verifiable example and a larger toy configuration.
- Measure the legitimate computation and a brute-force discrete-log attack as p grows.
- Demonstrate an active man-in-the-middle attack against unauthenticated Diffie–Hellman.
- Explain how digital signatures or certificates authenticate the exchanged values and defeat the attack.
- Design a key-distribution and key-management policy covering generation, storage, rotation, revocation, compromise recovery and session-key lifetime.

Evidence Expected: Protocol derivation, message-sequence diagram, implementation, MITM demonstration, growth analysis and key-management policy.

Q5 – ECC and Public-Key Strategy Comparison (5 Marks)

Problem Context – Constrained Devices

Remote medical devices need public-key security with manageable key size, computation and communication overhead.

Your Challenge

- Explain the mathematical basis of elliptic-curve public-key cryptography and demonstrate a toy ECC key-generation/key-agreement example.
- Compare RSA, Diffie–Hellman and ECC in key size, computational cost, bandwidth and deployment considerations.

- Explain why ECC can provide comparable security with smaller public-key material than traditional RSA-style parameters.
- Identify implementation requirements such as secure randomness, parameter validation and protected private keys.
- Recommend a public-key strategy for constrained devices and justify it using the experimental evidence from Q2–Q4.

Evidence Expected: Toy ECC example, RSA/DH/ECC comparison table and a defended recommendation.

Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks)

Problem Context – Large Confidential Document Exchange

The healthcare network must transfer large files with confidentiality, integrity and sender authenticity. The team is comparing direct public-key protection with hybrid encryption and must also consider future quantum threats.

Your Challenge

- Design a digital-signature process using hash-then-sign and explain signing and verification.
- Compare RSA, DSA/DSS and ECDSA at the conceptual level with respect to keys, signing/verification, randomness and security assumptions.
- Design hybrid encryption: encrypt the large payload with a random symmetric session key, then protect or establish that session key using RSA or Diffie–Hellman; authenticate the exchange.
- Test tampering of the file, signature and protected session key.
- Compare direct RSA encryption of large data with hybrid encryption using operation counts and file-size growth.
- Explain the basic principle of quantum key distribution/quantum cryptography and how eavesdropping can be detected.
- Explain why sufficiently capable quantum computers threaten RSA and ECC and propose a migration/readiness strategy without claiming that quantum cryptography itself replaces ordinary cryptographic key management.

Deployment Decision

Choose a practical architecture for the healthcare network and justify the choice using performance, security, key-management burden and long-term risk.

Evidence Expected: Signature flow, hybrid-encryption design, tamper tests, growth comparison, quantum-security explanation and final architecture.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 Marks)

Review the evidence from Q1–Q6 and identify one significant computational or security limitation that becomes important as message volume, key size, number of clinics or attacker capability increases.

A. Diagnose the Bottleneck or Weakness (2 Marks)

- Identify the operation or missing security property responsible for the limitation.
- Explain how its cost or risk changes with input/key size or repeated use.
- Use measured operation counts or attack evidence from earlier tasks.

B. Define the Design Objective (1 Mark)

State a measurable redesign objective, such as reducing public-key operations, preventing a man-in-the-middle, reducing key-management exposure, detecting weak keys, or improving secure bulk-data handling.

C. Design the Revised Strategy (3 Marks)

Develop a revised architecture using the appropriate combination of symmetric encryption, secure modes, RSA/DH/ECC, digital signatures, key management and hybrid encryption. Explain the mathematical/security property exploited, operations reduced, additional state or keys required, and new trade-offs.

D. Integrate Quantum Readiness (1 Mark)

Identify which RSA/ECC dependencies are threatened by future quantum computing and define a migration approach. Explain separately what quantum key distribution can contribute and what it cannot replace.

E. Predict, Implement and Validate (2 Marks)

Predict the improvement before implementation. Compare original and redesigned solutions using operation counts, attack steps, key-management events, verification results and execution time where useful. Verify that the redesign preserves required functionality.

F. Defend the Final Design (2 Marks)

Defend the final decision against the original alternatives. Discuss residual risks, deployment cost, usability, key compromise, MITM attacks, weak keys, large-file performance and long-term quantum risk.

Evidence Expected: Diagnosis, measurable objective, revised architecture/protocol, prediction, comparable experimental evidence and final engineering defence.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Symmetric cryptosystems, DES/3DES/Blowfish/RC5, block/stream ciphers and modes	7
Q2	Number theory, modular arithmetic and prime selection	6
Q3	RSA encryption/decryption, attacks, weak keys and padding	7
Q4	Diffie–Hellman, MITM authentication and key management	7
Q5	ECC and RSA/DH/ECC comparison	5
Q6	Digital signatures, hybrid encryption and quantum readiness	7
Q7	Day-4 integrated diagnose, redesign and validation	11
TOTAL		50

Submission Requirements

Submit your assignment report containing Q1–Q7 evidence, source code for student-implemented core algorithms, raw experimental data, tables/graphs and protocol/message-sequence diagrams. Libraries may be used for cross-checking standard primitives, but the mathematical core and experiments must be your own.

All attack demonstrations must use only student-created toy parameters and test data. Do not use classroom code to protect real patient data.

Assessment Verification

Students may be asked to explain algorithm design, source code, test data, mathematical predictions, experimental results, security analysis and the Day-4 redesign during a brief verification/viva. Both the reasoning and the evidence must be understood by the student.

- Algorithmic and protocol design
- Source code
- Test-data and key-set design
- Experimental results
- Predictions
- Security analysis
- Design Challenge

during a brief verification discussion/viva. The student should therefore be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook cryptosystems.

It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Attack → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.

SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Department of Information Technology

CSA5103 – Cryptography and Network Security

ASSIGNMENT 2 – CO2

Design, Analyse and Validate Public-Key and Hybrid Cryptographic Strategies for a Multi-Site Healthcare Network

Guide to this report

The assignment sheet (reproduced unchanged on the preceding pages) asks seven questions. Each question is answered in its own section below, in the order in which the sheet lists its "Your Challenge" bullets. Every claim is tied to an experiment that was run: the experiment is described in the section, summarised in a table or figure, and its full source code is in the appendices. Table 1 shows where to find the evidence expected for each question.

Table 1. *Mark distribution and map from each question to its evidence*

Q	Assessed area	Marks	Evidence expected by the sheet	Where answered	Source (appendix)
Q1	Symmetric cryptosystems, DES/3DES/Blowfish/ RC5, block/stream ciphers, modes	7	Algorithm comparison, mode comparison, test results, pattern/IV demonstration, justified recommendation	Section 1	A (symmetric.py), F (run_q1.py)
Q2	Number theory, modular arithmetic, prime selection	6	Algorithm derivation, operation counts, pseudocode, test cases, scaling analysis	Section 2	B (ntcore.py), F (run_q2.py)
Q3	RSA encryption/decryption, attacks, padding	7	Correct RSA, attack results, factoring evidence, padding analysis	Section 3	C (rsa.py), F (run_q3.py)
Q4	Diffie–Hellman, MITM, authentication, key management	7	Derivation, pseudocode, protocol diagrams, MITM demonstration, key-management policy	Section 4	D (dh.py), F (run_q4.py)

Q5	ECC and RSA/DH/ECC comparison	5	Toy ECC derivation, comparison table, defended recommendation	Section 5	D (ecc.py), F (run_q5.py)
Q6	Digital signatures, hybrid encryption, quantum readiness	7	Signature/hybrid flow, tamper tests, QKD vs PQC, migration plan	Section 6	E (hybrid.py), F (run_q6.py)
Q7	Day-4 diagnose, redesign and validate	11	Diagnosis, measurable objective, revised architecture, predictions vs measurements, defence	Section 7	E (redesign.py), F (run_q7.py)

Conventions and assumptions used throughout

- **Test data are synthetic.** No real patient data were used. Patient identifiers, doses and lab values in the examples are invented for illustration (the sheet requires student-created test data).
- **Attacker model.** The attacker can observe, record, replay, drop and modify every message on the network (as stated in the scenario) but cannot read the private memory of a device unless a test says that a key was stolen. Mallory/Eve are used for the active/passive attacker.
- **Cipher substrate.** Because the symmetric primitives must be the student's own code, the bulk-data layer in Q4–Q7 uses the student-implemented Blowfish in CTR mode with HMAC-SHA256 (encrypt-then-MAC). Production systems should use AES-256-GCM or ChaCha20-Poly1305; the structure of the protocol is identical and only the cipher call changes. SHA-256, HMAC and the pseudo-random generator come from Python's standard library.
- **Randomness.** Experiments use a seeded random.Random so every number in this report can be reproduced exactly. A deployed system must use an operating-system cryptographic generator; Q6 shows concretely what goes wrong when an ECDSA nonce repeats.
- **Timing figures** come from pure-Python code on a shared cloud machine. They are used to compare designs with each other, not to quote hardware speeds; operation counts (exact and machine independent) are the primary evidence and times are secondary. Run-to-run time variation of up to about ±40% was observed.

- **Verification discipline.** Each implementation was checked against published known-answer vectors or an independent library before being used (Section 1.2 for the symmetric ciphers, Section 3.2 for RSA, Section 5.4 for P-256). The consolidated regression test (Appendix G) passes 36 of 36 checks.
- **Prediction before measurement.** For Q3 and Q7 the expected result of every test was written down before the test was run; the tables show the prediction next to the observation.

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 marks)

Problem context. Protecting clinical records. The network must choose among classical symmetric ciphers and the principal block-cipher modes. DES, 3DES, Blowfish and RC5 were implemented from scratch, tested against published vectors, and compared on structure, cost, mode behaviour and security margin.

1.1 Structure, block size, key characteristics and round operation

All four ciphers are 64-bit-block designs. DES, 3DES and Blowfish are Feistel networks (each round replaces the pair (L, R) by (R, L ⊕ F(R, K))), so decryption is the same circuit with the round keys reversed. RC5 is a Feistel-like design whose distinguishing feature is **data-dependent rotation**: the rotation amount is taken from the other half of the block.

Table 2. *Structure and key characteristics of the four ciphers*

Property	DES	3DES (EDE)	Blowfish	RC5-32/12/16
Block size	64 bits	64 bits	64 bits	64 bits (2 × 32-bit words)
Key size	56 effective bits (64 with 8 parity bits)	112 or 168 bits nominal (2 or 3 DES keys)	32–448 bits (variable)	Variable, 0–2040 bits (16 bytes used here)
Rounds	16	48 (3 × 16)	16	Variable; r = 12 here
Round function	Expansion E (32→48), XOR 48-bit subkey, eight 6→4 S-boxes, permutation P	Three DES passes: C = $E^{K_3}(D^{K_2}(E^{K_1}(P)))$	F(x) = (($S_0[a]$ + $S_1[b]$) ⊕ $S_2[c]$) + $S_3[d]$; four 8→32 S-boxes	A = ((A⊕B) <<< B) + S[2i]; B = ((B⊕A) <<< A) + S[2i+1]
Key schedule	PC-1, two 28-bit halves rotated by 1 or 2, PC-2 →	Three DES schedules	18 P-words + 1024 S-box words from digits of π, then 521	2(r+1) = 26 words from key via 3·max(t, c) mixing steps

	sixteen 48-bit subkeys		block encryptions to mix in the key	
State kept per key	128 bytes of subkeys	384 bytes	4,168 bytes (P + S-boxes)	104 bytes
Non-linear component	8 fixed S-boxes	same as DES	Key-dependent S-boxes	Data-dependent rotations (no S-boxes)
Mathematical weaknesses to know	Exhaustive search of 2^{56} keys; 4 weak and 12 semi-weak keys	Meet-in-the-middle: ~2^{112} work; 64-bit block → birthday attacks (Sweet32)	64-bit block; weak-key classes (colliding S-box entries) exist but are not exploitable at 16 rounds; expensive key setup	Smaller public analysis than AES; 64-bit block

Why 3DES is not 168-bit secure. If the cipher were simply applied twice with independent keys, a meet-in-the-middle attack (encrypt forward under every K_1, decrypt backward under every K_2, and match in the middle) needs about $2^2 \times 2^{56}$ rather than 2^{112} operations. Section 1.7 demonstrates exactly this attack on a reduced-key double-DES. For EDE with three keys the same idea reduces 168 bits to about 112 bits of effective strength.

1.2 Implementation and verification

Each cipher was written from the published specification. Tables for DES (IP, FP, E, P, PC-1, PC-2, eight S-boxes) were entered by hand; the Blowfish constants are **computed** (the 8,336 hexadecimal digits of π are generated with Machin's formula $\pi = 16 \arctan(1/5) - 4 \arctan(1/239)$ in big-integer arithmetic) rather than typed. Known-answer tests (KAT) and a round-trip test over five message shapes were run for every cipher.

Table 3. *Known-answer tests (single block)*

Cipher	Computed ciphertext	Published/reference value	Match
DES	85E813540F0AB405	85E813540F0AB405	Yes
Blowfish	4EF997456198DD78	4EF997456198DD78	Yes
RC5-32/12/16	21A5DBEE154B8F6D	21A5DBEE154B8F6D	Yes
3DES (EDE, 3 keys)	A826FD8CE53B855F	A826FD8CE53B855F	Yes

The 3DES vector is the NIST three-key example; DES, Blowfish and RC5 are the standard published test vectors. In addition: 3DES and Blowfish outputs were compared with an independent library on random inputs (identical), RC5 matched a second published vector (key 915F4619… → F7C013AC5B2B8952), and 3DES with $K_1 = K_2 = K_3$ reproduced single DES exactly (tested in Appendix G).

Table 4. *CBC round trips with PKCS#7 padding: 20 of 20 pass*

Cipher	Message (student-created)	Plain bytes	CBC cipher bytes	First block of ciphertext	Decrypts correctly
DES	empty	0	8	ad4b05475dee3b18	Yes
DES	1 byte	1	8	d7097e1f78a928f2	Yes
DES	exact 8 bytes	8	16	b5cc1a99c9ff8183	Yes
DES	record 37 B	37	40	44fc891d94daa5c8	Yes
DES	lab report 1000 B	1000	1008	f45c38b1693c8857	Yes
3DES	empty	0	8	d06a88d38fba9159	Yes
3DES	1 byte	1	8	728600f3a90096ad	Yes
3DES	exact 8 bytes	8	16	81bd262ac224e994	Yes
3DES	record 37 B	37	40	65ed48fb10ccf4ae	Yes
3DES	lab report 1000 B	1000	1008	6dfe46edd6f98c26	Yes
Blowfish	empty	0	8	f908d5464189ee01	Yes
Blowfish	1 byte	1	8	eb77de5012ba052e	Yes
Blowfish	exact 8 bytes	8	16	fabcde158d4398c8	Yes
Blowfish	record 37 B	37	40	2cbc2e2da958906c	Yes
Blowfish	lab report 1000 B	1000	1008	770dff2a4f8fac70	Yes
RC5-32/12/16	empty	0	8	88e7bc54f1e8310a	Yes
RC5-32/12/16	1 byte	1	8	9c28f2e69ae12b04	Yes
RC5-32/12/16	exact 8 bytes	8	16	6ad1f7c32625679d	Yes

RC5-32/12/16	record 37 B	37	40	a0b13a1623e3665b	Yes
RC5-32/12/16	lab report 1000 B	1000	1008	90ef93092ae0dcfc	Yes

The empty message, a one-byte message and an exact 8-byte message are the boundary cases: with PKCS#7 padding an empty plaintext still produces one full padding block and an exact multiple of the block size gains an extra block, so the receiver can always remove padding unambiguously.

1.3 Block cipher versus stream cipher

Table 5. *Block versus stream cipher*

Aspect	**Block cipher (DES, 3DES, Blowfish, RC5)**	**Stream cipher (RC4; CTR/OFB keystream)**
Unit of work	Fixed 64-bit block, permutation keyed by K	One byte or bit at a time, XORed with a keystream
Needs padding	ECB and CBC need padding; CFB/OFB/CTR do not	No
Latency	Waits for a full block (or uses CFB-8)	Immediate; good for continuous telemetry
Main risk	Pattern leakage (ECB); small block (64-bit) limits data per key	Keystream reuse is fatal: $c_1 \oplus c_2 = p_1 \oplus p_2$
Error behaviour	Mode dependent (Table 7)	A flipped bit stays one flipped bit (malleable)
Measured here	Blowfish-CTR 99 ms for 64 KB	RC4 7 ms for 64 KB (pure Python)

A block cipher in CTR, OFB or CFB mode is itself a stream cipher, so the two families are not disjoint. RC4 was implemented only to demonstrate keystream reuse (Section 1.5): it also has statistical biases in its early output and is prohibited in TLS (RFC 7465), so it is not recommended. **Where each belongs in the network:** block ciphers in an authenticated mode for stored records and file transfer; a stream construction (CTR/ChaCha-style keystream with a per-message nonce and a MAC) for continuous device telemetry where latency matters.

1.4 Modes of operation: ECB, CBC, CFB, OFB and CTR

Table 6. *Properties of the five modes (derived from the algorithms, then confirmed by experiment)*

Mode	**IV / nonce**	**Padding**	**Equal plaintext blocks**	**Encrypt parallel**	**Decrypt parallel**	**Error from 1 flipped ciphertext bit**
ECB	none	yes	Equal ciphertext blocks (leaks pattern)	Yes	Yes	That block only (random garbage)
CBC	unpredictable IV, per message	yes	Hidden	No (chained)	Yes	That block garbled + same bit flipped in next block
CFB	unpredictable IV	no	Hidden	No	Yes	Same bit in that block + next block garbled
OFB	unique IV	no	Hidden	No (keystream can be precomputed)	No (same)	Only that bit
CTR	unique nonce, counter	no	Hidden	Yes	Yes	Only that bit

Table 7. *Measured error propagation (ECB, CBC, CFB, OFB, CTR)*

Mode	**Ciphertext blocks damaged? (plaintext blocks wrong)**	**Total plaintext bits wrong (80-byte message, 1 ciphertext bit flipped in block 2)**
ECB	[2]	38
CBC	[2, 3]	35
CFB	[2, 3]	42
OFB	[2]	1
CTR	[2]	1

The measurements agree with the theory: ECB damages one block (about half of its 64 bits), CBC damages block 2 and flips exactly one bit in block 3, CFB flips one bit in block 2 and garbles block 3, and OFB/CTR change exactly one bit. The last two are therefore **malleable**: an attacker who knows the plaintext can flip chosen bits without knowing the key. This is why every use of these modes in Q4–Q7 is followed by a MAC.

1.5 Demonstrations: ECB pattern leakage and IV/nonce misuse

(a) ECB reveals structure

A 96×96 grey-scale image (a "medical cross", one byte per pixel, 1,152 blocks) was encrypted with Blowfish in ECB, CBC and CTR using the same key. Figure 1 shows the outcome; Table 8 gives the block statistics.

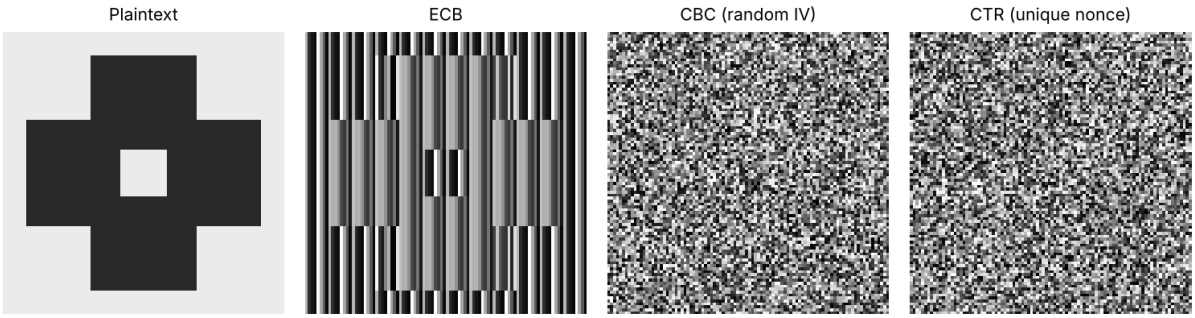


Figure 1. *ECB leaks the outline of the image; CBC and CTR do not (student-implemented Blowfish)*

Table 8. *Distinct 64-bit blocks in the image data*

Data	Blocks	Distinct blocks
plain	1152	4
ECB	1152	4
CBC	1152	1152
CTR	1152	1152

Text example: five 16-byte diagnosis records (DX=DIABETES, DX=ASTHMA, …) with only two distinct diagnoses give 11 ciphertext blocks of which only **5** are distinct under ECB, but **11** of 11 are distinct under CBC. An eavesdropper under ECB learns which patients share a diagnosis without any key.

(b) Reusing an IV or a CTR nonce is dangerous

- **CBC, same IV for two messages.** The two messages begin PATIENT and PATIENT ; the first ciphertext blocks are equal: yes (the plaintext first blocks are equal: yes). With a fixed IV an observer learns exactly how long the common prefix of two messages is.
- **CTR, same nonce for two messages.** $c_1 \oplus c_2$ equals $p_1 \oplus p_2$: yes. Using the guessable header PATIENT 20481 DIAGNOSIS: as a known fragment, the attacker XORs it into the stream and reads the second message's start: PATIENT 77310 DIAGNOSIS: . With a unique nonce the relation no longer holds: yes.
- **RC4 with a repeated key** has the same flaw (keystream reuse reveals $p_1 \oplus p_2$): yes.

Rule adopted: CBC/CFB need a fresh unpredictable IV; OFB and CTR need a never-repeating nonce (the redesign in Q7 derives a different key for every file and also keeps a strictly increasing counter, so reuse is impossible by construction).

1.6 Operation counts, execution time and the effect of message size

Table 9. *Counted operations for a 4,096-byte message (CTR mode)*

Cipher	Key setup (ms)	Block calls	Rounds	S-box look-ups	XOR/add ops	Time for 4 KB (ms)
DES	2.300	512	8192	65536	16384	1,156.5
3DES	6.740	1536	24576	196608	49152	3,507.2
Blowfish	5.173	512	8192	32768	24576	5.7
RC5-32/12/16	0.076	512	6144	0	36864	4.7

Per block, the counts follow the structure: DES performs 16 rounds with 8 S-box look-ups each, 3DES three times that, Blowfish 16 rounds with 4 look-ups, and RC5 12 rounds with no look-ups at all. Blowfish pays instead at start-up: the 521 block encryptions of its key schedule make key setup about 68 times slower than RC5's, a real cost for devices that rekey often. (Our DES is slower than Blowfish mainly because it permutes bit-lists; a hardware or table-driven DES is far faster. The counts, not these Python times, are the evidence.)

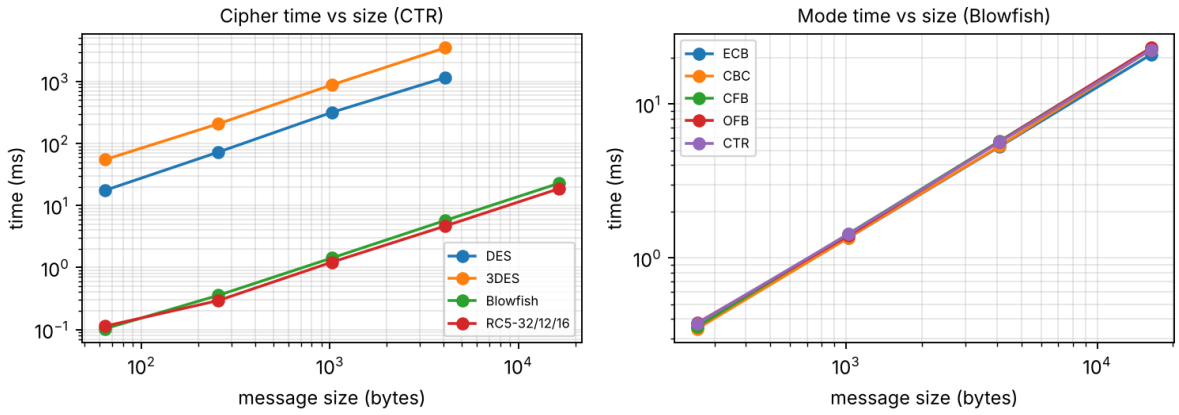


Figure 2. *Time versus message size (log-log): every cipher and mode is linear in the number of blocks*

Table 10. *Time in ms by mode and message size*

Mode (Blowfish)	256 B	1 KB	4 KB	16 KB	Block calls for 16 KB
ECB	0.4	1.4	5.3	21.0	2049
CBC	0.3	1.4	5.3	22.5	2049
CFB	0.4	1.4	5.7	23.1	2048
OFB	0.4	1.4	5.7	23.2	2048
CTR	0.4	1.4	5.7	22.6	2048

All five modes cost one block-cipher call per block (2,049 for 16 KB including the padding block), so the differences between them are within measurement noise; the choice of mode is therefore made on security and parallelism, not speed. Doubling the message doubles the time (linear growth in Figure 2). Key setup is a one-off cost, so for large messages it is irrelevant, but for many tiny messages under frequently changing keys Blowfish's setup dominates.

1.7 Key-space and block-size evidence

Exhaustive search. DES has 2^{56} = 72,057,594,037,927,936 keys; a machine testing 10^{12} keys per second would need about 20 hours, which is within reach of dedicated hardware (and of key-search clusters), so DES must be treated as broken. **Meet-in-the-middle demonstration.** Double DES was attacked with 12 unknown bits in each of two keys. A brute-force search needs 2^{24} = 16,777,216 double-encryptions; the meet-in-the-middle attack needed only 8,192 single DES operations plus a table look-up and found the secret in 38 s. It returned 4 candidate pairs, all equivalent: the 12 unknown bits include the DES parity bit of the first key byte

in each key, which DES ignores, so each real key appears twice (0xa4c/0x3e7, 0xa5c/0x3e7, 0xa4c/0x3f7, 0xa5c/0x3f7; true pair 0xa5c/0x3e7). **Block size.** With a 64-bit block, collisions between ciphertext blocks are expected after about 2^{32} blocks (32 GB); in CBC two equal ciphertext blocks reveal the XOR of two plaintext blocks (the Sweet32 attack). Any key must therefore be changed long before 32 GB of data, which is easily reached by a hospital image archive.

1.8 Evaluation and ranked recommendation

Table 11. *Scoring of the four ciphers for the healthcare network (scores are the student's judgement from the measured data above)*

Criterion (1 = poor, 5 = good)	DES	3DES	Blowfish	RC5-32/12/16
Key-space margin against search	1	3 (112-bit)	5 (up to 448-bit)	4 (128-bit key)
Public cryptanalysis record	2 (broken by search)	3 (deprecated)	4	3
Block size (64-bit limit on data per key)	1	1	1	1
Speed (round operations per block)	3	1 (3× DES)	4	5
Memory/state and start-up cost	4	3	2 (4 KB; slow setup)	5 (104 B; instant setup)
Suitability for small clinic devices	2	1	3	5
Total (out of 30)	13	12	19	23

Evaluation and recommendation (Q1)

Ranking among the four requested ciphers: (1) RC5-32/12/16 for constrained remote devices; (2) Blowfish for central servers that load a key once and process large volumes; (3) 3DES only for compatibility with legacy equipment, with data per key limited far below 32 GB; (4) DES, never.

Where block vs stream is appropriate: block cipher in CTR (or CBC with random IV) **plus a MAC** for stored records and file transfer; keystream/CTR with per-message nonce and MAC for continuous telemetry. ECB is never appropriate for records with repeated fields.

Caveat the evaluation must state: none of the four has a 128-bit block. For anything new, AES-256-GCM or ChaCha20-Poly1305 should replace them; the rest of this report keeps the structure but swaps only the cipher call. Larger key space alone does not make a cipher safe: 3DES has 168 key bits but 112-bit strength, and a 64-bit block limits the data protected by any key.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 marks)

Problem context. RSA key generation depends on modular arithmetic, modular exponentiation and generating large primes. The routines below were written from first principles (no use of pow(a, b, m) or gcd from libraries in the engine; the built-ins appear only as cross-checks), instrumented to count multiplications, divisions and loop iterations, and tested on designed cases.

2.1 Algorithms, derivation and pseudocode

Euclid's algorithm and the extended Euclidean algorithm

Since gcd(a, b) = gcd(b, a mod b), repeated division strictly shrinks the pair, and the number of steps is O(log b) (worst case consecutive Fibonacci numbers). Working backwards through the quotients expresses the gcd as a combination $a \cdot x + b \cdot y = g$. When $g = 1$ this gives the inverse: $a \cdot x \equiv 1 \pmod{b}$, so $x \bmod b = a^{-1}$.

```
EXTENDED-EUCLID(a, b):
  (x0, y0, x1, y1) <- (1, 0, 0, 1)
  while b != 0:
     q <- a div b
     (a, b) <- (b, a - q*b)
     (x0, x1) <- (x1, x0 - q*x1)
     (y0, y1) <- (y1, y0 - q*y1)
  return (a, x0, y0)
```

```
MOD-INVERSE(a, m):
  (g, x, y) <- EXTENDED-EUCLID(a mod m, m)
  if g != 1: error "no inverse"
  return x mod m
```

Fast modular exponentiation (square-and-multiply)

Write the exponent in binary, $e = \Sigma\, e_i\, 2^i$. Then $b^e = \Pi\, (b^{(2^i)})$ over the set bits, so repeated squaring gives all $b^{(2^i)}$ and one multiplication is added for each 1 bit. For a k-bit exponent this costs k squarings plus (number of 1 bits) multiplications: at most 2k and on average 1.5k multiplications, instead of $e - 1 \approx 2^k$ for repeated multiplication. Reducing mod m after every product keeps numbers below m^2.

```
MOD-EXP(b, e, m):
  r <- 1; b <- b mod m
  while e > 0:
     if e is odd: r <- r*b mod m
     e <- e div 2
     if e > 0: b <- b*b mod m
```

```
MILLER-RABIN(n, k):
  write n-1 = 2^s * d, d odd
  repeat k times:
     a <- random in [2, n-2]; x <- a^d mod n
     if x = 1 or x = n-1: continue
     repeat s-1 times: x <- x^2 mod n;
```

```
return r                    if x = n-1: break
                            else: return COMPOSITE
                        return PROBABLY-PRIME
```

Euler's totient and the condition gcd(e, φ(n)) = 1

For n = pq with distinct primes, φ(n) = (p−1)(q−1) because the integers in 1..n that share a factor with n are the p multiples of q and the q multiples of p, with n counted twice. By Euler's theorem M^φ(n) ≡ 1 (mod n) when gcd(M, n) = 1; so if e·d ≡ 1 (mod φ(n)), then (M^e)^d ≡ M. The inverse d exists **only if gcd(e, φ(n)) = 1**; otherwise M ↦ M^e is not one-to-one and decryption is ambiguous (demonstrated in Section 3.4).

2.2 Designed test cases and results

Table 12. *Number-theory tests: 14 of 14 pass*

Test (expected value derived by hand or from a theorem)	**Result**	**Expected**	**Pass**
gcd(1071, 462)	21	21	Yes
gcd(17, 0)	17	17	Yes
gcd(0, 0)	0	0	Yes
egcd(240, 46) -> (g, x, y)	(2, -9, 47)	(2, -9, 47)	Yes
240*(-9) + 46*47	2	2	Yes
modinv(17, 3120)	2753	2753	Yes
modinv(3, 11)	4	4	Yes
modinv(6, 9) (gcd = 3)	ValueError	ValueError	Yes
modexp(4, 13, 497)	445	445	Yes
modexp(7, 0, 13)	1	1	Yes
modexp(5, 3, 1)	0	0	Yes
modexp(2, 10, 1000)	24	24	Yes
modexp(123456789, 987654321, 1000000007) = pow()	652541198	652541198	Yes
2000 random cross-checks vs Python built-ins (failures)	0	0	Yes

The cases include edge conditions the sheet asks for: gcd with zero, an inverse that must not exist (gcd 3), exponent 0, modulus 1, and 2,000 random cross-checks against Python's built-ins with no disagreement. A Bezout identity check (240·(−9) + 46·47 = 2) confirms the coefficients, not only the gcd.

2.3 Inverse by trial search versus extended Euclid; naive versus fast exponentiation

Table 13. *Modular inverse of a random a (mean of 20 random a per modulus)*

Modulus m (prime)	Trial search: multiplications	Extended Euclid: divisions	Trial time (µs)	Euclid time (µs)	Step ratio
101	54	5.5	10.3	1.70	10 ×
1,009	457	7.2	88.0	2.68	63 ×
10,007	4,027	9.2	754.4	3.95	435 ×
100,003	51,381	11.5	12,110.5	6.81	4,468 ×
1,000,003	591,536	12.6	115,097.1	9.32	46,947 ×
10,000,019	5,212,372	14.8	1,024,954.3	12.52	353,381 ×

Table 14. *Modular exponentiation 7^e mod 1,000,003 with e = 2^k − 1 (worst case: all bits set)*

Exponent bits	Exponent	Naive multiplications	Square-and-multiply	Naive time (µs)	S&M time (µs)
4	15	14	7	9.5	4.6
8	255	254	15	41.6	3.5
12	4,095	4,094	23	749.2	6.1
16	65,535	65,534	31	12,737.9	12.9
20	1,048,575	1,048,574	39	200,038.3	17.2

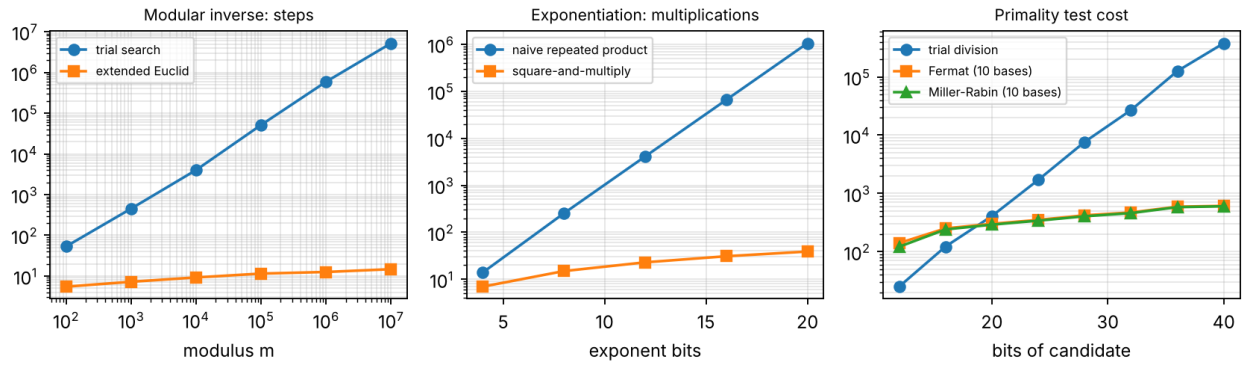


Figure 3. *Left: inverse steps; centre: exponentiation multiplications; right: primality-test cost (all log scale)*

Trial search grows linearly with m (about m/2 steps on average) while the extended Euclidean algorithm grows with the **number of digits** of m: the step ratio grows from tens to hundreds of thousands in six decades. For exponentiation, naive repeated multiplication needs e − 1 products (exactly $2^{20}-2$ for a 20-bit exponent) while square-and-multiply needs only 2k−1; at 1,024 bits the naive method would need about 2^{1024} products, which is physically impossible, while square-and-multiply needs about 1,536 on average. These two algorithms are what makes RSA computable at all.

2.4 Euler's function and admissible public exponents

Table 15. *φ(n) by formula versus by counting; which e satisfy gcd(e, φ) = 1*

p	q	n = pq	φ(n) = (p−1)(q−1)	Count of k < n with gcd(k, n) = 1	Equal ?	Admissible e from {3, 5, 7, 17, 65537}
3	11	33	20	20	Yes	3, 7, 17
7	13	91	72	72	Yes	5, 7, 17
11	13	143	120	120	Yes	7, 17
61	53	3233	3120	3120	Yes	7, 17

For (p, q) = (3, 11), φ = 20 and e = 5 is excluded (gcd 5); for (7, 13), φ = 72 and e = 3 is excluded because gcd(3, 72) = 3; for (11, 13), φ = 120 and both 3 and 5 are excluded. This is why a key generator must test the condition and, if it fails, choose new primes rather than silently proceed.

2.5 Primality tests: trial division, Fermat and Miller–Rabin

Fermat test. If n is prime then a^(n−1) ≡ 1 (mod n) for gcd(a, n) = 1. It is cheap, but a **Carmichael number** satisfies the congruence for *every* base coprime to n, so the test cannot expose it by any choice of base. **Miller–Rabin** strengthens the test by examining the chain a^d, a^(2d), … and demanding that a 1 can only be preceded by ±1; for an odd composite at most one quarter of the bases are "liars", so k random rounds give error $\leq 4^{-k}$, and no Carmichael number escapes this bound.

Table 16. *Exact liar counts by exhaustive enumeration over all bases (Carmichael numbers 561–8911; 2047 = 23×89 is the smallest strong pseudoprime to base 2; 91 and 221 are ordinary composites)*

n	Factorisation	Bases a in [2, n−2]	Fermat liars (all bases)	Miller–Rabin liars	Fermat, a = 2	MR, a = 2
561	3x11x17	558	318 (57.0%)	8 (1.43%)	passes	fails
1105	5x13x17	1102	766 (69.5%)	28 (2.54%)	passes	fails
1729	7x13x19	1726	1294 (75.0%)	160 (9.27%)	passes	fails
2465	5x17x29	2462	1790 (72.7%)	68 (2.76%)	passes	fails
2821	7x13x31	2818	2158 (76.6%)	268 (9.51%)	passes	fails
6601	7x23x41	6598	5278 (80.0%)	328 (4.97%)	passes	fails
8911	7x19x67	8908	7126 (80.0%)	1780 (19.98%)	passes	fails
2047	23x89	2044	482 (23.6%)	240 (11.74%)	passes	passes
91	7x13	88	34 (38.6%)	16 (18.18%)	fails	fails
221	13x17	218	14 (6.4%)	4 (1.83%)	fails	fails

For each Carmichael number the Fermat test accepts **every** base coprime to n, so 57–80% of all bases are liars and rises towards 100% as n gains small factors (a few extra bases that share a factor with n are the only ones that expose it). Miller–Rabin liars are 1.4–20% of the bases, never above the 25% bound (8911 comes closest at 20%). A base of 2 alone is not safe either: 2047 passes both tests at base 2 but is exposed by other bases (only 12% of bases are Miller–Rabin liars).

Table 17. *False-acceptance rates from 4,000 random trials per cell*

n	Fermat, 1 random base: accepted	Fermat, 5 bases	Miller–Rabin, 1 base	Miller–Rabin, 5 bases
561	56.3%	5.9%	1.4%	0.00%

1105	69.4%	16.7%	2.4%	0.00%
1729	74.8%	23.2%	9.2%	0.00%
2047	22.9%	0.1%	11.7%	0.00%

All odd composites below 100,000 (40,408 numbers): the fixed-base Fermat test (a = 2) wrongly accepted 78, a random-base Fermat test 140, and one round of Miller–Rabin only 27. Using Korselt's criterion there are exactly 16 Carmichael numbers below 100,000 (561, 1105, 1729, 2465, 2821, 6601, 8911, 10585, …) and base-2 Fermat accepts 16 of 16.

Table 18. *Recorded operations when the candidate is prime (worst case for each test)*

Bits of n	**Prime n tested**	**Trial division: divisions**	**Fermat, 10 bases: multiplications**	**Miller–Rabin, 10 bases: multiplications**	**Trial time (ms)**
12	2,689	25	140	121	0.01
16	60,139	122	250	240	0.02
20	664,319	407	300	290	0.08
24	11,611,559	1,703	350	340	0.34
28	233,639,761	7,642	420	404	1.49
32	2,906,862,457	26,957	470	458	5.45
36	63,587,087,359	126,081	590	580	25.91
40	575,450,724,599	379,291	610	600	77.02

Trial division needs about $\sqrt{n}/2$ divisions, so every 2 additional bits double the work: from 12 to 40 bits the count rises from 25 to 379,291. Fermat and Miller–Rabin need k exponentiations of about $1.5 \cdot b$ multiplications each, so their cost grows only linearly in b (here 140 to 610 multiplications for 10 bases). At 512 bits (the size of each RSA prime in a 1,024-bit modulus) trial division would need about 2^{255} divisions; Miller–Rabin needs about $10 \times 776 = 7{,}760$ multiplications.

2.6 Prime density and the number of candidates

The prime number theorem says the number of primes below x is about $x / \ln x$, so the chance that a random b-bit number is prime is about $1/(b \ln 2)$. Restricting to odd candidates doubles it, so the expected number of

odd candidates to try is **b ln 2 / 2**: about 355 for a 1,024-bit prime and 710 for a 2,048-bit prime, growing linearly with the size. Checking the theory on small ranges:

Table 19. *Prime density among odd integers*

Range	Primes among odd numbers	Observed density	Predicted 2 / ln(midpoint)
1,000 – 10,000	1,061	0.2358	0.2322
10,000 – 100,000	8,363	0.1858	0.1832
100,000 – 1,000,000	68,906	0.1531	0.1513

Table 20. *Measured candidate counts for full prime generation (sieve by 45 small primes, then Miller–Rabin with 20 rounds)*

Prime size (bits)	Primes generated	Mean odd candidates tried	Theory b ln 2 / 2	Mean time (s)	Mean modular multiplications
32	60	14.8	11.1	0.0003	1,022
64	60	25.7	22.2	0.0007	2,248
128	60	40.8	44.4	0.0020	5,207
256	60	89.4	88.7	0.0077	14,300
512	40	142.1	177.4	0.0442	38,055
1024	15	488.9	354.9	0.5453	193,407

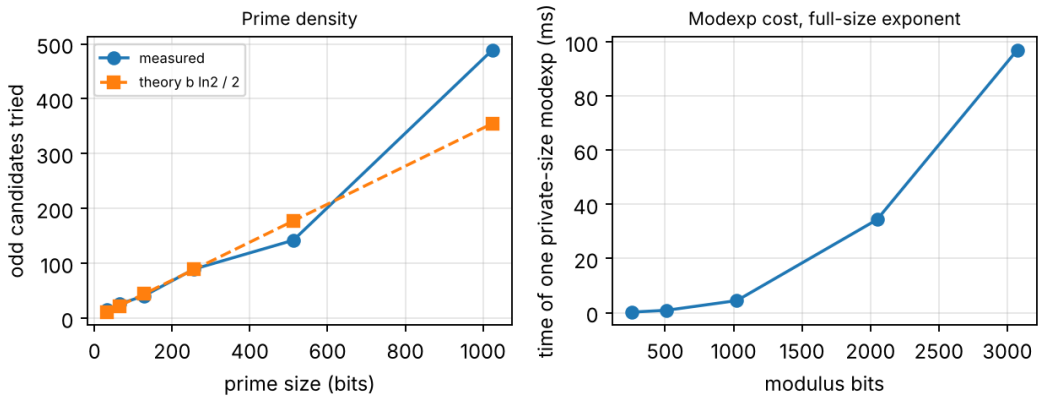


Figure 4. *Left: candidates tried versus prime size, with the theoretical line; right: cost of one full-size modular exponentiation*

The count is a geometric random variable with large variance, so single means for 512 and 1,024 bits (40 and 15 samples) deviate from theory by up to 40%, but the growth is linear as predicted. Most candidates are rejected by the small-prime sieve or by the first Miller–Rabin round, so the cost is dominated by the one or two exponentiations that a true prime must pass: that is why a 1,024-bit prime takes about half a second here but a 2,048-bit modulus (two 1,024-bit primes) takes seconds.

2.7 Scaling toward 1,024-, 2,048- and 3,072-bit moduli

Table 21. *How the engine scales*

Modulus bits	Multiplications in one full-size exponentiation	Measured time (ms)	Expected candidates for each prime	Trial division to factor/test a prime	Naive exponentiation
256	394	0.2	44	≈ 2^63	≈ 2^128
512	776	0.9	89	≈ 2^127	≈ 2^256
1024	1,520	4.5	177	≈ 2^255	≈ 2^512
2048	3,095	34.4	355	≈ 2^511	≈ 2^1024
3072	4,592	97.1	532	≈ 2^767	≈ 2^1536

Because the multiplications are on numbers of b bits, the cost of one exponentiation grows about as b^3 (b multiplications, each b^2 with schoolbook arithmetic): moving from 1,024 to 2,048 bits multiplied the measured time by 7.7 and to 3,072 bits by 22, while the number of candidates per prime grows only linearly. Key

generation therefore stays practical for moduli of 3,072 bits and beyond, whereas trial division and naive exponentiation become impossible beyond a few dozen bits. The engineering consequences: use CRT for private-key operations (about 3–4× faster), keep e small and fixed (65,537), and generate keys once per device.

2.8 Evaluation and recommendation

Table 22. *Ranking: Miller–Rabin > Fermat > trial division for large primes*

Method	Cost growth	Error behaviour	Fails on	Appropriate use
Trial division	√n: exponential in bits	Never wrong	Nothing, but infeasible above ~50 bits	Proof of concept; sieving by small primes
Fermat (random bases)	k exponentiations	One-sided; liar fraction can be near 60%	All Carmichael numbers	Teaching; never for key generation
Miller–Rabin	k exponentiations	Composite accepted with probability $\leq 4^{-k}$	No known deterministic failure for random bases	Production: sieve + Miller–Rabin (20+ rounds), optionally Lucas

Evaluation and recommendation (Q2)

Where the engine is only a proof of concept: trial division, naive exponentiation and brute-force inversion (kept for comparison); the time of our code is for comparison only.

Where limits appear: the Fermat test fails on Carmichael numbers (561, 1105, 1729 …), and a fixed base also fails on 2047; the plain Euclidean engine is safe but not constant-time, so a deployed key must use a constant-time big-number library to avoid timing leaks.

Recommendation: sieve by small primes, then at least 20 Miller–Rabin rounds with random bases (error $\leq 2^{-40}$ for a random composite, far lower in practice), check gcd(e, φ) = 1, use 65,537 as e, and use CRT for decryption.

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 marks)

Problem context. An online banking-style system for the healthcare network uses textbook RSA for short tokens. The security team must decide whether correct decryption is sufficient evidence of security. It is not: this section shows, by experiment, that textbook RSA is correct and still unsafe, and that padding and sound key generation remove the weaknesses.

3.1 RSA keys and a hand-checkable example

Key generation: choose distinct primes p, q; n = pq; φ = (p−1)(q−1); choose e with gcd(e, φ) = 1 (here 65,537, or 17 in the small example); d = e^{-1} mod φ by extended Euclid. Encryption C = M^e mod n and decryption M = C^d mod n use the student-written square-and-multiply routine. Because e·d = 1 + kφ(n), M^(ed) = M·(M^φ)^k ≡ M (mod n) for every M coprime to n, and by the Chinese remainder theorem also for the rare M that is a multiple of p or q (Section 3.3).

Worked example (verified by hand and by the program). p = 61, q = 53, n = 3,233, φ = 3,120, e = 17; extended Euclid gives d = 2,753 and e·d mod φ = 1 (17 × 2753 = 46,801 = 15 × 3120 + 1). For M = 65: the binary exponent 17 = 10001_2 gives the square/multiply trace below, ending in C = **2,790**. Decrypting, 2790^2753 mod 3233 = **65**.

Table 23. *Left-to-right square-and-multiply for 65^17 mod 3233 (5 squarings, 2 multiplications)*

Step	Next bit of e = 10001_2	Operation	Running value mod 3233
1	1	square + multiply	65
2	0	square	992
3	0	square	1232
4	0	square	1547
5	1	square + multiply	2790

3.2 Implementation and cross-checks

Implemented in rsa.py (Appendix C): key generation with Miller–Rabin primes, textbook encryption/decryption with a range check 0 ≤ M < n, CRT decryption, OAEP (SHA-256, MGF1), PKCS#1 v1.5 signatures, and the attacks used below. The code was checked against an independent library in both directions:

Table 24. *Interoperability of the student implementations with the "cryptography" library (script xcheck.py)*

Cross-check	**Result**
own OAEP ciphertext decrypts in library	Yes
library OAEP ciphertext decrypts in own code	Yes
own PKCS#1 v1.5 signature verifies in library	Yes
library signature verifies in own code	Yes
P-256 public key d*G equals library	Yes
own ECDSA signature verifies in library	Yes
library ECDSA signature verifies in own code	Yes
3DES block equals library	Yes
Blowfish block equals library	Yes

3.3 Test design: predictions written before running

Each test below was given an explicit prediction first. The "match" column compares the prediction with the observed result; all 11 predictions matched.

Table 25. *Prediction versus result for the RSA test suite*

Test	**Prediction (written first)**	**Observation**	**Match**
T1 hand example C = 65^17 mod 3233	C = 2790, M recovered = 65	C = 2790; M recovered = 65	Yes
T2 deterministic encryption	same token -> identical ciphertext	Identical ciphertexts	Yes
T3 4-digit token dictionary attack	recovers 4821 with <= 10,000 encryptions, no private key	Token 4821 found in 0.26 s	Yes
T4 e = 3, M^3 < n	cube root over the integers recovers M when M < n^(1/3) (about 341 bits for 1024-bit n), and fails above	Messages of 10, 101 and 341 bits recovered; 343 bits not recovered	Yes

T5 boundary 0, 1, n-1	0->0, 1->1, n-1 -> n-1 (odd e); unconcealed messages = (1+gcd(e-1,p-1))(1+gcd(e-1,q-1))	0→0, 1→1, n−1→n−1; unconcealed 25 = predicted 25	Yes
T6 invalid parameters	collisions for gcd(e,phi) != 1; failures for p = q or composite q	e = 3: 15 of 91 values reached; p = q: 110 of 121 fail; q = 9: 14 of 63 fail; guard rejects e	Yes
T7 factoring toy moduli	trial steps = (p_small - 1)/2; Fermat steps = (p+q)/2 - ceil(sqrt n) + 1 (exact)	All trial and Fermat step counts equal the formulas (Table below)	Yes
T9 two moduli sharing one prime	gcd(n1, n2) reveals the shared prime instantly; unrelated moduli give gcd = 1	Shared prime found; unrelated gcd = 1	Yes
T8 1024-bit modulus with close primes	Fermat factors it in a handful of steps; the size of n does not help	1 step, gap 992	Yes
T10 Hastad broadcast (e=3, 3 recipients)	CRT + integer cube root recovers M from public data only; OAEP defeats it	Textbook M recovered; OAEP not recovered	Yes
T11 OAEP	randomised: two encryptions differ; bit-flipped ciphertexts all rejected; wrong label rejected; max message 62 bytes for 1024-bit n	200 of 200 bit flips rejected; wrong label and long message rejected; max 62 bytes	Yes

(a) Deterministic encryption, repeated messages, small message space

Encrypting the same token twice gives identical ciphertext (yes); therefore an eavesdropper sees when a message repeats, and if the message space is small the attacker simply encrypts every candidate with the **public** key. All 10,000 four-digit tokens were encrypted in 0.26 s and the intercepted ciphertext was looked up: the recovered token was **4821** with no private key. Under OAEP, three encryptions of the same token were all different (yes) and all decrypted correctly (yes).

(b) Small message with a small exponent

Table 26. *e = 3, 1,024-bit modulus: no modular reduction takes place when M < n^(1/3) (≈ 341 bits)*

Message size (bits)	**$M^3 < n$?**	**Integer cube root of C recovers M?**
10	Yes	Yes
101	Yes	Yes
341	Yes	Yes
343	No	No

If $M^e < n$ the modular reduction never happens, so C is just M^3 as an integer and an ordinary cube root recovers M. The threshold observed (341 bits works, 343 bits fails) is exactly n^(1/3) for a 1,024-bit n. Small exponents are efficient but make short or structured messages (session keys, PINs, short tokens) readable without the private key unless padding enlarges the message to full size.

(c) Boundary messages and unconcealed messages

For n = 3233, e = 17: M = 0 → 0, M = 1 → 1, M = n−1 → 3232 (because (−1)^e = −1 for odd e). These messages are unchanged by encryption, as are other **unconcealed messages**: the number of M with $M^e \equiv M$ is (1 + gcd(e−1, p−1))(1 + gcd(e−1, q−1)); the prediction 25 equals the brute-force count 25. A message $M \geq n$ is rejected by the implementation (yes) because M and M + n would encrypt to the same ciphertext (yes).

(d) Invalid parameters

Table 27. *Invalid RSA parameters and their consequences*

Case	**Prediction**	**Observed**
e = 3, p = 7, q = 13 (gcd(e, phi) = 3)	not injective	76 messages collide; image has 15 of 91 values
e = 2 (even, gcd(e, phi) = 2)	not injective	63 collisions; image 28 values
p = q = 11 (n = 121), phi taken as (p-1)(q-1) = 100 instead of p(p-1) = 110	decryption fails for some M	110 of 121 messages fail
q = 9 not prime (n = 63), phi computed as (p-1)(q-1) = 48	decryption fails for some M	14 of 63 messages fail
key generator guard: modinv(e = 3, phi = 72)	rejects (no inverse)	rejected

Taken together: (i) e must be coprime to φ, otherwise encryption is many-to-one and no d exists (15 image values for 91 messages with e = 3, p = 7, q = 13); (ii) the two primes must be distinct, since for n = p^2 the totient is p(p−1), not $(p-1)^2$; (iii) both factors must be prime or φ is miscalculated. The key generator in this report rejects all three by construction (distinct-prime test, Miller–Rabin, and the gcd(e, φ) check that triggers re-generation).

3.4 Factoring n gives φ(n) and therefore d

The security of RSA rests on the difficulty of finding p and q from n. Once p and q are known, φ(n) = (p−1)(q−1) is one multiplication, and $d = e^{-1} \bmod \varphi$ is one extended-Euclid call: the private key is fully reconstructed. Conversely, knowing φ(n) gives p and q (as roots of $x^2 - (n - \varphi + 1)x + n$), and knowing d allows n to be factored probabilistically, so the three problems are equivalent in difficulty.

Demonstration. n = 176,664,370,995,209 (48 bits), e = 65,537. Fermat's method found p, q = 13,291,511, 13,291,519; $d = e^{-1} \bmod \varphi$ = 79,553,811,317,153, equal to the true d (yes); a captured ciphertext then decrypted to 123,456,789,012 — the original message.

3.5 Factoring attacks on weak moduli: trial division and Fermat

Trial division tests odd divisors up to √n, so it needs (p_small − 1)/2 steps; Fermat's method searches for a with $a^2 - n$ a perfect square b^2, giving n = (a−b)(a+b), starting at ⌈√n⌉: if p and q are close the very first a works. The number of Fermat steps is **exactly** (p + q)/2 − ⌈√n⌉ + 1; both formulas were used as predictions and compared with the counted steps.

Table 28. *Weak toy moduli: trial division and Fermat, predicted versus counted steps (exact matches)*

Modulus type	**Bits**	**Predicted trial steps**	**Counted trial steps**	**Predicted Fermat steps**	**Counted Fermat steps**	**Factored by**
balanced random 24+24 bits	48	5,911,115	5,911,115	67,691	67,691	both
close primes (gap 8)	48	6,645,755	6,645,755	1	1	both
small factor 3 x 40-bit prime	42	1	1	544,227,843,143	> 3,000,000	trial only
moderately close (gap 4150)	59	341,646,045	not run (> 2^24 steps)	1	1	Fermat only

The two methods are complementary: trial division is instant when a factor is small (3 × 40-bit prime: 1 step) but needs millions of steps for balanced primes, while Fermat is instant for close primes (1 step) but fails to finish for a small factor (> 3,000,000 steps; the exact requirement was 5.4×10^{11}). **Size does not help a badly generated key:** a genuine 1024-bit modulus built from two 512-bit primes that differ by only 992 was factored by Fermat in **1 step**. The extra size of the modulus gives no protection when the primes are chosen carelessly.

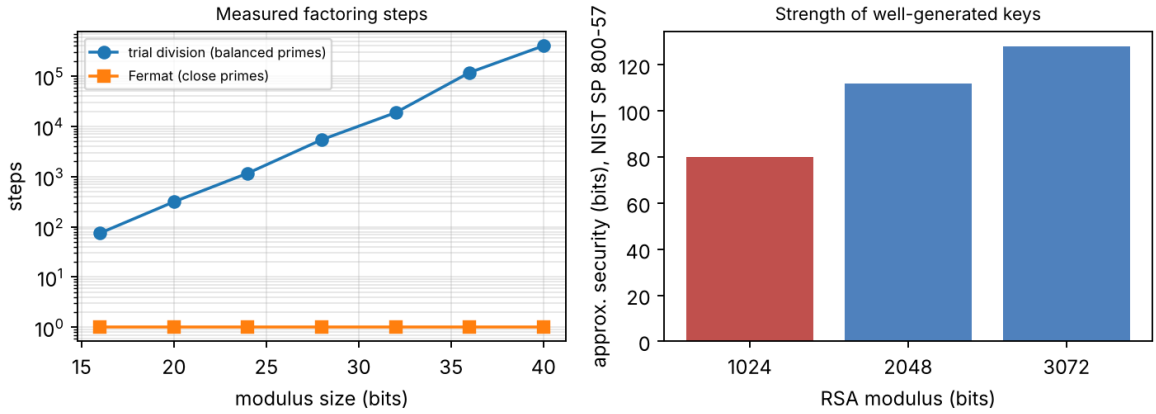


Figure 5. *Left: measured factoring steps (trial division on balanced primes grows with the modulus; Fermat on close primes stays at 1 step). Right: approximate strength of well-generated keys (NIST SP 800-57)*

- **Shared prime between two moduli.** If two keys share a prime (from a faulty random generator), gcd(n_1, n_2) returns it instantly and both keys fall: shared prime found = yes; unrelated moduli give gcd = 1. This is a documented real-world failure of low-entropy key generation.
- **Håstad broadcast attack.** The same textbook-encrypted message sent with e = 3 to three recipients (three 1,024-bit moduli; M = 400 bits) is recovered by the Chinese remainder theorem (the three ciphertexts determine M^3 modulo $n_1n_2n_3 > M^3$) followed by an integer cube root: recovered = yes. The same attack on OAEP ciphertexts recovers nothing (yes), because each recipient's ciphertext hides a different randomised value.

3.6 Why a longer key is not automatically a safer key

- **Generation quality is the real security parameter.** A 1,024-bit modulus from close primes fell in 1 step; a modulus with a small factor falls to trial division; two moduli sharing a prime fall to a gcd. The size of n only matters if the factors are random, large, and independent.

- **Size matters too, for well-generated keys.** Using NIST SP 800-57 estimates, a random 1,024-bit modulus gives about 80 bits of security (now considered inadequate), 2,048 bits about 112, and 3,072 bits about 128. The best classical algorithm, the number field sieve, is sub-exponential, so doubling the modulus does not double the security bits.
- **Other implementation attacks make long keys irrelevant:** small exponent with unpadded short message, shared prime, repeated message, timing and fault attacks on CRT decryption, and padding oracles on badly checked padding.
- **Weak-key audit.** Section 7.6 turns these findings into an automatic enrolment check (audit_rsa): it rejected 100% of seeded weak keys and raised 0 false alarms.

3.7 Padding: OAEP, randomisation and why they matter

Textbook RSA is a deterministic, malleable, multiplicative permutation: $E(M_1)\cdot E(M_2) = E(M_1M_2)$, so an attacker can transform a ciphertext into the encryption of a related message (for example multiply by 2^e to double a dose), and deterministic encryption lets him compare guesses. **OAEP** (Figure 6) fixes both problems: a fresh random seed is mixed into the whole block through two mask-generation rounds, so the encoding is randomised, every message becomes full-width (removing the small-message/small-e attack), and the integrity hash lets the decryptor detect any modification.

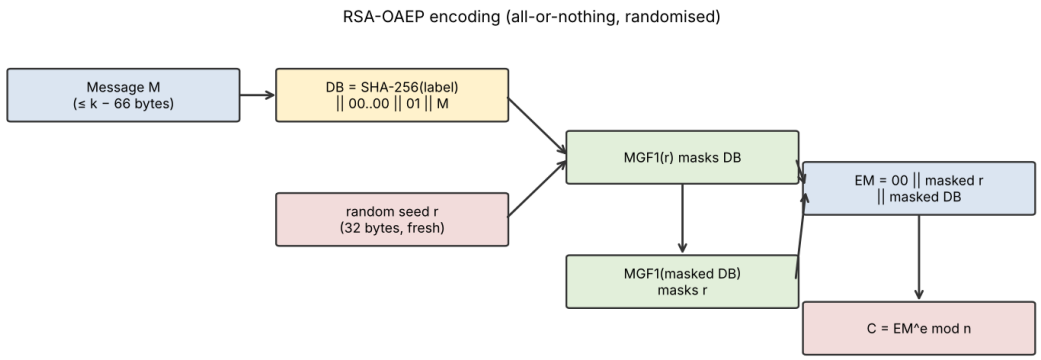


Figure 6. *OAEP encoding: a random seed and two MGF1 masks make encryption randomised and tamper-evident*

Table 29. *OAEP behaviour*

OAEP test (1,024-bit key, SHA-256)	Result

Encrypt/decrypt round trip	Yes
Maximum message length (k − 2·hLen − 2)	62 bytes
Ciphertext length	128 bytes (textbook: also 128)
One random bit flipped in the ciphertext → decryption rejected	200 of 200
Wrong label → rejected	Yes
Message longer than 62 bytes → rejected	Yes
Two encryptions of one message differ	Yes
Håstad broadcast attack succeeds	No

Two engineering notes. First, **error handling**: the implementation returns a single generic error for every failure, because distinguishing "bad padding" from "bad hash" lets a chosen-ciphertext attacker (Bleichenbacher/Manger) decrypt a message with a few thousand queries. Second, a 512-bit modulus cannot even hold an OAEP block with SHA-256 (it needs at least 528 bits), which is another reason legacy 512-bit RSA is unusable. RSA-OAEP is for short items (keys and tokens); bulk data is handled by the hybrid construction of Section 6.

Evaluation (Q3)

Textbook RSA is correct but **not** safe: it is deterministic (4,821 recovered from a 10,000-entry dictionary in 0.26 s), malleable, leaks small messages ($e = 3$) and fails completely for weakly generated moduli (1 Fermat step for a 1,024-bit key with close primes).

Correct decryption and key length are therefore not evidence of security. The required controls are: strong random distinct primes, $\gcd(e, \varphi) = 1$, $e = 65{,}537$, OAEP (or another randomised padding), a single generic decryption error, and an automatic weak-key audit at enrolment.

Q4 – Diffie–Hellman, MITM, Authentication and Key Management (7 marks)

Problem context. Two clinics must establish a secret key over a network that an attacker can fully observe, without a pre-shared secret. The task is to derive and test Diffie–Hellman (DH), show why unauthenticated DH is vulnerable to a man-in-the-middle (MITM), and design authentication and key-management controls.

4.1 Derivation of the Diffie–Hellman exchange

Public parameters: a prime p and a generator g of a large subgroup of order q in Z_p^*. Alice picks a secret a and publishes A = g^a mod p; Bob picks b and publishes B = g^b mod p. Each side computes K = B^a = (g^b)^a = g^(ab) = (g^a)^b = A^b (mod p), which is equal because exponents multiply commutatively. An eavesdropper sees p, g, A, B and would have to compute g^(ab) — the computational Diffie–Hellman problem — which is at least as easy as the discrete-logarithm problem (find a from A). The raw group element is never used directly; both sides hash it through a key-derivation function (HKDF-SHA256) together with a context string to obtain the symmetric key.

```
DH-EXCHANGE(p, q, g):                    VALIDATE(y):  # every received public value
  Alice: a <- random in [2, q-1]; A <- g^a mod p      return 1 < y < p-1  and  y^q mod p = 1
  Bob  : b <- random in [2, q-1]; B <- g^b mod p
  Alice -> Bob: A      Bob -> Alice: B          SESSION-KEY(K, context):
  Alice: if not VALIDATE(B): abort                 return HKDF-SHA256(K, context)
  Bob  : if not VALIDATE(A): abort
  Alice: K <- B^a mod p ;  Bob: K <- A^b mod p
```

4.2 Hand-checkable example and a larger test configuration

Toy values (p = 23, g = 5). a = 6, b = 15: A = 5^6 mod 23 = **8**, B = 5^{15} mod 23 = **19**; Alice computes 19^6 mod 23 = **2**, Bob computes 8^{15} mod 23 = **2**. They agree. With p this small, Eve finds a by trying all exponents: 5^x ≡ 8 gives x = 6.

Larger configuration. A 64-bit safe prime p = 2q + 1 was generated (p = 12,368,844,221,164,651,439; q = 6,184,422,110,582,325,719, also prime) with g = 693,578,628,178,437,067 of order q. A = 4,392,567,835,750,509,621; B = 9,003,666,665,822,369,926; both sides obtained the same K = 3,859,699,690,021,916,781 (yes); HKDF output (session key) = 2114c89ae258a30a23a393c67a729697….

Using a safe prime means the only proper subgroups have orders 1, 2 and q, which blocks small-subgroup attacks (Table 31).

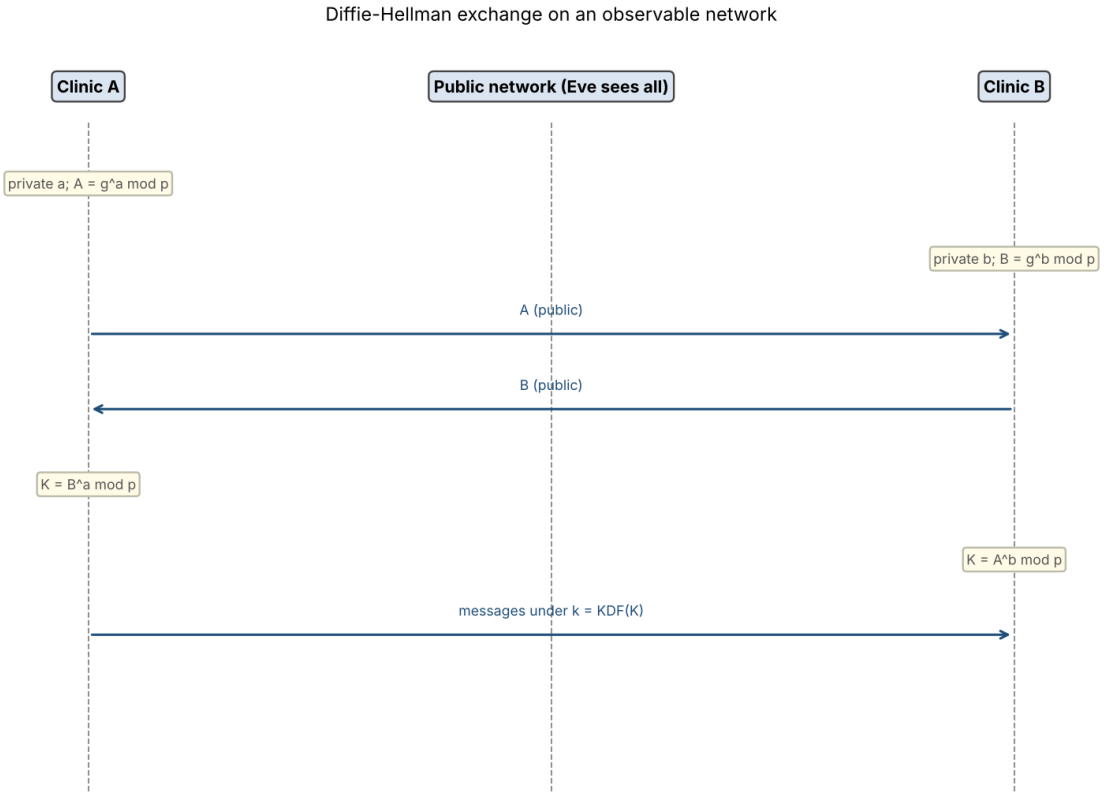


Figure 7. *Diffie–Hellman message flow on an observable network*

4.3 Cost of the legitimate parties versus the attacker

Table 30. *Discrete-log attack cost versus legitimate cost (safe primes, exponent in the upper half of [1, q])*

Bits of p	Legitimate g^x: multiplications	Legitimate time (µs)	Brute-force log: steps	Brute-force time (s)	Baby-step giant-step: steps	BSGS time (s)
12	15	3.6	869	0.000	60	0.0000
16	21	5.6	19,906	0.003	282	0.0000
20	27	6.5	358,978	0.051	1,215	0.0002

24	35	8.0	6,179,740	0.968	4,972	0.0007
26	34	8.5	17,401,494	2.806	8,504	0.0010
28	44	9.6	96,431,800	16.973	19,664	0.0032
32	45	21.9	not run	–	78,054	0.0288
36	49	24.0	not run	–	232,833	0.0643
40	59	19.2	not run	–	1,314,992	0.4450
44	61	19.8	not run	–	4,290,768	2.0331

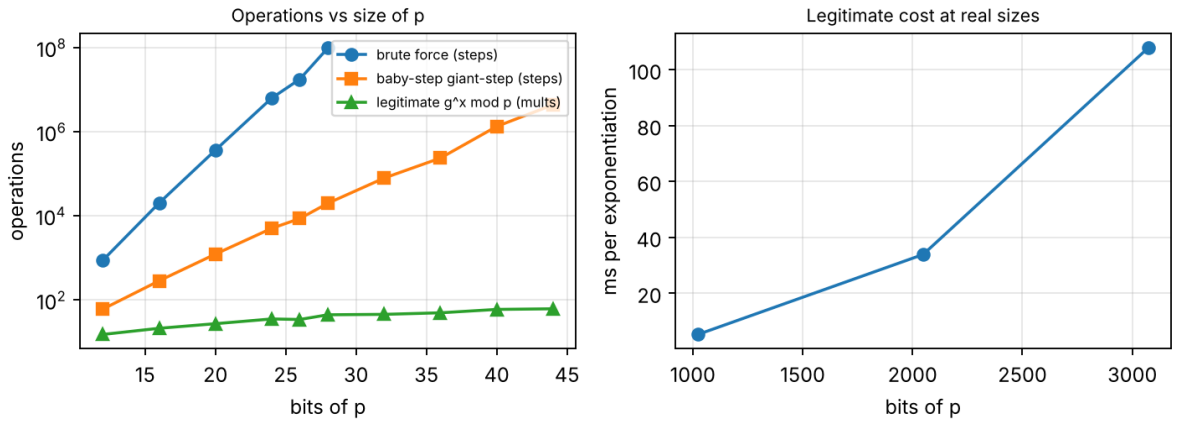


Figure 8. *Left: attack steps grow exponentially (brute force 2^n, BSGS 2^(n/2)) while legitimate work grows linearly; right: legitimate cost at real sizes*

The legitimate cost grows with the number of bits (about 1.5 × bits multiplications), whereas brute force needs about q steps and baby-step giant-step about √q steps, i.e. each attack step count roughly doubles for every 2 extra bits (brute force) or every 4 (BSGS). At real sizes the legitimate exponentiation costs 5.2 ms (1,024 bits), 33.8 ms (2,048 bits) and 108 ms (3,072 bits) in our Python. A generic attack on a 2,048-bit prime-order group would need about 2^{1024} steps; the real attack uses the number-field sieve, which is why 2,048-bit groups give about 112-bit strength. Exponents must come from a good random source: a predictable exponent reduces the work of the attacker to guessing the exponent.

4.4 Parameter validation

Table 31. *Degenerate public values force a predictable key (1 or p − 1 gives K = 1 or ±1) and must be rejected*

Received public value y	**Accepted by VALIDATE?**	**Resulting key if accepted**
y = 1	No	1
y = p − 1	No	1
y = g^a (honest)	Yes	random group element

4.5 Active attack: man-in-the-middle against unauthenticated DH

DH provides secrecy against a **passive** observer but no authentication: neither side knows whose public value it received. Mallory sits on the path, replaces A by g^{m_2} toward Bob and B by g^{m_1} toward Alice, and thereby shares one key with each victim ($K_{AM} = A^{m_1}$, $K_{BM} = B^{m_2}$). He then decrypts every message, can read or modify it, and re-encrypts it for the other side (Figure 9). The victims see correct decryption and cannot tell.

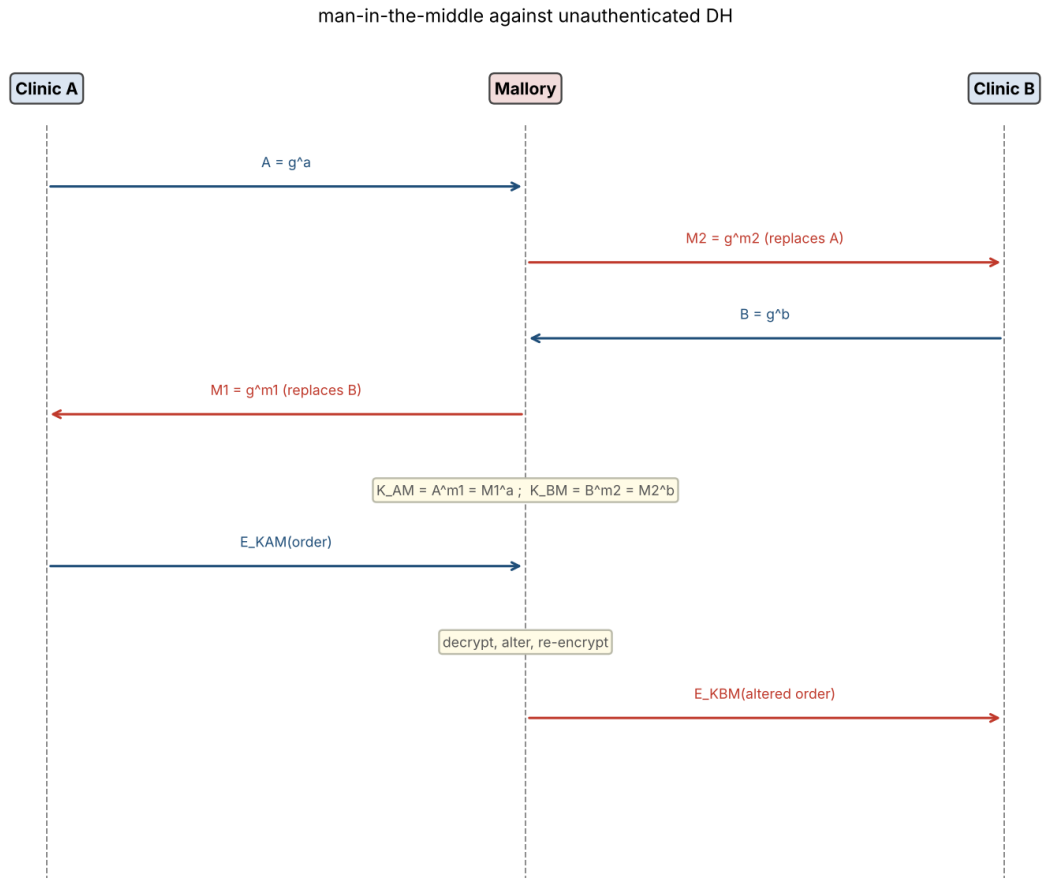


Figure 9. *MITM against unauthenticated DH: Mallory shares a different key with each clinic*

Experiment (50 independent runs). Alice sent TRANSFER PATIENT 20481 TO CLINIC 7; DOSE 5MG encrypted under her session key. In **50 of 50** runs Mallory read it exactly and forwarded a modified order (TRANSFER PATIENT 20481 TO CLINIC 7; DOSE 50MG) that Bob accepted. The two victims held different keys (K_Alice ≠ K_Bob: yes). Mallory's work was two key pairs and two key derivations, a trivial cost.

4.6 Defence: authenticated Diffie–Hellman with certificates and signatures

The remedy is to bind each public DH value to an identity. Each clinic holds a long-term signing key (RSA-2048 or ECDSA) whose public half is certified by the hospital's certificate authority (CA). In the signed exchange (Figure 10, a station-to-station-style design) the responder signs the transcript (A, B, both identities); the initiator verifies the certificate chain and the signature, and signs the transcript in turn. Three design details matter:

- **Sign both values and the identities,** not only your own value: otherwise an old signed value can be replayed in a new session or a certificate can be misbound (unknown key-share).
- **Use fresh ephemeral exponents** every session, so that a replayed message cannot match a new transcript and compromise of a long-term key reveals no past session (Section 4.7).
- **Verify the certificate** (signature by the CA, validity period, revocation) before trusting the identity.

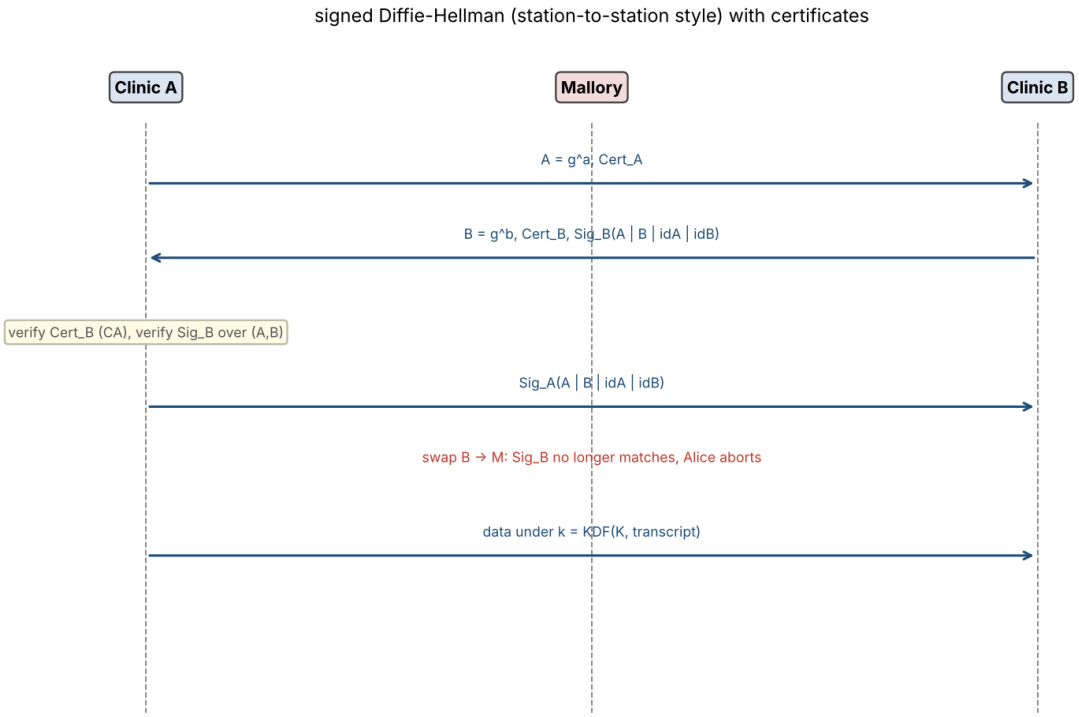


Figure 10. *Signed Diffie–Hellman: a substituted value no longer matches the signed transcript*

Table 32. *Authenticated DH versus the same attacks (RSA-1024 signatures used for speed in the test; the final design uses ECDSA)*

Test	Result	Interpretation
Honest sessions with signature check	20/20	Authentication does not break normal operation
MITM substitutes his own DH value in both directions (50 runs)	0/50 accepted	Signature covers values Mallory changed; Alice aborts

Replay of Bob's signature from an old session (50 runs)	0/50 accepted	Transcript includes the new Alice value
Certificate for "ClinicB" signed by a key other than the CA	rejected	Identity not bound to a trusted CA
Genuine CA-signed certificates	accepted	Trust chain works

4.7 Forward secrecy: ephemeral DH versus RSA key transport

If the session key is sent encrypted under the hospital's long-term RSA key, an attacker who records traffic now and steals that key later can decrypt **every** past session. If instead each session uses a fresh DH exponent that is erased after use, the stolen long-term key opens nothing. Experiment: 10 recorded sessions; the long-term RSA key was then "stolen". Key transport: **10 of 10** sessions recovered. Ephemeral DH: **0 of 10** (the attacker would need a discrete log for each session).

4.8 Key-management policy

Table 33. *Key-management policy for the clinic network*

Key / secret	**Created by and where stored**	**Lifetime and rotation**	**Compromise response**
CA signing key	Generated offline; hardware security module (HSM); never on a network host	5–10 years; ceremony with two operators	Re-root; re-issue all certificates; announce through out-of-band channel
Clinic identity (signing) key and certificate	Generated on the device (secure element / TPM); private key never exported	1–2 years; certificates short-lived (≤ 1 year)	Revoke certificate (CRL/OCSP), enrol new key, audit sessions of the period
DH/ECDH ephemeral exponent	Generated per session from the OS random source; kept in RAM only	One session; erased after key derivation	Nothing to revoke; past sessions stay safe (forward secrecy)

Session key (derived by HKDF)	Both ends from K and the transcript; not stored	Rekey after 8 files, 1 hour, or 2^28 blocks for 64-bit ciphers	New handshake
Storage key-encryption key (KEK)	Hospital HSM; wraps per-record data keys	Annual rotation; re-wrap keys, do not re-encrypt data	Rotate KEK; re-wrap; revoke access
Public parameters (p, g or curve)	Standardised (RFC 7919 FFDHE-2048 or P-256); not secret	Review at each audit; migrate when guidance changes	Replace group; versioned in the protocol suite id

Operational rules: random exponents come from the operating-system generator; every received public value is validated; logs record handshake identities and certificate serial numbers (not keys); each protocol message carries a suite identifier so that algorithms can be replaced; and the root key is rotated on a schedule published in advance.

Evaluation and recommendation (Q4)

Unauthenticated DH is a correct key-agreement primitive but is defeated by an active attacker in 50 of 50 runs; with signed transcripts and CA-certified identities the same attack succeeded 0 of 50 times, and old signatures could not be replayed.

Recommended: ephemeral DH (preferably elliptic-curve, Section 5) signed by certified identity keys, validated public values, HKDF-derived session keys, rekeying, and the policy in Table 33. This gives secrecy, authentication and forward secrecy; the cost is a certificate infrastructure and the discipline of revocation.

Q5 – ECC and Public-Key Strategy Comparison (5 marks)

Problem context. Remote medical devices need public-key security with manageable key size, computation and communication overhead. Elliptic-curve cryptography (ECC) is evaluated against RSA and finite-field Diffie–Hellman using a toy curve that can be verified by hand, a 20-bit curve whose discrete logarithm can be attacked, and the real NIST P-256 curve checked against an independent library.

5.1 Structure of elliptic-curve cryptography

An elliptic curve over a prime field is the set of points (x, y) with $y^2 = x^3 + ax + b \pmod p$, $4a^3 + 27b^2 \neq 0$, together with a point at infinity O that acts as identity. Chord-and-tangent geometry defines an addition that makes the points an abelian group: for $P_1 \neq P_2$, $\lambda = (y_2 - y_1)/(x_2 - x_1)$; for doubling, $\lambda = (3x_1^2 + a)/(2y_1)$; then $x_3 = \lambda^2 - x_1 - x_2$ and $y_3 = \lambda(x_1 - x_3) - y_1$ (all mod p, with a modular inverse for each division). Hasse's theorem bounds the number of points: $|N - (p + 1)| \leq 2\sqrt{p}$. **Scalar multiplication** kP is computed by double-and-add (the additive analogue of square-and-multiply). A public key is Q = dG for a base point G of large prime order n; recovering d from Q and G is the **elliptic-curve discrete-logarithm problem (ECDLP)**. ECDH is DH with kP in place of g^k: the shared secret is $d_aQ_B = d_ad_BG = d_BQ_A$.

5.2 Toy curve verified by hand: $y^2 = x^3 + 2x + 2$ over GF(17)

Exhaustive enumeration finds **19** points including O (Hasse allows 9.8–26.2). G = (5, 1) has order 19 (19G = O, checked), so every non-identity point is a generator. **Hand calculation of 2G:** $\lambda = (3 \cdot 5^2 + 2)/(2 \cdot 1) = 77/2 \equiv 9 \cdot 9 = 81 \equiv 13 \pmod{17}$ since $2^{-1} = 9$; $x_3 = 13^2 - 5 - 5 = 159 \equiv 6$; $y_3 = 13(5 - 6) - 1 = -14 \equiv 3$. So 2G = (6, 3), which is exactly what the program returns ((6, 3)).

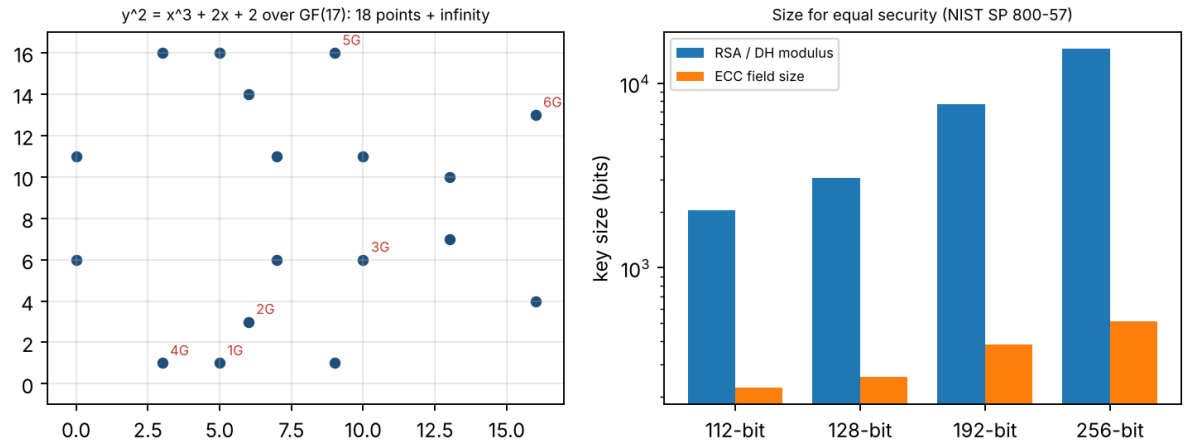


Figure 11. *Left: the 18 affine points of the toy curve with multiples of G labelled; right: key sizes needed for equal security (NIST SP 800-57)*

Table 34. *Multiples of G = (5, 1) (the 19th multiple is O, so the order is 19)*

k	1	2	3	4	5	6	7	8	9
kG	(5, 1)	(6, 3)	(10, 6)	(3, 1)	(9, 16)	(16, 13)	(0, 6)	(13, 7)	(7, 6)
k	10	11	12	13	14	15	16	17	18
kG	(7, 11)	(13, 10)	(0, 11)	(16, 4)	(9, 1)	(3, 16)	(10, 11)	(6, 14)	(5, 16)

Toy ECDH. Alice's secret d_a = 3, Bob's d_B = 7. Q_A = 3G = (10, 6), Q_B = 7G = (0, 6). Alice computes 3·Q_B = (6, 3); Bob computes 7·Q_A = (6, 3); both equal 21G = 2G (21 mod 19 = 2) = (6, 3). The shared secret is the x-coordinate (6), hashed by a KDF in practice.

5.3 Generic attack cost on a 20-bit curve

A curve with p = 627,547 (20 bits) was built by random search, its group order counted by Euler's criterion, and a base point of order n = 627,815 found. ECDH worked (yes). Using the public key, brute force and baby-step giant-step were run against the discrete logarithm.

Table 35. *ECDLP attack cost grows with the group size; the legitimate cost grows only with the bit length*

Bits of p	Points on curve	Order of G	Legit scalar	Legit time (µs)	Brute force: steps	Brute time (s)	BSGS: steps	BSGS time (s)

			mult: point ops					
12	2,839	2,839	16	26	2,469	0.003	99	0.0001
14	8,482	4,241	17	45	4,036	0.005	127	0.0002
16	44,413	44,413	20	33	31,048	0.040	358	0.0005
18	156,840	156,840	27	62	120,750	0.182	701	0.0010
20	699,835	699,835	30	77	375,159	0.575	1,285	0.0021

The pattern mirrors the finite-field DH data (Q4): legitimate work grows with the number of bits (16–30 point operations), brute force with the group order, and baby-step giant-step with its square root (1,285 steps for 627,815 elements). The crucial difference is for **large** sizes: the best known attacks on a well-chosen curve are still these generic square-root methods (Pollard's rho), so a 256-bit curve gives about 2^{128} work, whereas finite-field DH and RSA are attacked by the sub-exponential number-field sieve and therefore need 3,072-bit parameters for the same 128-bit strength.

5.4 Real curve NIST P-256: correctness and cost

Table 36. *Verification of P-256 (curve constants typed from the standard are confirmed by the library cross-check)*

Check on the student-written P-256 arithmetic	**Result**
Base point G satisfies the curve equation	Yes
n · G = O (order verified)	Yes
d·G agrees with the library public key (5 random d)	Yes
ECDH shared secret identical on both sides	Yes
ECDH shared secret equals the library's ECDH output	Yes

One 256-bit scalar multiplication needs about 255 point doublings and 126 additions, each with one modular inversion (extended Euclid), and took **12.0 ms** in our affine-coordinate Python (production code uses projective coordinates and avoids most inversions). Section 5.6 compares this with RSA and DH of equal strength.

5.5 Implementation requirements: randomness, curve and parameter validation

Secure randomness. An ECDSA signature uses a one-time nonce k; if the same k is ever used twice, the private key follows from two signatures: $k = (z_1 - z_2)/(s_1 - s_2)$, $d = (sk - z_1)/r$ (demonstrated in Section 6.3, key recovered). A weak generator in a device therefore destroys ECC security completely, so devices need a hardware or OS entropy source (or deterministic nonces, RFC 6979).

Public-key validation (invalid-curve attack). Scalar multiplication formulas use p and a but never b, so a device that does not check that a received point lies on **its** curve can be made to compute d·Q' on a different curve $y^2 = x^3 + 2x + b'$ chosen by the attacker. If that curve's group has a small subgroup of prime order r, the result reveals d mod r. Demonstration on a toy curve over p = 8,191 (secret d unknown to the attacker, group order 8,052): with only **4 queries** the attacker obtained d mod 23, 11, 3 and 19 and, by the Chinese remainder theorem, recovered d = **4,131** (success: yes; the true secret is 4,131). The same device with validation turned on rejected the malicious point (yes). Required checks: point is not O, coordinates are in range, the point satisfies the curve equation, and its order is n (for cofactor-1 curves such as P-256, nQ = O).

Table 37. *Invalid-curve attack: each malicious point leaks one residue of the secret*

Congruence	Curve parameter b' used by attacker	Prime order r of the small subgroup	Value learned: d mod r
1	2	23	14
2	3	11	6
3	6	3	0
4	10	19	8

Other requirements: use standardised curves (P-256, Curve25519) rather than self-chosen ones, reject weak or twisted parameters, use constant-time scalar multiplication to defeat timing side channels, derive keys through a KDF, and separate keys by purpose (a signing key is never reused for key agreement).

5.6 Comparison of RSA, DH and ECC

Table 38. *RSA, DH and ECC compared (sizes from the standards, times measured with the student code)*

Property (about 128-bit security unless stated)	RSA-3072	Finite-field DH-3072	ECC P-256

Public key size	384 B (+ exponent)	384 B	64 B (65 B uncompressed, 33 B compressed)
Private key size	384 B (or 5 CRT values)	32 B exponent (256-bit)	32 B
Handshake bytes on the wire (both public values)	771 B (key transport)	768 B	130 B (66 B compressed)
Private operation, measured	86 ms (modexp, 3,072-bit)	86 ms (3,072-bit modexp)	12 ms (scalar mult)
Key generation, measured	6.6 s	milliseconds (per-session exponent)	12 ms
Public/verify operation	cheap: RSA-2048 verify 0.1 ms	one exponentiation	ECDSA verify 26 ms (two scalar mults)
Best known attack	Number-field sieve (sub-exponential)	Number-field sieve	Pollard rho (√n, generic)
Main implementation risks	Padding, weak primes, e = 3	Group validation, small subgroups, MITM	Nonce reuse, invalid points, side channels
Standards for deployment	RSA-OAEP, PKCS#1 v1.5 / PSS	RFC 7919 groups	NIST P-256, Curve25519, ECDSA/EdDSA

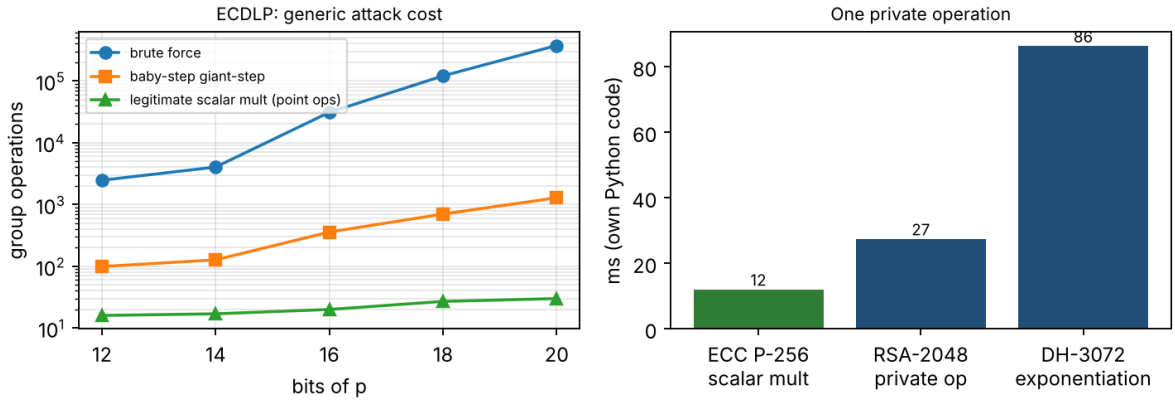


Figure 12. *Left: generic attack cost on the toy curve; right: one private operation in the student code (ECC P-256 vs RSA-2048 and 3,072-bit modular exponentiation)*

Table 39. *Equivalent strengths (NIST SP 800-57 Part 1)*

Security strength	RSA modulus (bits)	DH modulus (bits)	ECC key (bits)
112-bit	2,048	2,048	224–255
128-bit	3,072	3,072	256–383
192-bit	7,680	7,680	384–511
256-bit	15,360	15,360	512+

Reading the evidence. For the same 128-bit strength ECC needs 32-byte private and 64-byte public keys against 384 bytes for RSA/DH: a handshake of 130 bytes versus 768 bytes, which matters on low-bandwidth links. Key generation is **171 times faster** than RSA-2048 (and 542 times faster than RSA-3072) because ECC needs no prime search. A private operation is 7 times cheaper than at equal strength with RSA-3072 even with our unoptimised affine arithmetic. The advantage reverses only for RSA **verification** with e = 65,537, which is very cheap; a device that mostly verifies (for instance firmware updates) may keep RSA for that role.

Recommendation (Q5) for constrained devices

Use **ECDH (P-256 or Curve25519) for key agreement and ECDSA/EdDSA for authentication**, with validated public keys and a hardware or OS random source. Reasons shown above: 6× smaller public keys, about 7× less private-key computation than RSA-3072 or finite-field DH of equal strength, key generation in milliseconds instead of seconds, and 130-byte handshakes.

Keep RSA-OAEP/PSS only where legacy partners require it and for cheap signature verification on very small devices; never keep RSA keys below 2,048 bits. Defend the ECC choice with the controls that were shown to matter: nonce quality (Section 6.3), point validation (Table above), standard curves, and constant-time code.

Neither RSA nor ECC resists a large quantum computer (Section 6.7); the design must therefore stay crypto-agile (Section 7.7).

Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 marks)

Problem context. The healthcare network must transfer large files while guaranteeing confidentiality, integrity and sender authenticity, and must also consider future quantum computers. This section designs and tests a signature process and a hybrid-encryption scheme, then analyses the quantum threat.

6.1 Digital-signature process: hash-then-sign

Signing a large report directly with RSA is impossible (the message must be smaller than n) and would be slow. The report M is first hashed with SHA-256 to a 32-byte digest, which is padded (PKCS#1 v1.5 encoding in the implementation; PSS in new systems) and raised to the private exponent: σ = EM(h)^d mod n. The verifier recomputes the hash of the received file, applies the public exponent to the signature and compares the result with the encoding of the new hash (Figure 13). A signature proves **integrity** (any change alters the hash), **authenticity** (only the holder of d can produce σ) and supports **non-repudiation**, none of which a MAC provides.

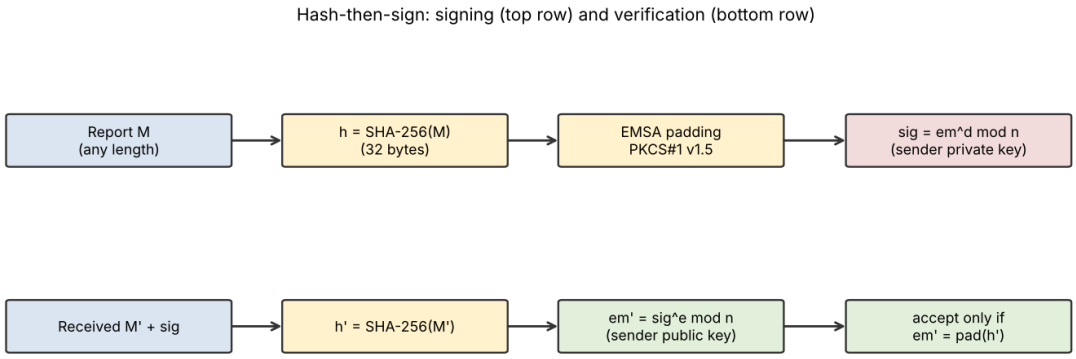


Figure 13. *Hash-then-sign: signing (top) and verification (bottom)*

Table 40. *Signature behaviour on genuine and altered data*

Verification test (2,048-bit key, 1,380-byte report)	Result	Expected	Pass
genuine report + genuine signature	accepted	accepted	Yes
one bit of the report flipped	rejected	rejected	Yes

one bit of the signature flipped	rejected	rejected	Yes
report truncated by one byte	rejected	rejected	Yes
verified under another sender's key	rejected	rejected	Yes
signature from a different message	rejected	rejected	Yes

The collision-resistant hash is what makes this secure: flipping one random bit of the report changed on average **128.6** of the 256 digest bits (min 106, max 149, 200 trials), the avalanche effect. The student-written signatures were also verified by an independent library (and library signatures verified by our code, Table in Section 3.2).

6.2 RSA, DSA and ECDSA signatures compared

Table 41. *Conceptual and measured comparison (DSA was implemented with 512-bit / 128-bit parameters purely to demonstrate the algorithm and nonce behaviour)*

Aspect	**RSA (PKCS#1 v1.5 / PSS)**	**DSA (finite field)**	**ECDSA (P-256)**
Core operation	Sign: m^d mod n; verify: s^e mod n	r = (g^k mod p) mod q; s = k^{-1}(h + xr) mod q	r = x(kG) mod n; s = k^{-1}(h + dr) mod n
Hardness assumption	Integer factoring (RSA problem)	Discrete log in Z_p*	Elliptic-curve discrete log
Randomness needed to sign	None for v1.5 (deterministic); salt for PSS	**Fresh secret nonce k per signature**	**Fresh secret nonce k per signature**
Effect of a repeated or biased nonce	Not applicable	Private key recovered (Section 6.3)	Private key recovered (Section 6.3)
Signature size (128-bit level)	384 B (RSA-3072); 256 B (RSA-2048)	64 B (3,072/256 parameters)	64 B
Public key size	256–384 B	384 B (+ parameters)	65 B
Measured sign / verify (own code)	26.8 ms / 0.15 ms (2,048-bit)	toy 512-bit run only: not comparable	12.5 ms / 25.7 ms

Best use	Certificates and firmware (cheap verify)	Legacy compliance	Constrained signers; small signatures

6.3 Why signing randomness matters: nonce reuse

For DSA/ECDSA, two signatures (r, s_1) and (r, s_2) on hashes z_1, z_2 made with the **same** k have the same r (yes). Subtracting gives $k = (z_1 - z_2)/(s_1 - s_2)$ and then $d = (s_1k - z_1)/r$. Experiment: two P-256 signatures on dose = 5 mg and dose = 50 mg using one nonce: private key recovered = **yes**; the same attack on the DSA implementation: **yes**. Medical devices with a poor random generator must therefore use deterministic nonces (RFC 6979) or EdDSA, or prefer RSA-PSS for signing.

6.4 Hybrid encryption design

Public-key operations are orders of magnitude slower than symmetric encryption, and RSA-OAEP can carry only about 190 bytes per 2,048-bit block, so the file is encrypted with a **fresh random symmetric key k** and only k is protected by public-key cryptography (Figure 14). Two variants were implemented and tested:

- **Variant A (RSA key wrap):** k is encrypted with the receiver's RSA-2048 public key using OAEP; the sender signs the whole package with RSA.
- **Variant B (elliptic-curve key agreement):** the sender generates an ephemeral ECDH key, the shared point is hashed (HKDF) into k, the ephemeral public point travels in the package, and the sender signs with ECDSA. This variant also gives forward secrecy for the key.

In both, k is expanded by HKDF into an encryption key and an independent MAC key; the payload is encrypted with the student-implemented Blowfish in CTR mode and authenticated with HMAC-SHA256 over header, wrapped key and ciphertext (**encrypt-then-MAC**); a signature covers the complete package so that the receiver also learns who sent it. (A production system would use AES-256-GCM or ChaCha20-Poly1305 in place of Blowfish-CTR + HMAC.)

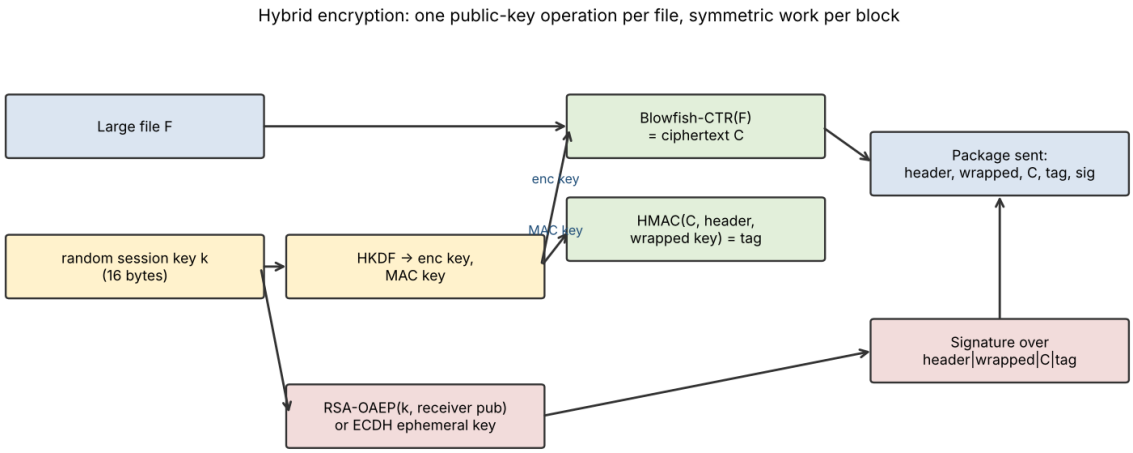


Figure 14. *Hybrid encryption package: one public-key operation protects the key; symmetric work protects the data*

6.5 Tamper tests on the file, signature and session key

Table 42. *Every tampered package was rejected (variant A: 100 trials per component; variant B: 25)*

Component altered (one random bit flipped)	**Trials**	**Variant A (RSA): rejected**	**Variant B (ECDH): rejected**	**Layer that caught it first**
Ciphertext (file body)	100	100	25 of 25	Signature (also MAC)
MAC tag	100	100	25 of 25	Signature (also MAC)
Wrapped session key / ephemeral point	100	100	25 of 25	Signature (also OAEP / point validation)
Signature	100	100	25 of 25	Signature check
Header / nonce	100	100	–	Signature (also MAC)

Table 43. *Defence in depth: each layer was tested with the other one disabled (40 trials each)*

Component altered	**MAC + OAEP only (signature check disabled)**	**Signature + OAEP only (MAC check disabled)**
ct	40 of 40	40 of 40

tag	40 of 40	40 of 40
signature	0 of 40	40 of 40
wrapped	40 of 40	40 of 40

The package is checked in the order signature → session-key unwrap → MAC, and the first failing check rejects it. Table above shows that **each layer works independently**: with the signature disabled the MAC and OAEP still reject 40 of 40 changes to the ciphertext, tag or wrapped key; with the MAC disabled the signature rejects all 40. The only change the MAC cannot see is a forged signature, which is the signature's job. Further tests: a package opened with the **wrong recipient key** is rejected (session key unwrap failed), and a **truncated** package is rejected (signature invalid). All honest packages decrypted correctly (round trip: variant A Yes, variant B Yes).

6.6 Direct RSA encryption versus hybrid encryption

Table 44. *Measured on the same machine with RSA-2048 (190 bytes of message per block)*

File size (bytes)	Direct RSA-OAEP: blocks	Direct: public-key operations (enc + dec)	Direct: encrypt + decrypt time (ms)	Direct: ciphertext bytes	Hybrid: public-key operations	Hybrid: time (ms)	Hybrid: package bytes
1,024	6	12	178	1,536	4	69	1,576
4,096	22	44	686	5,632	4	78	4,648
16,384	87	174	2,530	22,272	4	113	16,936

Table 45. *Extrapolation from the measured per-block RSA cost (29 ms per block encrypt + decrypt) and per-byte symmetric cost (2.88 µs/byte in pure Python)*

File size (bytes)	Direct: blocks	Direct: public-key operations	Direct: estimated time	Hybrid: public-key operations	Hybrid: estimated time	Direct ciphertext growth
1,000,000	5,264	10,528	153 s	4	3.0 s	1.35×

100,000,000	526,316	1,052,632	4.3 h	4	5 min	1.35×
1,000,000,000	5,263,158	10,526,316	42.5 h	4	48 min	1.35×

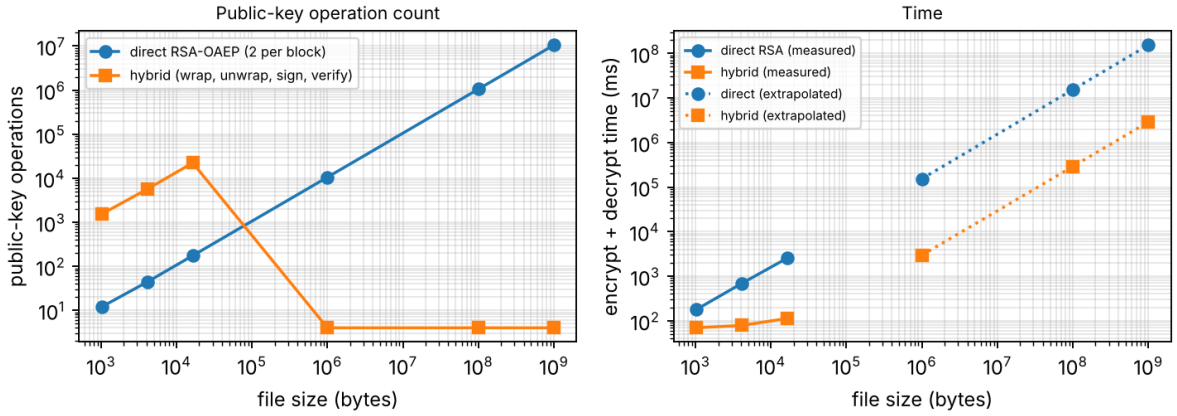


Figure 15. *Public-key operations and time versus file size: direct RSA grows linearly with the file, hybrid stays constant in public-key work*

Direct RSA needs one OAEP block per 190 bytes, so a 16 KB file needs 87 blocks and 174 public-key operations and its ciphertext grows by a factor of 1.36 (a 256-byte block per 190 bytes: +34.7%); hybrid encryption always uses 4 public-key operations (wrap, unwrap, sign, verify) and adds a fixed overhead of about 552 bytes. At 16 KB the hybrid design is 22 times faster, and the gap widens linearly: at 1 GB direct RSA would need more than 10 million public-key operations and over 43 hours in this code, against under an hour for the hybrid scheme even with our slow pure-Python cipher. (RSA decryption with the Chinese remainder theorem is 3–4 times faster and a hardware AES makes the symmetric part far faster, but neither changes the linear-versus-constant conclusion.)

6.7 Quantum threats, QKD and a migration strategy

(a) What QKD is, and what it is not

Quantum key distribution (QKD), e.g. BB84, uses the fact that measuring a quantum state disturbs it. Alice sends random bits encoded in randomly chosen bases (rectilinear or diagonal); Bob measures in randomly chosen bases; they publicly compare bases and keep the bits where the bases agreed (about half). An eavesdropper who measures in the wrong basis (half the time) randomises the result, which produces a 25%

error rate in the kept bits when every photon is intercepted. Alice and Bob compare a sample of the key; an error rate above a threshold (about 11% for secure key extraction) means abort.

Table 46. *BB84 simulation (5 runs per row); detection by sampling: 1 − (3/4)^m for m compared bits (m = 20 → 99.7%)*

Fraction of photons intercepted by Eve	**Sifted bits (of 4,000 sent)**	**Measured error rate of sifted key**	**Theory: 0.25·f + noise (1% channel noise)**
0%	1973	1.1%	1.0%
10%	1992	3.4%	3.5%
25%	1997	7.1%	7.1%
50%	1974	13.3%	13.2%
75%	1984	19.8%	19.4%
100%	1994	25.3%	25.5%

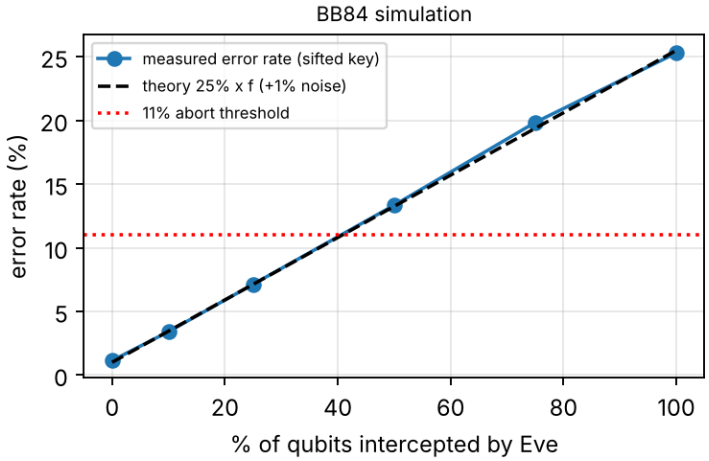


Figure 16. *Error rate rises with the interception fraction; above about 11% the key is discarded*

Limits of QKD. QKD only distributes keys; it does not authenticate the parties (the classical channel must be authenticated with a pre-shared key or digital signatures, so it cannot replace public-key signatures), it provides no signatures or non-repudiation, it needs dedicated optical links or trusted relay nodes, it has distance and rate limits, and real devices can be attacked through implementation side channels. Several national security agencies have published guidance favouring post-quantum cryptography over QKD for general use. **QKD is therefore not a replacement for the hybrid public-key system of a multi-site healthcare network.**

(b) Why a capable quantum computer threatens RSA, DH and ECC

Shor's algorithm solves integer factoring and discrete logarithms (including elliptic-curve logarithms) in polynomial time on a sufficiently large fault-tolerant quantum computer. All the schemes in this report whose security is factoring or discrete-log — RSA-OAEP key wrap, RSA signatures, finite-field DH, ECDH and ECDSA — would then be broken regardless of key length. Grover's algorithm only gives a quadratic speed-up for exhaustive search, so symmetric keys and hashes are not broken: a 128-bit key retains roughly 64-bit quantum search cost, and 256-bit keys and SHA-256/384 remain adequate. The risk is already present as **"harvest now, decrypt later"**: recorded ciphertext that must stay confidential for 20+ years (patient records) could be decrypted once a quantum computer exists, so the recorded key exchange matters today.

(c) Migration and readiness strategy

Table 47. *Quantum-readiness plan (NIST published FIPS 203, 204 and 205 in August 2024; FIPS 206 (FN-DSA) and a backup KEM, HQC, selected in March 2025, were still in draft/standardisation as of the NIST pages consulted in 2026)*

Dependency in the design	**Quantum exposure**	**Action**	**Timing**
RSA / ECDH key establishment (Variants A and B)	Broken by Shor; harvest-now-decrypt-later applies	Add a post-quantum KEM (ML-KEM, FIPS 203) in a hybrid combination with ECDH; combine both secrets in the KDF	First: start pilots now
RSA / ECDSA signatures and certificates	Broken by Shor; forgery only possible once a quantum computer exists	Move to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205) in certificates; dual-sign during the transition	Next: align with CA and vendor roadmaps
Symmetric encryption and MACs	Grover: key strength halves	Use 256-bit keys (AES-256); retire 64-bit-block ciphers	Now
Hashes (SHA-256, HKDF)	Collision/preimage margins reduced but adequate	Keep SHA-256 or move to SHA-384/512	No change needed

Long-term archived records	Exposed if wrapped by RSA/ECC keys	Re-wrap data keys under a hybrid or PQ KEK; shorten retention of old keys	Plan with data-retention policy
Protocol agility	Algorithms will change again	Suite identifier in every message (implemented in Section 7); inventory of every cryptographic use	Now

Costs to plan for: ML-KEM-768 public keys and ciphertexts are about 1,184 and 1,088 bytes and ML-DSA-65 keys and signatures about 1,952 and 3,309 bytes, many times larger than the 65-byte ECC key and 64-byte signature, which affects constrained links and certificate chains. NIST's draft transition guidance proposes deprecating RSA/ECC at the 112-bit level around 2030 and disallowing them around 2035, which sets the schedule for the plan above. The recommended approach is therefore **hybrid classical + post-quantum** key establishment (secure if either component holds) and crypto-agility; QKD may serve a niche role on dedicated links but does not appear in the plan.

6.8 Deployment decision

Table 48. *Deployment comparison*

Criterion	**Direct RSA encryption**	**Hybrid RSA-wrap (A)**	**Hybrid ECDH (B) + signed**	**Chosen**
Performance for large files	Linear public-key cost; 22× slower at 16 KB, impossible at GB	Constant public-key cost	Constant; lowest cost on small devices	B
Security	Padding OK, but no integrity; all blocks hang on one key	Confidential, authenticated; no forward secrecy	Confidential, authenticated, forward secrecy per file	B
Maintenance burden	Simple, but unusable	RSA keys, OAEP, certificates	ECC code, point validation, nonce discipline	A ≈ B

Long-term (quantum) risk	High	High (RSA wrap)	High, but replaceable by hybrid KEM slot	B + PQ slot

Deployment decision (Q6)

Deploy **hybrid encryption with ephemeral ECDH key agreement, HKDF, encrypt-then-MAC payload protection (AES-256-GCM in production) and ECDSA/EdDSA signatures by certified identity keys**, with the RSA-OAEP wrapped variant retained only for partners that cannot do ECC.

Justification: constant public-key work per file (4 operations versus 174 for a 16 KB file), 100% detection of every tampered component, forward secrecy, small keys and signatures for devices, and a suite identifier plus hybrid-KEM slot that makes the quantum migration an upgrade instead of a redesign.

Residual risks accepted: nonce discipline in ECDSA signers (mitigated by deterministic nonces), certificate-revocation latency, and the pre-quantum exposure of today's key exchange until the hybrid KEM is rolled out.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (11 marks)

Problem context. The healthcare network has outgrown its first design: more clinics, larger files, more attacker capability. This section follows the required sequence Predict → Design → Implement → Measure → Analyse → Attack → Evaluate → Diagnose → Redesign → Validate, using the evidence of Q1–Q6 and new experiments.

7.1 A. Diagnose the bottleneck or weakness (2 marks)

The original design (the baseline used in this assignment) protects every file by public-key encryption block by block (RSA-OAEP, 190 bytes of message per 2,048-bit block), establishes session keys with unauthenticated Diffie–Hellman, and trusts whatever RSA keys the clinics present.

Identified limitation

Computational limitation (primary): the cost of protecting a file grows linearly with its size because every 190 bytes needs a private-key modular exponentiation (about 3,000 multiplications of 2,048-bit numbers, 29 ms in our code). **Security limitations that compound it:** no authentication of key agreement (MITM) and keys whose long-term compromise exposes all history.

Table 49. *Evidence supporting the diagnosis*

Evidence from earlier questions	Observation	Where
Operation counts (Q2, Q6)	One private operation ≈ 1,520 multiplications at 2,048 bits (3,095); a 16 KB file needs 174 public-key operations, a 1 MB file 10,528	Q2 Table, Q6 Section 6.6
Measured time (Q6)	Direct RSA takes 2.5 s for 16 KB versus 0.11 s hybrid (22×)	Q6 Section 6.6
Size growth (Q3, Q6)	Ciphertext 1.35× the plaintext under direct RSA-OAEP (+34.7%); hybrid adds a fixed 548 bytes	Q6 Section 6.6
Key agreement (Q4)	Unauthenticated DH: MITM succeeded 50 of 50; RSA key transport exposed 10 of 10 past sessions after key theft	Q4 Sections 4.5, 4.7
Key quality (Q3)	A 1,024-bit modulus with close primes fell in 1 Fermat step; shared primes fall to a gcd	Q3 Section 3.5

Scale (this section)	For 25 clinics × 100 files/day × 1 MB the baseline needs 26,320,000 public-key operations per day (≈ 106 hours of computation in our code)	Table 53

The property at risk is therefore **availability and scalability of confidentiality** (the baseline cannot protect large files in a practical time) together with **authenticated key establishment and forward secrecy**. The root cause is architectural: public-key cryptography is being used for bulk data, and key agreement is not bound to identities. Increasing the key size would make both worse (Q2: cost grows about as the cube of the modulus size).

7.2 B. Define the design objective (1 mark)

Measurable objective

Protect clinical files of any size **with a constant number of public-key operations per session**, and authenticate every key exchange, such that, in testing: (1) public-key operations per clinic stay independent of file size and number of files (≤ 14 scalar multiplications per session, 0 per file); (2) MITM success 0 of 50; (3) past sessions exposed after theft of long-term keys 0 of 10; (4) every tampered, replayed or reordered package rejected (100%); (5) 100% of seeded weak RSA keys rejected with 0 false alarms; (6) fixed per-file overhead of ≤ 64 bytes; (7) algorithms replaceable without redesign (suite identifier and hybrid-KEM slot).

7.3 C. Design the revised architecture (3 marks)

The revised design combines the components analysed in Q1–Q6: a certified identity per clinic, a signed ephemeral elliptic-curve key agreement per session (Q4, Q5), HKDF key derivation, a separate key per file with encrypt-then-MAC payload protection (Q1, Q6), a strictly increasing per-session file counter against replay, rekeying, an enrolment audit for legacy RSA keys (Q3), and a suite identifier with a second-secret slot for post-quantum hybrid key establishment (Q6).

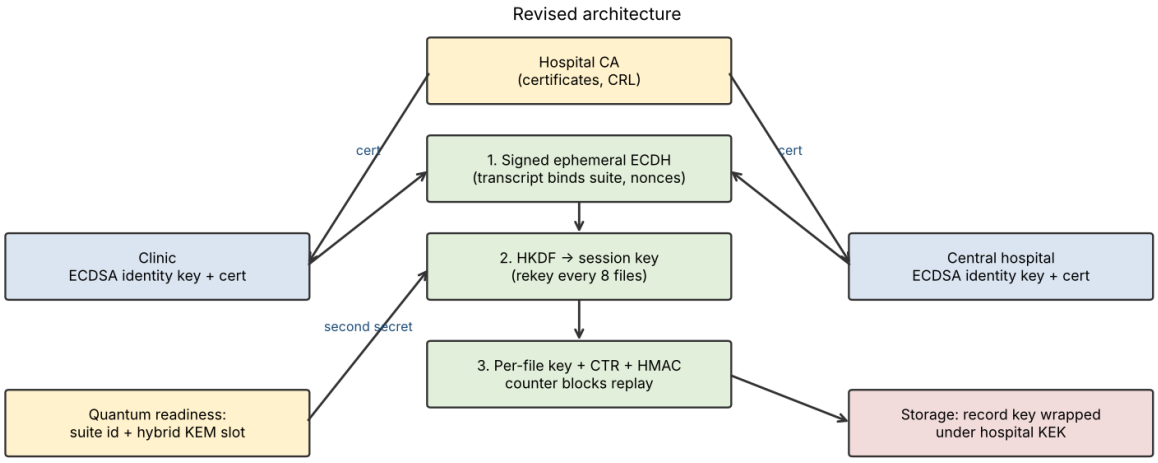


Figure 17. *Revised architecture: certified identities, signed ephemeral ECDH, per-file keys with MAC, rekeying, and a quantum-readiness slot*

Table 50. *Revised design: components, reasons and keys*

Component	**Choice**	**Mathematical / security reason (evidence)**	**Key or state needed**
Identity and authentication	ECDSA P-256 keys; certificates issued by the hospital CA	Signature over the transcript defeats MITM (Q4: 0/50); small keys and signatures (Q5)	Long-term signing key per clinic and CA key (HSM)
Key agreement	Ephemeral ECDH P-256, public values validated (not O, on curve, order n)	Forward secrecy (Q4: 0/10 exposed); invalid-curve attack blocked (Q5)	Ephemeral scalar, erased after use
Key derivation	HKDF-SHA256 over the shared point and the transcript hash; optional second secret	Binds keys to identities, nonces and suite; combiner for post-quantum slot	None stored
Bulk protection	Per-file key from the session key; Blowfish-CTR + HMAC-SHA256	Constant public-key cost (Q6); MAC stops CTR malleability (Q1);	Session key, file counter

	(AES-256-GCM in production)	unique key per file prevents nonce reuse	
Replay/ordering	Strictly increasing file counter checked by the receiver	Replay and reorder rejected (Section 7.5)	Last counter per session
Rekeying	New handshake after 8 files (or 1 hour)	Limits data per key (64-bit block, Q1) and the effect of a session-key leak	Counter and timer
Legacy RSA keys	Enrolment audit (audit_rsa) before any certificate is issued	Rejects close primes, small factors, shared primes, e = 3 (Q3)	List of enrolled moduli
Quantum readiness	Suite identifier in the transcript; optional second KEM secret combined by HKDF	Downgrade attacks detected; secure if either secret holds (Section 7.7)	Suite list; KEM keys when deployed

Trade-offs. The redesign needs a certificate authority and revocation process, adds a handshake round (about 0.38 s per session in our Python, microseconds in hardware), and requires per-session state (counter and timer). It also depends on ECDSA nonce quality (deterministic nonces in production). These costs are accepted because they are one-off per session, while the baseline's cost recurs in every block.

7.4 D. Predictions made before implementation (1 mark)

Workload used for the test: 4 clinics × 3 files × 16 KB. Predictions were computed from the per-operation costs measured in Q5 and Q6 **before** the redesign was run: 2,088 public-key operations and about 30 s for the baseline (87 blocks × 2 operations per file); 56 scalar multiplications and about 1.8 s for the redesign (14 per handshake: 2 key generations + 2 ECDH + 2 signatures + 4 verifications × 2); MITM success 0; past sessions exposed 0; weak-key audit 100% detection with 0 false alarms.

7.5 E. Implement and validate (2 marks)

Table 51. *Prediction versus measurement for the performance workload*

Quantity (4 clinics × 3 files × 16 KB)	**Predicted**	**Measured**	**Comment**
Public-key operations, baseline	2,088	2,088	Exact (counted)
Time, baseline (s)	30.4	41.9	Measured time higher (machine load); same order
Scalar multiplications, redesign	56	56	Exact (counted)
Time, redesign (s)	1.81	2.65	Measured time higher (machine load); same order
Speed-up of the redesign	17×	16×	Prediction within a factor 1.1
Per-file overhead (bytes)	≤ 64	42	vs 5,888 extra bytes per file in the baseline
All files decrypt correctly	yes	Yes / Yes	Baseline / redesign

Counted operations match the predictions exactly (2,088 and 56). Absolute times were about 40% above the predictions because the cloud machine was busier than when the unit costs were measured; the **ratio** (baseline/redesign = 16×) was close to the predicted 17×. A handshake took 0.38 s per clinic in our Python implementation (14 scalar multiplications); after it, the 3 files per clinic cost no further public-key operations.

Security validation

Table 52. *Attack results against the baseline and the redesign*

Attack or fault	**Baseline (original design)**	**Redesigned system**	**Evidence**
Man-in-the-middle replaces both ephemeral values (50 runs)	50 of 50 succeed (Q4)	0 of 50 succeed; all 50 aborted: A rejects B signature (transcript mismatch)	Signed transcript
Certificate not issued by the hospital CA	Not checked	Rejected (A rejects B certificate or key)	Certificate check

Downgrade: attacker removes suite 2 from the offered list (20 runs)	No suite negotiation	0 of 20 succeed; 20 aborted	Suite list inside the signed transcript
Past sessions after theft of both long-term keys (10 sessions)	10 of 10 exposed (RSA key transport)	0 of 10 exposed	Ephemeral ECDH
One random bit flipped in a package (100 runs)	Direct RSA-OAEP detects only if the block fails	100/100 rejected	HMAC
Replay of an old file package	Not detected	Rejected: replayed or reordered package	Strictly increasing counter
Reordered packages	Not detected	Rejected: replayed or reordered package	Counter

Functional behaviour. File round trips for sizes 0, 1, 7, 8, 1,000 and 65,536 bytes all matched their SHA-256 digests (yes). Over 20 files the session rekeyed 2 times (after 8 and 16 files) and "old session key cannot open new files", so a leaked old session key does not open later files.



Figure 18. *Baseline versus redesign on the same workload: public-key operations, execution time and security events*

Table 53. *Extrapolation of the daily workload (from measured unit costs)*

Network scale: 25 clinics × 100 files/day × 1 MB	**Baseline (direct RSA-OAEP)**	**Redesign**

Public-key operations per day	26,320,000	350 scalar multiplications (one handshake per clinic)
Computation per day in our Python code	≈ 106 hours	≈ 4 hours, almost all of it the slow pure-Python cipher (negligible with AES hardware)
Long-term private-key uses per day	13,160,000 decryptions	25 signatures

7.6 Enrolment gate for legacy RSA keys

Table 54. *Weak-key audit: all 14 seeded weak keys detected; none of the 20 good keys flagged*

Seeded weak key type	Keys tested	Rejected by audit_rsa
close primes	4	4
small factor	4	4
shared prime	2	2
e = 3	2	2
n = p^2	2	2
Good random 1,024-bit keys (control)	20	0 (false alarms)

The audit rejects a public key if e < 65,537, if n has a prime factor below 50, if Fermat's method succeeds within 100,000 steps (close primes), if n is a perfect square, or if it shares a prime with any other enrolled modulus (pairwise gcd). It costs microseconds per key and removes the failure mode in which a 1,024-bit key is factored in one step (Q3).

7.7 D. Quantum-readiness integration (1 mark)

What is at risk. The ECDH key agreement and the ECDSA signatures are broken by Shor's algorithm (Section 6.7); the Blowfish/HMAC (AES-256 in production) layer and SHA-256 are not. The practical danger is recorded traffic today decrypted later, so key establishment must change first, signatures second.

Implemented hook. The key derivation accepts a second secret and the offered suites are part of the signed transcript. Test (with a random 32-byte value standing in for an ML-KEM shared secret — **no ML-KEM was implemented**, the stand-in only demonstrates the combiner): the key derived from both secrets differs from the key derived from the ECDH secret alone (yes); an attacker who has broken the ECDH part (simulated by

giving him the exact shared point) but not the second secret **cannot** open a file (opens: no); the legitimate parties can (yes). The downgrade test in Table above (0 of 20) shows that an active attacker cannot strip the post-quantum suite. A deployment would replace the stand-in by an ML-KEM-768 encapsulation and later add ML-DSA certificates (Table in Section 6.7).

Limits of this quantum-readiness claim

The redesign is **crypto-agile**, not post-quantum: until an ML-KEM implementation is added and tested, recorded sessions remain exposed to a future quantum adversary. QKD is not part of the plan (Section 6.7).

7.8 F. Defend the final design (2 marks)

Table 55. *Alternatives considered and the defence of the final design*

Alternative	Performance	Security	Deployment and maintenance	Verdict
1. Baseline: direct RSA-OAEP per block + plain DH	29 ms per 190 bytes; 106 h/day at network scale	MITM 50/50; no integrity; no forward secrecy	Simple code but unusable for large files	Rejected
2. Hybrid with RSA key transport (Variant A)	Constant public-key cost (4 operations)	Confidential and authenticated; **no forward secrecy**; RSA keys large	Familiar; slow key generation (2.1 s RSA-2048 vs 12 ms)	Acceptable for legacy partners only
3. Unauthenticated ephemeral DH + symmetric	Cheapest	Defeated by an active attacker (50/50)	No certificates needed	Rejected
4. Chosen: certified, signed ephemeral ECDH + per-file keys + MAC + counters + audit	56 scalar mults for 4 clinics; 0 per file; 42 s → 2.6 s	MITM 0/50; 0/10 exposed; 100% tamper/replay detection	Needs CA and revocation; ECDSA nonce care; suite id	**Selected**

5. PQ-only (ML-KEM + ML-DSA) now	Larger keys (≈ 1.2 KB, 3.3 KB signatures)	Resists quantum; younger cryptanalysis record	Tooling and certificate ecosystem still maturing	Next stage via the hybrid slot

Residual risks

Table 56. *Residual risks and mitigations*

Residual risk	Why it remains	Mitigation
ECDSA nonce failure in a device with a poor random source	Repeated k reveals the private key (Q6, key recovered)	Deterministic nonces (RFC 6979) or EdDSA; hardware RNG; certificate short lifetimes
Compromise of a clinic's long-term key	Attacker can impersonate that clinic until revoked	Short-lived certificates, OCSP/CRL, device secure element; forward secrecy protects past sessions
CA compromise	Attacker can issue valid identities	Offline HSM root, dual control, transparency logs
Pure-Python Blowfish-CTR + HMAC used for the experiment	64-bit block; not a production cipher	Replace with AES-256-GCM / ChaCha20-Poly1305 (interface identical)
Harvest-now-decrypt-later on today's ECDH	Shor's algorithm	Roll out hybrid ML-KEM; shorten retention of recorded traffic
Side channels (timing) in the pure-Python code	Variable-time arithmetic	Use constant-time library implementations in deployment
Operational errors (expired certificates, clock drift)	Human and process factors	Monitoring and automated renewal

Conclusion (Q7)

The diagnosis (public-key encryption of bulk data plus unauthenticated, non-forward-secret key agreement) was confirmed by measurement and removed by the redesign: public-key work fell from 2,088 operations

to 56 for the test workload (and from millions to hundreds per day at network scale), the time fell about 16×, MITM and replay attacks failed in every run, forward secrecy held in 10 of 10 sessions, and every tampered package was rejected. The final design is explained by the mathematics established earlier — square-and-multiply cost (Q2), the weaknesses of unpadded and badly generated RSA (Q3), the difficulty of discrete logarithms and the need to authenticate (Q4), the efficiency of ECC (Q5), and hybrid signed encryption (Q6) — and it leaves a defined path to post-quantum security.

Requirement Coverage Matrix

The table lists every "Your Challenge" bullet of the assignment sheet and where it is answered, so that the marker can check the sheet against the evidence.

Table 57. *Coverage of the assignment sheet*

Q	Requirement of the sheet	Answered in
Q1	Explain structure, block size, key characteristics and round operation of DES, 3DES, Blowfish, RC5	1.1, Table of structure
Q1	Implement at least three algorithms and verify with known vectors	1.2 (all four implemented, KATs, library cross-check)
Q1	Block versus stream ciphers and where each is appropriate	1.3
Q1	ECB, CBC, CFB, OFB, CTR: IV/nonce, pattern leakage, error propagation, parallelism	1.4
Q1	Demonstrate ECB reveals patterns; IV/nonce reuse is dangerous	1.5
Q1	Compare operation count and execution time for increasing message sizes; rank and recommend	1.6, 1.7, 1.8
Q2	GCD, extended Euclid, modular inverse, fast modular exponentiation with proof of correctness and tests	2.1, 2.2
Q2	Trial-search inverse vs extended Euclid; naive vs square-and-multiply	2.3
Q2	Euler's totient and the condition gcd(e, φ) = 1	2.1, 2.4
Q2	Compare trial division, Fermat, Miller–Rabin incl. Carmichael numbers 561, 1105, 1729; record multiplications, divisions, false acceptance	2.5
Q2	Prime density and candidate count; scaling to 1,024/2,048/3,072 bits	2.6, 2.7
Q3	Generate RSA keys; implement and verify encryption/decryption with a hand-checkable example	3.1, 3.2
Q3	Deterministic encryption, repeated messages, small-message/small-exponent, boundary messages, invalid parameters	3.3

Q3	Factoring n gives φ(n) and d; trial division and Fermat on weak moduli, close primes, small factors	3.4, 3.5
Q3	Why a longer key is not enough; why weak generation weakens RSA; padding, OAEP and randomisation	3.6, 3.7
Q4	Derive DH; pseudocode; test small and larger configuration	4.1, 4.2, 4.3
Q4	MITM demonstration; signatures/certificates defeat it; forward secrecy; key-management policy	4.5–4.8
Q5	Toy ECC key agreement; RSA/DH/ECC comparison; implementation requirements; recommendation	5.1–5.6
Q6	Hash-then-sign design; RSA/DSA/ECDSA comparison; hybrid encryption; tamper tests; direct RSA vs hybrid	6.1–6.6
Q6	QKD principle and limits; quantum threat to RSA/ECC; migration strategy; deployment decision	6.7, 6.8
Q7	A. Diagnose; B. Define objective; C. Revised design; D. Predictions and quantum readiness; E. Validate; F. Defend	7.1–7.8

References

1. NIST, FIPS 46-3 Data Encryption Standard (withdrawn 2005); NIST SP 800-67 Rev. 2, Recommendation for the Triple Data Encryption Algorithm (TDEA) Block Cipher; NIST SP 800-131A Rev. 2, Transitioning the Use of Cryptographic Algorithms and Key Lengths.
2. B. Schneier, “Description of a New Variable-Length Key, 64-Bit Block Cipher (Blowfish)”, Fast Software Encryption, 1994. R. L. Rivest, “The RC5 Encryption Algorithm”, FSE 1994/1995.
3. NIST SP 800-38A, Recommendation for Block Cipher Modes of Operation. K. Bhargavan and G. Leurent, “On the Practical (In-)Security of 64-bit Block Ciphers: Collision Attacks on HTTP over TLS and OpenVPN” (Sweet32), 2016. RFC 7465, Prohibiting RC4 Cipher Suites.
4. G. L. Miller, “Riemann’s Hypothesis and Tests for Primality”, 1976; M. O. Rabin, “Probabilistic Algorithm for Testing Primality”, 1980; W. Alford, A. Granville, C. Pomerance, “There are infinitely many Carmichael numbers”, 1994.

5. R. Rivest, A. Shamir, L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems", CACM 1978. RFC 8017, PKCS #1: RSA Cryptography Specifications v2.2. M. Bellare and P. Rogaway, "Optimal Asymmetric Encryption", 1994. J. Håstad, "Solving simultaneous modular equations of low degree", 1988. D. Boneh, "Twenty Years of Attacks on the RSA Cryptosystem", 1999.
6. W. Diffie and M. Hellman, "New Directions in Cryptography", 1976. W. Diffie, P. van Oorschot, M. Wiener, "Authentication and Authenticated Key Exchanges", 1992. RFC 5869 (HKDF); RFC 7919 (Negotiated FFDHE groups).
7. N. Koblitz, "Elliptic Curve Cryptosystems", 1987; V. Miller, "Use of Elliptic Curves in Cryptography", 1985. NIST FIPS 186-5, Digital Signature Standard (DSS). RFC 6979, Deterministic Usage of DSA and ECDSA. NIST SP 800-57 Part 1 Rev. 5, Recommendation for Key Management. A. Antipa et al., "Validation of Elliptic Curve Public Keys", 2003 (invalid-curve attacks).
8. C. H. Bennett and G. Brassard, "Quantum cryptography: public key distribution and coin tossing", 1984. P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer", 1994. L. K. Grover, "A fast quantum mechanical algorithm for database search", 1996.
9. NIST FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA), August 2024; NIST Post-Quantum Cryptography Standardization pages (FIPS 206 FN-DSA draft; HQC selected March 2025 as a backup KEM); NIST IR 8547 (draft), Transition to Post-Quantum Cryptography Standards.

Appendices – Source Code and Test Output

All code is the student's own work unless stated. The files are reproduced in full. Python 3.12; the only external packages are matplotlib/numpy (figures) and cryptography (cross-checks in xcheck.py only).

Appendix A – symmetric.py – DES, 3DES, Blowfish, RC5, modes of operation, RC4 (Q1)

symmetric.py

```
"""symmetric.py - DES, 3DES, Blowfish, RC5 and block-cipher modes, written from scratch (Q1)."""
from collections import Counter
OPS = Counter()          # global operation counter: 'blocks', 'rounds', 'sbox', 'xor_add'

def _bits(b, n): return [(b >> (n - 1 - i)) & 1 for i in range(n)]
def _int(bits):
    v = 0
    for x in bits: v = (v << 1) | x
    return v
def _perm(val, table, n):
    return _int([_bits(val, n)[t - 1] for t in table])

# ---------------------------------------------------------------- DES
IP = [58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,
      57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7]
FP = [40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,
      36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25]
E  = [32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,12,13,12,13,14,15,16,17,16,17,18,19,20,21,20,21,22,23,24,25,
      24,25,26,27,28,29,28,29,30,31,32,1]
P  = [16,7,20,21,29,12,28,17,1,15,23,26,5,18,31,10,2,8,24,14,32,27,3,9,19,13,30,6,22,11,4,25]
PC1= [57,49,41,33,25,17,9,1,58,50,42,34,26,18,10,2,59,51,43,35,27,19,11,3,60,52,44,36,
      63,55,47,39,31,23,15,7,62,54,46,38,30,22,14,6,61,53,45,37,29,21,13,5,28,20,12,4]
PC2= [14,17,11,24,1,5,3,28,15,6,21,10,23,19,12,4,26,8,16,7,27,20,13,2,
      41,52,31,37,47,55,30,40,51,45,33,48,44,49,39,56,34,53,46,42,50,36,29,32]
SHIFTS = [1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1]
```

```
SB = [
 [14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7, 0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8,
  4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0, 15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13],
 [15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10, 3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5,
  0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15, 13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9],
 [10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8, 13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1,
  13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7, 1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12],
 [7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15, 13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9,
  10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4, 3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14],
 [2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9, 14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6,
  4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14, 11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3],
 [12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11, 10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8,
  9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6, 4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13],
 [4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1, 13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6,
  1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2, 6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12],
 [13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7, 1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2,
  7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8, 2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11]]


class DES:
    block = 8
    def __init__(self, key: bytes):
        assert len(key) == 8
        k = _perm(int.from_bytes(key, 'big'), PC1, 64)
        c, d = k >> 28, k & 0xFFFFFFF
        self.sub = []
        for s in SHIFTS:
            c = ((c << s) | (c >> (28 - s))) & 0xFFFFFFF
            d = ((d << s) | (d >> (28 - s))) & 0xFFFFFFF
            self.sub.append(_perm((c << 28) | d, PC2, 56))
    def _f(self, r, k):
        x = _perm(r, E, 32) ^ k
```

```
        out = 0
        for i in range(8):
            six = (x >> (42 - 6 * i)) & 0x3F
            row = ((six >> 4) & 2) | (six & 1)
            col = (six >> 1) & 0xF
            out = (out << 4) | SB[i][row * 16 + col]
        OPS['sbox'] += 8; OPS['xor_add'] += 2
        return _perm(out, P, 32)
    def _crypt(self, block, keys):
        v = _perm(int.from_bytes(block, 'big'), IP, 64)
        l, r = v >> 32, v & 0xFFFFFFFF
        for k in keys:
            l, r = r, l ^ self._f(r, k)
            OPS['rounds'] += 1
        return _perm((r << 32) | l, FP, 64).to_bytes(8, 'big')
    def encrypt_block(self, b): OPS['blocks'] += 1; return self._crypt(b, self.sub)
    def decrypt_block(self, b): OPS['blocks'] += 1; return self._crypt(b, self.sub[::-1])

class TripleDES:
    """EDE with three independent keys (24 bytes) or two keys (16 bytes, K3 = K1)."""
    block = 8
    def __init__(self, key: bytes):
        assert len(key) in (16, 24)
        k = [key[i:i + 8] for i in range(0, len(key), 8)]
        if len(k) == 2: k.append(k[0])
        self.d1, self.d2, self.d3 = DES(k[0]), DES(k[1]), DES(k[2])
    def encrypt_block(self, b): return self.d3.encrypt_block(self.d2.decrypt_block(self.d1.encrypt_block(b)))
    def decrypt_block(self, b): return self.d1.decrypt_block(self.d2.encrypt_block(self.d3.decrypt_block(b)))

# ---------------------------------------------------------------- Blowfish
def _pi_hex_words(n_words):
    """First n_words 32-bit words of the fractional part of pi (Machin: pi = 16 atan(1/5) - 4 atan(1/239))."""
```

```
    prec = 32 * n_words + 64
    one = 1 << prec
    def atan_inv(x):
        t = one // x; s = t; x2 = x * x; n = 1; sign = -1
        while t:
            t //= x2; n += 2; s += sign * (t // n); sign = -sign
        return s
    pi = 16 * atan_inv(5) - 4 * atan_inv(239)
    frac = (pi - 3 * one) >> 64
    total = 32 * n_words
    return [(frac >> (total - 32 * (i + 1))) & 0xFFFFFFFF for i in range(n_words)]
_PI = _pi_hex_words(18 + 1024)


class Blowfish:
    block = 8
    def __init__(self, key: bytes):
        assert 4 <= len(key) <= 56
        self.P = _PI[:18][:]
        self.S = [_PI[18 + 256 * i: 18 + 256 * (i + 1)][:] for i in range(4)]
        j = 0
        for i in range(18):
            w = 0
            for _ in range(4):
                w = (w << 8) | key[j % len(key)]; j += 1
            self.P[i] ^= w
        l = r = 0
        for i in range(0, 18, 2):
            l, r = self._enc(l, r); self.P[i], self.P[i + 1] = l, r
        for s in range(4):
            for i in range(0, 256, 2):
                l, r = self._enc(l, r); self.S[s][i], self.S[s][i + 1] = l, r
```

```
    def _F(self, x):
        S = self.S
        OPS['sbox'] += 4; OPS['xor_add'] += 3
        return ((((S[0][x >> 24] + S[1][(x >> 16) & 255]) & 0xFFFFFFFF) ^ S[2][(x >> 8) & 255]) + S[3][x &
255]) & 0xFFFFFFFF
    def _enc(self, l, r):
        P = self.P
        for i in range(16):
            l ^= P[i]; r ^= self._F(l); l, r = r, l
            OPS['rounds'] += 1
        l, r = r, l
        return l ^ P[17], r ^ P[16]
    def _dec(self, l, r):
        P = self.P
        for i in range(17, 1, -1):
            l ^= P[i]; r ^= self._F(l); l, r = r, l
            OPS['rounds'] += 1
        l, r = r, l
        return l ^ P[0], r ^ P[1]
    def encrypt_block(self, b):
        OPS['blocks'] += 1
        l, r = self._enc(int.from_bytes(b[:4], 'big'), int.from_bytes(b[4:], 'big'))
        return l.to_bytes(4, 'big') + r.to_bytes(4, 'big')
    def decrypt_block(self, b):
        OPS['blocks'] += 1
        l, r = self._dec(int.from_bytes(b[:4], 'big'), int.from_bytes(b[4:], 'big'))
        return l.to_bytes(4, 'big') + r.to_bytes(4, 'big')

# ---------------------------------------------------------------- RC5-32/r/b
M32 = 0xFFFFFFFF
def _rol(x, n): n &= 31; return ((x << n) | (x >> (32 - n))) & M32 if n else x
def _ror(x, n): n &= 31; return ((x >> n) | (x << (32 - n))) & M32 if n else x
```

```
class RC5:
    block = 8
    def __init__(self, key: bytes, rounds=12):
        self.r = rounds
        t = 2 * (rounds + 1)
        c = max(1, (len(key) + 3) // 4)
        L = [0] * c
        for i in range(len(key) - 1, -1, -1):
            L[i // 4] = ((L[i // 4] << 8) + key[i]) & M32
        S = [(0xB7E15163 + i * 0x9E3779B9) & M32 for i in range(t)]
        i = j = A = B = 0
        for _ in range(3 * max(t, c)):
            A = S[i] = _rol((S[i] + A + B) & M32, 3)
            B = L[j] = _rol((L[j] + A + B) & M32, A + B)
            i = (i + 1) % t; j = (j + 1) % c
        self.S = S
    def encrypt_block(self, b):
        OPS['blocks'] += 1
        A = (int.from_bytes(b[:4], 'little') + self.S[0]) & M32
        B = (int.from_bytes(b[4:], 'little') + self.S[1]) & M32
        for i in range(1, self.r + 1):
            A = (_rol(A ^ B, B) + self.S[2 * i]) & M32
            B = (_rol(B ^ A, A) + self.S[2 * i + 1]) & M32
            OPS['rounds'] += 1; OPS['xor_add'] += 6
        return A.to_bytes(4, 'little') + B.to_bytes(4, 'little')
    def decrypt_block(self, b):
        OPS['blocks'] += 1
        A = int.from_bytes(b[:4], 'little'); B = int.from_bytes(b[4:], 'little')
        for i in range(self.r, 0, -1):
            B = _ror((B - self.S[2 * i + 1]) & M32, A) ^ A
            A = _ror((A - self.S[2 * i]) & M32, B) ^ B
```

```
        OPS['rounds'] += 1; OPS['xor_add'] += 6
      return ((A - self.S[0]) & M32).to_bytes(4, 'little') + ((B - self.S[1]) & M32).to_bytes(4, 'little')

# ------------------------------------------------------------- modes of operation
def pad(m, bs=8): n = bs - len(m) % bs; return m + bytes([n]) * n
def unpad(m):
    n = m[-1]
    if not 1 <= n <= 8 or m[-n:] != bytes([n]) * n: raise ValueError("bad padding")
    return m[:-n]
def xor(a, b): return bytes(x ^ y for x, y in zip(a, b))
def blocks(m, bs=8): return [m[i:i + bs] for i in range(0, len(m), bs)]

def ecb_enc(c, m): return b"".join(c.encrypt_block(b) for b in blocks(pad(m)))
def ecb_dec(c, x): return unpad(b"".join(c.decrypt_block(b) for b in blocks(x)))
def cbc_enc(c, iv, m):
    out, prev = [], iv
    for b in blocks(pad(m)):
        prev = c.encrypt_block(xor(b, prev)); out.append(prev)
    return b"".join(out)
def cbc_dec(c, iv, x):
    out, prev = [], iv
    for b in blocks(x):
        out.append(xor(c.decrypt_block(b), prev)); prev = b
    return unpad(b"".join(out))
def cfb_enc(c, iv, m):
    out, prev = [], iv
    for b in blocks(m):
        k = c.encrypt_block(prev); y = xor(b, k[:len(b)]); out.append(y); prev = y
    return b"".join(out)
def cfb_dec(c, iv, x):
    out, prev = [], iv
```

```
    for b in blocks(x):
        k = c.encrypt_block(prev); out.append(xor(b, k[:len(b)])); prev = b
    return b"".join(out)
def ofb(c, iv, m):                    # encrypt = decrypt
    out, state = [], iv
    for b in blocks(m):
        state = c.encrypt_block(state); out.append(xor(b, state[:len(b)]))
    return b"".join(out)
def ctr(c, nonce4, m, start=0):       # encrypt = decrypt; counter block = nonce(4) || counter(4)
    out = []
    for i, b in enumerate(blocks(m)):
        k = c.encrypt_block(nonce4 + ((start + i) & M32).to_bytes(4, 'big'))
        out.append(xor(b, k[:len(b)]))
    return b"".join(out)

# ------------------------------------------------------------- a simple stream cipher (RC4) for the block/stream
comparison
class RC4:
    def __init__(self, key):
        S = list(range(256)); j = 0
        for i in range(256):
            j = (j + S[i] + key[i % len(key)]) & 255; S[i], S[j] = S[j], S[i]
        self.S, self.i, self.j = S, 0, 0
    def crypt(self, m):
        S, i, j, out = self.S, self.i, self.j, bytearray()
        for b in m:
            i = (i + 1) & 255; j = (j + S[i]) & 255; S[i], S[j] = S[j], S[i]
            out.append(b ^ S[(S[i] + S[j]) & 255])
        self.i, self.j = i, j
        return bytes(out)
```

Appendix B – ntcore.py – number-theory engine (Q2)

ntcore.py

```
"""ntcore.py - number-theory engine written from scratch (Q2).
Every routine takes an optional Counter `c` and records basic operations:
  c['mul'] modular multiplications, c['div'] divisions/mod steps, c['loop'] loop iterations."""
import random
from collections import Counter

def gcd(a, b, c=None):
    while b:
        a, b = b, a % b
        if c is not None: c['div'] += 1; c['loop'] += 1
    return a

def egcd(a, b, c=None):
    """Iterative extended Euclid: returns (g, x, y) with a*x + b*y = g."""
    x0, y0, x1, y1 = 1, 0, 0, 1
    while b:
        q = a // b
        a, b = b, a - q * b
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1
        if c is not None: c['div'] += 1; c['loop'] += 1
    return a, x0, y0

def modinv(a, m, c=None):
    g, x, _ = egcd(a % m, m, c)
    if g != 1: raise ValueError("no inverse: gcd(a, m) != 1")
    return x % m

def modinv_trial(a, m, c=None):
```

```
    """Brute-force inverse: try every x in 1..m-1."""
    for x in range(1, m):
        if c is not None: c['mul'] += 1; c['loop'] += 1
        if (a * x) % m == 1: return x
    raise ValueError("no inverse")

def modexp(b, e, m, c=None):
    """Square-and-multiply (right-to-left)."""
    r, b = 1, b % m
    while e:
        if e & 1:
            r = (r * b) % m
            if c is not None: c['mul'] += 1
        e >>= 1
        if e:
            b = (b * b) % m
            if c is not None: c['mul'] += 1
        if c is not None: c['loop'] += 1
    return r

def modexp_naive(b, e, m, c=None):
    """Repeated multiplication: e-1 multiplications."""
    r = b % m
    for _ in range(e - 1):
        r = (r * b) % m
        if c is not None: c['mul'] += 1; c['loop'] += 1
    return r if e > 0 else 1 % m

def phi_by_factoring(n):
    r, m, p = n, n, 2
    while p * p <= m:
```

```
        if m % p == 0:
            while m % p == 0: m //= p
            r -= r // p
        p += 1
    if m > 1: r -= r // m
    return r

def is_prime_trial(n, c=None):
    if n < 2: return False
    if n < 4: return True
    if n % 2 == 0: return False
    i = 3
    while i * i <= n:
        if c is not None: c['div'] += 1; c['loop'] += 1
        if n % i == 0: return False
        i += 2
    return True

def is_prime_fermat(n, k=5, rng=random, c=None):
    """Fermat test with k random bases a (gcd(a,n)=1 required, otherwise composite)."""
    if n < 4: return n in (2, 3)
    if n % 2 == 0: return False
    for _ in range(k):
        a = rng.randrange(2, n - 1)
        if gcd(a, n) != 1: return False
        if modexp(a, n - 1, n, c) != 1: return False
    return True

def fermat_fixed_base(n, a, c=None):
    return gcd(a, n) == 1 and modexp(a, n - 1, n, c) == 1

def is_prime_mr(n, k=10, rng=random, c=None):
```

```
    if n < 4: return n in (2, 3)
    if n % 2 == 0: return False
    d, s = n - 1, 0
    while d % 2 == 0: d //= 2; s += 1
    for _ in range(k):
        a = rng.randrange(2, n - 1)
        x = modexp(a, d, n, c)
        if x in (1, n - 1): continue
        for _ in range(s - 1):
            x = (x * x) % n
            if c is not None: c['mul'] += 1
            if x == n - 1: break
        else:
            return False
    return True

SMALL = [p for p in range(3, 200) if is_prime_trial(p)]

def gen_prime(bits, rng, c=None, stats=None):
    """Random odd candidate with top bit set; screen by small primes, then Miller-Rabin."""
    tested = 0
    while True:
        cand = rng.getrandbits(bits) | (1 << (bits - 1)) | 1
        tested += 1
        if any(cand % p == 0 and cand != p for p in SMALL): continue
        if is_prime_mr(cand, 20, rng, c):
            if stats is not None: stats['candidates'] = tested
            return cand
```

Appendix C – rsa.py – RSA, OAEP, PKCS#1 v1.5 signatures, attacks (Q3, Q6)

rsa.py

```
"""rsa.py - textbook RSA, OAEP padding, PKCS#1 v1.5 signatures and attacks, written from scratch (Q3, Q6)."""
import hashlib, math
from ntcore import gcd, egcd, modinv, modexp, gen_prime, is_prime_mr

def generate(bits, rng, e=65537, close=False):
    """Return dict(n,e,d,p,q). Retries until gcd(e, phi) = 1 and p != q."""
    while True:
        p = gen_prime(bits // 2, rng)
        q = gen_prime(bits - bits // 2, rng)
        if p == q: continue
        phi = (p - 1) * (q - 1)
        if gcd(e, phi) != 1: continue
        n = p * q
        if n.bit_length() != bits: continue
        return {'n': n, 'e': e, 'd': modinv(e, phi), 'p': p, 'q': q}

def textbook_enc(m, n, e):
    if not 0 <= m < n: raise ValueError("message must satisfy 0 <= M < n")
    return modexp(m, e, n)
def textbook_dec(c, n, d): return modexp(c, d, n)
def crt_dec(c, k):
    p, q, d = k['p'], k['q'], k['d']
    mp, mq = modexp(c % p, d % (p - 1), p), modexp(c % q, d % (q - 1), q)
    h = modinv(q, p) * (mp - mq) % p
    return mq + h * q

def _xor(a, b): return bytes(x ^ y for x, y in zip(a, b))
def mgf1(seed, length):
```

```
    out, ctr = b'', 0
    while len(out) < length:
        out += hashlib.sha256(seed + ctr.to_bytes(4, 'big')).digest(); ctr += 1
    return out[:length]
H = 32
def oaep_encrypt(m, n, e, rng, label=b''):
    k = (n.bit_length() + 7) // 8
    if len(m) > k - 2 * H - 2: raise ValueError("message too long for OAEP")
    db = hashlib.sha256(label).digest() + b'\x00' * (k - len(m) - 2 * H - 2) + b'\x01' + m
    seed = rng.randbytes(H)
    mdb = _xor(db, mgf1(seed, k - H - 1))
    mseed = _xor(seed, mgf1(mdb, H))
    em = b'\x00' + mseed + mdb
    return modexp(int.from_bytes(em, 'big'), e, n).to_bytes(k, 'big')
def oaep_decrypt(c, key, label=b''):
    n, d = key['n'], key['d']; k = (n.bit_length() + 7) // 8
    err = ValueError("decryption error")          # one generic error only (avoids Manger-style oracle)
    if len(c) != k: raise err
    em = modexp(int.from_bytes(c, 'big'), d, n).to_bytes(k, 'big')
    y, mseed, mdb = em[0], em[1:1 + H], em[1 + H:]
    seed = _xor(mseed, mgf1(mdb, H)); db = _xor(mdb, mgf1(seed, k - H - 1))
    lh, rest = db[:H], db[H:]
    i = 0
    while i < len(rest) and rest[i] == 0: i += 1
    ok = (y == 0) & (lh == hashlib.sha256(label).digest()) & (i < len(rest) and rest[i] == 1)
    if not ok: raise err
    return rest[i + 1:]

DIGESTINFO_SHA256 = bytes.fromhex('3031300d060960864801650304020105000420')
def emsa_v15(msg, k):
    t = DIGESTINFO_SHA256 + hashlib.sha256(msg).digest()
```

```
    return b'\x00\x01' + b'\xff' * (k - len(t) - 3) + b'\x00' + t
def sign(msg, key):
    k = (key['n'].bit_length() + 7) // 8
    return modexp(int.from_bytes(emsa_v15(msg, k), 'big'), key['d'], key['n']).to_bytes(k, 'big')
def verify(msg, sig, n, e):
    k = (n.bit_length() + 7) // 8
    if len(sig) != k or int.from_bytes(sig, 'big') >= n: return False
    return modexp(int.from_bytes(sig, 'big'), e, n).to_bytes(k, 'big') == emsa_v15(msg, k)

# ---------------------------------------------------------- attacks
def trial_factor(n, counter=None):
    if n % 2 == 0: return 2, n // 2
    i = 3
    while i * i <= n:
        if counter is not None: counter['steps'] += 1
        if n % i == 0: return i, n // i
        i += 2
    return None
def fermat_factor(n, counter=None, max_steps=10**7):
    a = math.isqrt(n)
    if a * a < n: a += 1
    for _ in range(max_steps):
        if counter is not None: counter['steps'] += 1
        b2 = a * a - n; b = math.isqrt(b2)
        if b * b == b2: return a - b, a + b
        a += 1
    return None
def icbrt(x):
    lo, hi = 0, 1 << ((x.bit_length() + 2) // 3 + 1)
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if mid ** 3 <= x: lo = mid
```

```
        else: hi = mid - 1
    return lo
def crt(residues, moduli):
    N = math.prod(moduli); x = 0
    for r, m in zip(residues, moduli):
        Ni = N // m; x += r * Ni * modinv(Ni, m)
    return x % N
def hastad(cs, ns): return icbrt(crt(cs, ns))
def recover_d(n, e, p, q): return modinv(e, (p - 1) * (q - 1))
def next_prime(x, rng):
    x |= 1
    while not is_prime_mr(x, 25, rng): x += 2
    return x
```

Appendix D – dh.py and ecc.py – Diffie–Hellman, ECC, ECDH, ECDSA (Q4, Q5)

dh.py

```
"""dh.py - Diffie-Hellman, key derivation, symmetric channel and discrete-log attacks (Q4)."""
import hashlib, hmac, math
from ntcore import modexp, modinv, gcd, is_prime_mr, gen_prime


def safe_prime(bits, rng):
    """p = 2q + 1 with p and q both prime; returns (p, q)."""
    while True:
        q = gen_prime(bits - 1, rng)
        p = 2 * q + 1
        if is_prime_mr(p, 25, rng): return p, q
def subgroup_generator(p, q, rng):
    while True:
        h = rng.randrange(2, p - 1); g = modexp(h, 2, p)     # squares lie in the order-q subgroup
        if g != 1: return g


def keypair(p, q, g, rng):
```

```
    a = rng.randrange(2, q); return a, modexp(g, a, p)
def valid_public(y, p, q):
    """Parameter validation: 1 < y < p-1 and y has order q (so y^q = 1)."""
    return 1 < y < p - 1 and modexp(y, q, p) == 1
def shared(priv, pub, p): return modexp(pub, priv, p)

def hkdf(secret_int, info=b'', length=32):
    ikm = secret_int.to_bytes((secret_int.bit_length() + 7) // 8 or 1, 'big')
    prk = hmac.new(b'\x00' * 32, ikm, hashlib.sha256).digest()
    out, t, i = b'', b'', 1
    while len(out) < length:
        t = hmac.new(prk, t + info + bytes([i]), hashlib.sha256).digest(); out += t; i += 1
    return out[:length]

# discrete-log attacks (for measurement)
def dlog_brute(g, y, p, limit=None, counter=None):
    r = 1
    for x in range(0, (limit or p)):
        if r == y: return x
        r = r * g % p
        if counter is not None: counter['steps'] += 1
    return None
def dlog_bsgs(g, y, p, order):
    m = math.isqrt(order) + 1
    baby = {}; r = 1
    for j in range(m): baby.setdefault(r, j); r = r * g % p
    factor = modinv(modexp(g, m, p), p); gamma = y
    for i in range(m):
        if gamma in baby: return i * m + baby[gamma], m + i
        gamma = gamma * factor % p
    return None
```

ecc.py

```
"""ecc.py - elliptic-curve arithmetic over GF(p) (short Weierstrass y^2 = x^3 + a x + b), ECDH and ECDSA (Q5, Q6).
Points are (x, y) tuples; None is the point at infinity."""
import hashlib
from collections import Counter
from ntcore import modinv, modexp


class Curve:
    def __init__(self, p, a, b, G=None, n=None, name=''):
        self.p, self.a, self.b, self.G, self.n, self.name = p, a % p, b % p, G, n, name
    def on_curve(self, P):
        if P is None: return True
        x, y = P; return (y * y - (x * x * x + self.a * x + self.b)) % self.p == 0
    def neg(self, P): return None if P is None else (P[0], (-P[1]) % self.p)
    def add(self, P, Q, c=None):
        if P is None: return Q
        if Q is None: return P
        p = self.p
        if P[0] == Q[0]:
            if (P[1] + Q[1]) % p == 0: return None
            lam = (3 * P[0] * P[0] + self.a) * modinv(2 * P[1], p) % p
            if c is not None: c['dbl'] += 1; c['inv'] += 1
        else:
            lam = (Q[1] - P[1]) * modinv(Q[0] - P[0], p) % p
            if c is not None: c['add'] += 1; c['inv'] += 1
        x3 = (lam * lam - P[0] - Q[0]) % p
        return (x3, (lam * (P[0] - x3) - P[1]) % p)
    def mul(self, k, P, c=None):
        """Double-and-add."""
        R = None; Q = P
```

```
        if k < 0: k, Q = -k, self.neg(P)
        while k:
            if k & 1: R = self.add(R, Q, c)
            k >>= 1
            if k: Q = self.add(Q, Q, c)
        return R
    def order_of(self, P, N):
        """Order of P given the group order N (smallest divisor d of N with d*P = O)."""
        for d in sorted(divisors(N)):
            if self.mul(d, P) is None: return d
def divisors(n):
    ds, i = set(), 1
    while i * i <= n:
        if n % i == 0: ds.add(i); ds.add(n // i)
        i += 1
    return ds
def count_points(cv):
    """Naive point count by Euler's criterion (used only for toy sizes)."""
    p = cv.p; cnt = 1
    for x in range(p):
        r = (x * x * x + cv.a * x + cv.b) % p
        if r == 0: cnt += 1
        else: cnt += 2 if modexp(r, (p - 1) // 2, p) == 1 else 0
    return cnt
def sqrt_mod(r, p):
    """Tonelli-Shanks."""
    if r == 0: return 0
    if modexp(r, (p - 1) // 2, p) != 1: return None
    if p % 4 == 3: return modexp(r, (p + 1) // 4, p)
    q, s = p - 1, 0
    while q % 2 == 0: q //= 2; s += 1
```

```
    z = 2
    while modexp(z, (p - 1) // 2, p) != p - 1: z += 1
    m, c, t, rr = s, modexp(z, q, p), modexp(r, q, p), modexp(r, (q + 1) // 2, p)
    while t != 1:
        i, t2 = 0, t
        while t2 != 1: t2 = t2 * t2 % p; i += 1
        b = modexp(c, 1 << (m - i - 1), p); m, c, t, rr = i, b * b % p, t * b * b % p, rr * b % p
    return rr
def random_point(cv, rng):
    while True:
        x = rng.randrange(cv.p); y = sqrt_mod((x ** 3 + cv.a * x + cv.b) % cv.p, cv.p)
        if y is not None: return (x, y)

def ecdh_keypair(cv, rng):
    d = rng.randrange(1, cv.n); return d, cv.mul(d, cv.G)
def validate_public(cv, Q):
    """Reject infinity, off-curve points and points of wrong order (cofactor-1 curves)."""
    return Q is not None and 0 <= Q[0] < cv.p and 0 <= Q[1] < cv.p and cv.on_curve(Q) and cv.mul(cv.n, Q)
is None

# ECDSA
def _h(msg, n): return int.from_bytes(hashlib.sha256(msg).digest(), 'big') % n if n.bit_length() < 256 else
int.from_bytes(hashlib.sha256(msg).digest(), 'big')
def ecdsa_sign(cv, d, msg, rng, k=None):
    n = cv.n; z = _h(msg, n)
    while True:
        kk = k if k is not None else rng.randrange(1, n)
        R = cv.mul(kk, cv.G); r = R[0] % n
        if r == 0: continue
        s = modinv(kk, n) * (z + r * d) % n
        if s == 0: continue
        return r, s
```

```
def ecdsa_verify(cv, Q, msg, sig):
    n = cv.n; r, s = sig
    if not (1 <= r < n and 1 <= s < n): return False
    z = _h(msg, n); w = modinv(s, n)
    X = cv.add(cv.mul(z * w % n, cv.G), cv.mul(r * w % n, Q))
    return X is not None and X[0] % n == r


P256 = Curve(2 ** 256 - 2 ** 224 + 2 ** 192 + 2 ** 96 - 1, -3,
             0x5ac635d8aa3a93e7b3ebbd55769886bc651d06b0cc53b0f63bce3c3e27d2604b,
             (0x6b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c296,
              0x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837bf51f5),
             0xffffffff00000000ffffffffffffffffbce6faada7179e84f3b9cac2fc632551, 'P-256')
```

Appendix E – hybrid.py and redesign.py – hybrid encryption, DSA, redesigned protocol (Q6, Q7)

hybrid.py

```
"""hybrid.py - hybrid encryption + signatures (Q6, Q7).
Variant A: random session key wrapped with RSA-OAEP, bulk data with Blowfish-CTR + HMAC-SHA256
(encrypt-then-MAC), RSA signature over the package.
Variant B: ephemeral-static ECDH (P-256) -> HKDF -> same bulk layer, ECDSA signature.
(Blowfish-CTR is used because the cipher must be our own code; production systems use AES-GCM or
ChaCha20-Poly1305.)"""
import hmac, hashlib
import rsa, ecc
from dh import hkdf
from symmetric import Blowfish, ctr
MAGIC = b'HYB1'

def _bulk_keys(sk): return hkdf(int.from_bytes(sk, 'big'), b'enc', 16), hkdf(int.from_bytes(sk, 'big'), b'mac',
32)

def seal_rsa(data, sender_key, rcpt_pub, rng):
    n, e = rcpt_pub
```

```
    sk = rng.randbytes(16); ek, mk = _bulk_keys(sk)
    nonce = rng.randbytes(4); hdr = MAGIC + nonce
    wrapped = rsa.oaep_encrypt(sk, n, e, rng)
    ct = ctr(Blowfish(ek), nonce, data)
    tag = hmac.new(mk, hdr + wrapped + ct, hashlib.sha256).digest()
    body = hdr + wrapped + ct + tag
    return {'hdr': hdr, 'wrapped': wrapped, 'ct': ct, 'tag': tag, 'sig': rsa.sign(body, sender_key)}

def open_rsa(pkg, sender_pub, rcpt_key, check_sig=True, check_mac=True):
    hdr, wrapped, ct, tag = pkg['hdr'], pkg['wrapped'], pkg['ct'], pkg['tag']
    body = hdr + wrapped + ct + tag
    if check_sig and not rsa.verify(body, pkg['sig'], *sender_pub): raise ValueError('signature invalid')
    try: sk = rsa.oaep_decrypt(wrapped, rcpt_key)
    except ValueError: raise ValueError('session key unwrap failed')
    ek, mk = _bulk_keys(sk)
    if check_mac and not hmac.compare_digest(tag, hmac.new(mk, hdr + wrapped + ct,
hashlib.sha256).digest()): raise ValueError('MAC invalid')
    return ctr(Blowfish(ek), hdr[4:], ct)

def seal_ecdh(data, sender_d, rcpt_Q, rng, cv=ecc.P256):
    ed, eQ = ecc.ecdh_keypair(cv, rng)
    if not ecc.validate_public(cv, rcpt_Q): raise ValueError('bad recipient key')
    Z = cv.mul(ed, rcpt_Q)
    ep = b'\x04' + eQ[0].to_bytes(32, 'big') + eQ[1].to_bytes(32, 'big')
    sk = hkdf(Z[0], b'ecdh|' + ep, 16); ek, mk = _bulk_keys(sk)
    nonce = rng.randbytes(4); hdr = MAGIC + nonce
    ct = ctr(Blowfish(ek), nonce, data)
    tag = hmac.new(mk, hdr + ep + ct, hashlib.sha256).digest()
    sig = ecc.ecdsa_sign(cv, sender_d, hdr + ep + ct + tag, rng)
    return {'hdr': hdr, 'wrapped': ep, 'ct': ct, 'tag': tag, 'sig': sig}

def open_ecdh(pkg, sender_Q, rcpt_d, cv=ecc.P256):
```

```
    hdr, ep, ct, tag = pkg['hdr'], pkg['wrapped'], pkg['ct'], pkg['tag']
    if not ecc.ecdsa_verify(cv, sender_Q, hdr + ep + ct + tag, pkg['sig']): raise ValueError('signature invalid')
    eQ = (int.from_bytes(ep[1:33], 'big'), int.from_bytes(ep[33:], 'big'))
    if not ecc.validate_public(cv, eQ): raise ValueError('invalid ephemeral key')
    Z = cv.mul(rcpt_d, eQ)
    sk = hkdf(Z[0], b'ecdh|' + ep, 16); ek, mk = _bulk_keys(sk)
    if not hmac.compare_digest(tag, hmac.new(mk, hdr + ep + ct, hashlib.sha256).digest()): raise
ValueError('MAC invalid')
    return ctr(Blowfish(ek), hdr[4:], ct)

# direct RSA-OAEP encryption of a large file, block by block
def direct_rsa_encrypt(data, n, e, rng):
    k = (n.bit_length() + 7) // 8; cap = k - 2 * 32 - 2
    return [rsa.oaep_encrypt(data[i:i + cap], n, e, rng) for i in range(0, len(data), cap)], cap
def direct_rsa_decrypt(blocks_, key): return b''.join(rsa.oaep_decrypt(b, key) for b in blocks_)

# DSA (finite-field) - toy-size parameters
from ntcore import gen_prime, is_prime_mr, modexp, modinv
def dsa_params(rng, pbits=512, qbits=128):
    q = gen_prime(qbits, rng)
    while True:
        k = rng.getrandbits(pbits - qbits) | (1 << (pbits - qbits - 1)); k -= k % 2
        p = k * q + 1
        if p.bit_length() == pbits and is_prime_mr(p, 20, rng): break
    while True:
        h = rng.randrange(2, p - 1); g = modexp(h, (p - 1) // q, p)
        if g != 1: return p, q, g
def _z(msg, q): return int.from_bytes(hashlib.sha256(msg).digest(), 'big') >> max(0, 256 - q.bit_length())
def dsa_sign(par, x, msg, rng, k=None):
    p, q, g = par
    while True:
```

```
    kk = k if k is not None else rng.randrange(1, q)
    r = modexp(g, kk, p) % q
    if r == 0: continue
    s = modinv(kk, q) * (_z(msg, q) + x * r) % q
    if s: return r, s
def dsa_verify(par, y, msg, sig):
  p, q, g = par; r, s = sig
  if not (0 < r < q and 0 < s < q): return False
  w = modinv(s, q); u1, u2 = _z(msg, q) * w % q, r * w % q
  return (modexp(g, u1, p) * modexp(y, u2, p) % p) % q == r
```

redesign.py

```
"""redesign.py - Q7 revised strategy: certified, signed ephemeral ECDH (P-256) -> HKDF -> per-file keys,
Blowfish-CTR + HMAC-SHA256 (encrypt-then-MAC), replay counter, automatic rekeying, suite
negotiation bound into the transcript.
(Production would use AES-GCM / ChaCha20-Poly1305 in place of Blowfish-CTR + HMAC; the structure
is identical.)"""
import hashlib, hmac, math
from collections import Counter
import ecc
from ecc import P256
from dh import hkdf
from symmetric import Blowfish, ctr
from rsa import gcd
from rsa import fermat_factor

PKOPS = Counter()      # counts public-key scalar multiplications (keygen, ECDH, sign, 2 per verify)
def pt(Q): return b'\x04' + Q[0].to_bytes(32, 'big') + Q[1].to_bytes(32, 'big')
def unpt(b): return (int.from_bytes(b[1:33], 'big'), int.from_bytes(b[33:], 'big'))
SUITES = {1: 'P256-ECDHE + ECDSA + Blowfish-CTR/HMAC', 2: 'suite 1 + second (post-quantum)
secret combined in the KDF'}
```

```
class Identity:
    def __init__(self, name, rng, ca=None):
        self.name = name; self.d, self.Q = ecc.ecdh_keypair(P256, rng)
        self.cert = None
        if ca: self.cert = ca.issue(name, self.Q, rng)
    def sign(self, msg, rng): PKOPS['sign'] += 1; return ecc.ecdsa_sign(P256, self.d, msg, rng)
class CA(Identity):
    def issue(self, name, Q, rng): body = name.encode() + b'|' + pt(Q); return (name, Q, self.sign(body, rng))
def check_cert(cert, ca_Q):
    name, Q, sig = cert; PKOPS['verify'] += 1
    return ecc.validate_public(P256, Q) and ecc.ecdsa_verify(P256, ca_Q, name.encode() + b'|' + pt(Q), sig)

def transcript(suite_list, A_eph, B_eph, nA, nB, idA, idB):
    return hashlib.sha256(b'|'.join([bytes(suite_list), pt(A_eph), pt(B_eph), nA, nB, idA.encode(),
idB.encode()])).digest()

def derive(Zx, th, second=None):
    ikm = Zx.to_bytes(32, 'big') + (second or b'')
    return hkdf(int.from_bytes(ikm, 'big'), b'session|' + th, 32)

class Session:
    """One direction-agnostic session context built after the handshake."""
    def __init__(self, key, rekey_after=8):
        self.key, self.send_ctr, self.recv_ctr, self.rekey_after = key, 0, -1, rekey_after
    def needs_rekey(self): return self.send_ctr >= self.rekey_after
    def _fk(self, n): return hkdf(int.from_bytes(self.key, 'big'), b'file|%d' % n, 48)
    def send(self, data):
        n = self.send_ctr; self.send_ctr += 1; k = self._fk(n)
        ct = ctr(Blowfish(k[:16]), n.to_bytes(4, 'big'), data)
        hdr = b'F1' + n.to_bytes(8, 'big')
        return hdr + ct + hmac.new(k[16:], hdr + ct, hashlib.sha256).digest()
    def recv(self, pkg):
```

```
    hdr, ct, tag = pkg[:10], pkg[10:-32], pkg[-32:]; n = int.from_bytes(hdr[2:], 'big')
    k = self._fk(n)
    if not hmac.compare_digest(tag, hmac.new(k[16:], hdr + ct, hashlib.sha256).digest()): raise ValueError('MAC invalid')
    if n <= self.recv_ctr: raise ValueError('replayed or reordered package')
    self.recv_ctr = n
    return ctr(Blowfish(k[:16]), n.to_bytes(4, 'big'), ct)

def handshake(A, B, ca_Q, rng, suites=(1,), mitm=None, second=None, suite_tamper=None):
  """A initiates. mitm: optional function (A_eph, B_eph) -> (A_eph seen by B, B_eph seen by A). Returns (sessA, sessB) or raises ValueError."""
  ea, EA = ecc.ecdh_keypair(P256, rng); PKOPS['keygen'] += 1
  nA = rng.randbytes(16)
  eb, EB = ecc.ecdh_keypair(P256, rng); PKOPS['keygen'] += 1
  nB = rng.randbytes(16)
  EA_at_B, EB_at_A = (EA, EB) if mitm is None else mitm(EA, EB)
  suites_at_B = list(suites) if suite_tamper is None else suite_tamper(list(suites))
  # B side: validate, sign transcript as B saw it, derive key
  if not check_cert(A.cert, ca_Q) or not ecc.validate_public(P256, EA_at_B): raise ValueError('B rejects A')
  thB = transcript(suites_at_B, EA_at_B, EB, nA, nB, A.name, B.name)
  sigB = B.sign(thB, rng)
  ZB = P256.mul(eb, EA_at_B); PKOPS['ecdh'] += 1
  # A side: validate B's certificate and ephemeral key, check B's signature over transcript as A sees it
  if not check_cert(B.cert, ca_Q) or not ecc.validate_public(P256, EB_at_A): raise ValueError('A rejects B certificate or key')
  thA = transcript(list(suites), EA, EB_at_A, nA, nB, A.name, B.name)
  PKOPS['verify'] += 1
  if not ecc.ecdsa_verify(P256, B.cert[1], thA, sigB): raise ValueError('A rejects B signature (transcript mismatch)')
  sigA = A.sign(thA, rng)
  ZA = P256.mul(ea, EB_at_A); PKOPS['ecdh'] += 1
```

```
    PKOPS['verify'] += 1
    if not ecc.ecdsa_verify(P256, A.cert[1], thB, sigA) : raise ValueError('B rejects A signature (transcript
mismatch)')
    return Session(derive(ZA[0], thA, second)), Session(derive(ZB[0], thB, second))

# ---- enrolment gate for legacy RSA public keys (weak-key audit, Q3)
def audit_rsa(n, e, others=()):
    issues = []
    if e < 65537: issues.append('small exponent')
    if any(n % s == 0 for s in (3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47)): issues.append('small factor')
    c = Counter()
    if fermat_factor(n, c, max_steps=100000): issues.append('close primes')
    r = math.isqrt(n)
    if r * r == n: issues.append('perfect square')
    for o in others:
        if o != n and gcd(n, o) > 1: issues.append('shared prime')
    return issues
```

Appendix F – Experiment drivers and helpers (run_q1.py – run_q7.py, xcheck.py, seq.py, flow.py)

run_q1.py

```
"""run_q1.py - experiments for Q1 (symmetric ciphers and modes). All data are student-created toy data."""
import json, time, random, math, sys
import numpy as np
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from symmetric import *
import symmetric as S
rng = random.Random(2026)
R = {}
def ops_snapshot(): return dict(S.OPS)
def timed(f, *a):
```

```
    S.OPS.clear(); t = time.perf_counter(); out = f(*a); dt = time.perf_counter() - t
    return out, dt, dict(S.OPS)

# ---- 1. known-answer tests and round trips on student-created messages
KATS = {'DES': (DES(bytes.fromhex('133457799BBCDFF1')), '0123456789ABCDEF',
'85E813540F0AB405'),
        'Blowfish': (Blowfish(bytes(8)), '0000000000000000', '4EF997456198DD78'),
        'RC5-32/12/16': (RC5(bytes(16)), '0000000000000000', '21A5DBEE154B8F6D')}
R['kat'] = {n: {'got': c.encrypt_block(bytes.fromhex(p)).hex().upper(), 'expected': e} for n, (c, p, e) in
KATS.items()}
tdes = TripleDES(bytes.fromhex('0123456789ABCDEF23456789ABCDEF01456789ABCDEF0123'))
R['kat']['3DES (EDE, 3 keys)'] = {'got':
tdes.encrypt_block(bytes.fromhex('5468652071756663')).hex().upper(), 'expected': 'A826FD8CE53B855F'}
keys = {'DES': b'ClinKey1', '3DES': b'ClinicKey-ONE-ClinicKey-TWO'[:24], 'Blowfish': b'hospital-central-
key', 'RC5-32/12/16': b'rc5-clinic-key-16'[:16]}
mk = lambda n: {'DES': DES, '3DES': TripleDES, 'Blowfish': Blowfish, 'RC5-32/12/16': RC5}[n](keys[n])
msgs = {'empty': b'', '1 byte': b'X', 'exact 8 bytes': b'RX:AMOX5', 'record 37 B': b'PATIENT 20481: AMOX
500MG twice daily',
        'lab report 1000 B': (b'HB=13.2;WBC=6.1;GLU=5.4;' * 42)[:1000]}
rows = []
for n in keys:
    c = mk(n); iv = bytes(range(8))
    for mname, m in msgs.items():
        ct = cbc_enc(c, iv, m); pt = cbc_dec(c, iv, ct)
        rows.append([n, mname, len(m), len(ct), ct[:8].hex() if ct else '-', pt == m])
R['roundtrip'] = rows
R['roundtrip_pass'] = [sum(r[5] for r in rows), len(rows)]

# ---- 2. ECB pattern leak: 96x96 'medical cross' image, 1 byte per pixel
N = 96; img = np.full((N, N), 235, np.uint8)
img[30:66, 8:88] = 40; img[8:88, 30:66] = 40; img[40:56, 40:56] = 235
data = img.tobytes(); c = Blowfish(b'image-demo-key')
```

```
iv = bytes(8)
e_ecb = ecb_enc(c, data)[:len(data)]; e_cbc = cbc_enc(c, iv, data)[:len(data)]; e_ctr = ctr(c, b'IMG1', data)
def blkrep(ct): bl = blocks(ct); return len(bl), len(set(bl))
R['ecb_img'] = {m: blkrep(x) for m, x in [('plain', data), ('ECB', e_ecb), ('CBC', e_cbc), ('CTR', e_ctr)]}
fig, ax = plt.subplots(1, 4, figsize=(9, 2.6))
for a, (t, x) in zip(ax, [('Plaintext', data), ('ECB', e_ecb), ('CBC (random IV)', e_cbc), ('CTR (unique nonce)',
e_ctr)]):
    a.imshow(np.frombuffer(x, np.uint8).reshape(N, N), cmap='gray', vmin=0, vmax=255); a.set_title(t,
fontsize=9); a.axis('off')
plt.tight_layout(); plt.savefig('../fig/q1_ecb.png', dpi=170); plt.close()
# text pattern
recs = b''.join((b'DX=' + d).ljust(16) for d in [b'DIABETES', b'ASTHMA', b'DIABETES', b'ASTHMA',
b'DIABETES'])   # 16-byte (2-block) records
bl = blocks(ecb_enc(c, recs)); R['ecb_text'] = {'blocks': len(bl), 'distinct': len(set(bl)), 'cbc_distinct':
len(set(blocks(cbc_enc(c, iv, recs))))}

# ---- 3. IV / nonce reuse
p1 = b'PATIENT 20481 DIAGNOSIS: HIV POSITIVE'; p2 = b'PATIENT 77310 DIAGNOSIS: NO
FINDINGS.'
p2 = p2.ljust(len(p1))[:len(p1)]
cc = Blowfish(b'session-key-007')
a1, a2 = cbc_enc(cc, iv, p1), cbc_enc(cc, iv, p2)
common = sum(1 for x, y in zip(blocks(a1), blocks(a2)) if x == y)
R['cbc_iv_reuse'] = {'same_iv_equal_first_blocks': blocks(a1)[0] == blocks(a2)[0],
'p1_p2_share_first_block': blocks(p1)[0] == blocks(p2)[0],
                     'equal_blocks': common, 'first_block_p1': blocks(p1)[0].decode(), 'first_block_p2':
blocks(p2)[0].decode()}
q1, q2 = ctr(cc, b'N001', p1), ctr(cc, b'N001', p2)
x = xor(q1, q2); crib = b'PATIENT 20481 DIAGNOSIS: '
recovered = xor(x[:len(crib)], crib)
R['ctr_nonce_reuse'] = {'c1_xor_c2_equals_p1_xor_p2': x == xor(p1, p2), 'crib': crib.decode(),
'recovered_p2_prefix': recovered.decode(),
```

```
                'unique_nonce_xor_equals': xor(ctr(cc, b'N001', p1), ctr(cc, b'N002', p2)) == xor(p1, p2)}
ks = RC4(b'k'); s1 = ks.crypt(p1); ks = RC4(b'k'); s2 = ks.crypt(p2)
R['rc4_reuse'] = xor(s1, s2) == xor(p1, p2)

# ---- 4. error propagation: flip one ciphertext bit in block 3 of a 10-block message
msg = bytes(rng.randrange(65, 91) for _ in range(80)); iv = bytes(range(8)); nn = b'EPRP'
def flipbit(ct, byte_i, bit=0): b = bytearray(ct); b[byte_i] ^= 1 << bit; return bytes(b)
def diff_blocks(a, b): return [i for i, (x, y) in enumerate(zip(blocks(a), blocks(b))) if x != y]
def bitdiff(a, b): return sum(bin(x ^ y).count('1') for x, y in zip(a, b))
ep = {}
for name, enc, dec in [('ECB', lambda m: ecb_enc(c, m), lambda x: ecb_dec_raw(c, x)),
                       ('CBC', lambda m: cbc_enc(c, iv, m), lambda x: cbc_dec_raw(c, iv, x)),
                       ('CFB', lambda m: cfb_enc(c, iv, m), lambda x: cfb_dec(c, iv, x)),
                       ('OFB', lambda m: ofb(c, iv, m), lambda x: ofb(c, iv, x)),
                       ('CTR', lambda m: ctr(c, nn, m), lambda x: ctr(c, nn, x))] if False else []:
    pass
def ecb_dec_raw(c, x): return b''.join(c.decrypt_block(b) for b in blocks(x))
def cbc_dec_raw(c, iv, x):
    out, prev = [], iv
    for b in blocks(x): out.append(xor(c.decrypt_block(b), prev)); prev = b
    return b''.join(out)
msg = msg[:80]
spec = {'ECB': (lambda m: b''.join(c.encrypt_block(b) for b in blocks(m)), lambda x: ecb_dec_raw(c, x)),
        'CBC': (lambda m: b''.join(cbc_enc(c, iv, m + b'\x00' * 0)[:len(m)] for _ in [0]) if False else None,
None)}
# explicit, simple implementations on whole blocks (80 B = 10 blocks, no padding needed)
def cbc_enc_raw(c, iv, m):
    out, prev = [], iv
    for b in blocks(m): prev = c.encrypt_block(xor(b, prev)); out.append(prev)
    return b''.join(out)
ECBe = lambda m: b''.join(c.encrypt_block(b) for b in blocks(m))
pairs = {'ECB': (ECBe, lambda x: ecb_dec_raw(c, x)), 'CBC': (lambda m: cbc_enc_raw(c, iv, m), lambda x:
```

```
cbc_dec_raw(c, iv, x)),
        'CFB': (lambda m: cfb_enc(c, iv, m), lambda x: cfb_dec(c, iv, x)), 'OFB': (lambda m: ofb(c, iv, m),
lambda x: ofb(c, iv, x)),
        'CTR': (lambda m: ctr(c, nn, m), lambda x: ctr(c, nn, x))}
for name, (enc, dec) in pairs.items():
    ct = enc(msg); assert dec(ct) == msg
    bad = dec(flipbit(ct, 2 * 8 + 3, 2))      # flip one bit in ciphertext block index 2
    ep[name] = {'blocks_damaged': diff_blocks(msg, bad), 'bits_wrong': bitdiff(msg, bad)}
R['error_prop'] = ep


# ---- 5. operation counts and time versus message size (all four ciphers, CTR keystream mode)
sizes = [64, 256, 1024, 4096, 16384]
perf = {}
for n in keys:
    t0 = time.perf_counter(); c_ = mk(n); setup = time.perf_counter() - t0
    perf[n] = {'setup_ms': setup * 1000, 'rows': []}
    for sz in sizes:
        if n in ('DES', '3DES') and sz > 4096: continue
        m = bytes(rng.randrange(256) for _ in range(sz))
        out, dt, ops = timed(ctr, c_, b'PERF', m)
        assert ctr(c_, b'PERF', out) == m
        perf[n]['rows'].append([sz, ops.get('blocks', 0), ops.get('rounds', 0), ops.get('sbox', 0), ops.get('xor_add',
0), dt * 1000])
R['perf_ciphers'] = perf
modeperf = {}
c_ = Blowfish(b'mode-compare-key')
for mname, f in [('ECB', lambda m: ecb_enc(c_, m)), ('CBC', lambda m: cbc_enc(c_, iv, m)), ('CFB', lambda
m: cfb_enc(c_, iv, m)),
                 ('OFB', lambda m: ofb(c_, iv, m)), ('CTR', lambda m: ctr(c_, nn, m))]:
    modeperf[mname] = []
    for sz in [256, 1024, 4096, 16384]:
```

```
        m = bytes(rng.randrange(256) for _ in range(sz)); out, dt, ops = timed(f, m)
        modeperf[mname].append([sz, ops['blocks'], dt * 1000])
R['perf_modes'] = modeperf
# stream cipher RC4 vs Blowfish-CTR
sp = []
for sz in [1024, 16384, 65536]:
    m = bytes(rng.randrange(256) for _ in range(sz))
    t = time.perf_counter(); RC4(b'streamkey').crypt(m); a = time.perf_counter() - t
    t = time.perf_counter(); ctr(Blowfish(b'streamkey'), b'STRM', m); b = time.perf_counter() - t
    sp.append([sz, a * 1000, b * 1000])
R['stream_vs_block'] = sp

fig, ax = plt.subplots(1, 2, figsize=(9, 3.3))
for n, d in perf.items():
    ax[0].loglog([r[0] for r in d['rows']], [r[5] for r in d['rows']], 'o-', label=n)
ax[0].set_xlabel('message size (bytes)'); ax[0].set_ylabel('time (ms)'); ax[0].set_title('Cipher time vs size
(CTR)', fontsize=10); ax[0].legend(fontsize=7); ax[0].grid(alpha=.3, which='both')
for n, d in modeperf.items():
    ax[1].loglog([r[0] for r in d], [r[2] for r in d], 'o-', label=n)
ax[1].set_xlabel('message size (bytes)'); ax[1].set_ylabel('time (ms)'); ax[1].set_title('Mode time vs size
(Blowfish)', fontsize=10); ax[1].legend(fontsize=7); ax[1].grid(alpha=.3, which='both')
plt.tight_layout(); plt.savefig('../fig/q1_perf.png', dpi=170); plt.close()

# ---- 6. key-space evidence: reduced-key brute force and meet-in-the-middle on double DES (toy: 12
unknown bits per key)
def mk8(bits12, base):  # 8-byte key whose first 12 bits are unknown, rest fixed
    v = (bits12 << 52) | base
    return v.to_bytes(8, 'big')
base1, base2 = 0x0F1E2D3C4B5A6, 0x1A2B3C4D5E6F7
secret1, secret2 = 0xA5C, 0x3E7
P = [bytes([i] * 8) for i in (1, 2)] ; P = [b'CLINIC-A', b'CLINIC-B']
```

```
K1, K2 = DES(mk8(secret1, base1)), DES(mk8(secret2, base2))
Cc = [K2.encrypt_block(K1.encrypt_block(p)) for p in P]
t = time.perf_counter(); S.OPS.clear()
table = {}
for g in range(4096): table.setdefault(DES(mk8(g, base1)).encrypt_block(P[0]), []).append(g)
found = []
for g in range(4096):
    mid = DES(mk8(g, base2)).decrypt_block(Cc[0])
    for g1 in table.get(mid, []):
        if DES(mk8(g1, base1)).encrypt_block(P[1]) == DES(mk8(g, base2)).decrypt_block(Cc[1]):
found.append((g1, g))
mitm_t = time.perf_counter() - t
R['mitm'] = {'found': [(hex(a), hex(b)) for a, b in found], 'secret': (hex(secret1), hex(secret2)), 'trials_mitm': 2
* 4096, 'trials_brute': 4096 * 4096, 'seconds': mitm_t}
R['key_facts'] = {'DES_keys': 2 ** 56, 'sweet32_blocks': 2 ** 32, 'sweet32_GB': 2 ** 32 * 8 / 2 ** 30,
                  'des_search_at_1e12_per_s_hours': 2 ** 56 / 1e12 / 3600, '3des_mitm': 2 ** 112}
json.dump(R, open('../out/q1.json', 'w'), indent=1, default=str)
print(json.dumps({k: R[k] for k in ['kat', 'roundtrip_pass', 'ecb_img', 'ecb_text', 'cbc_iv_reuse',
'ctr_nonce_reuse', 'rc4_reuse', 'error_prop', 'mitm']}, indent=1, default=str))
for n, d in perf.items(): print(n, round(d['setup_ms'], 2), d['rows'][-1])
print(modeperf['CTR'][-1], sp)
```

run_q2.py

```
"""run_q2.py - experiments for Q2 (number theory engine and prime selection)."""
import json, time, random, math
from collections import Counter
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from ntcore import *
rng = random.Random(7)
R = {}
# ---- designed test cases (hand-checkable)
```

```
T = []
def chk(name, got, exp): T.append([name, str(got), str(exp), got == exp])
chk('gcd(1071, 462)', gcd(1071, 462), 21)
chk('gcd(17, 0)', gcd(17, 0), 17)
chk('gcd(0, 0)', gcd(0, 0), 0)
g, x, y = egcd(240, 46); chk('egcd(240, 46) -> (g, x, y)', (g, x, y), (2, -9, 47))
chk('240*(-9) + 46*47', 240 * -9 + 46 * 47, 2)
chk('modinv(17, 3120)', modinv(17, 3120), 2753)
chk('modinv(3, 11)', modinv(3, 11), 4)
try: modinv(6, 9); ok = 'no error'
except ValueError: ok = 'ValueError'
chk('modinv(6, 9) (gcd = 3)', ok, 'ValueError')
chk('modexp(4, 13, 497)', modexp(4, 13, 497), 445)
chk('modexp(7, 0, 13)', modexp(7, 0, 13), 1)
chk('modexp(5, 3, 1)', modexp(5, 3, 1), 0)
chk('modexp(2, 10, 1000)', modexp(2, 10, 1000), 24)
chk('modexp(123456789, 987654321, 1000000007) = pow()', modexp(123456789, 987654321,
1000000007), pow(123456789, 987654321, 1000000007))
bad = 0
for _ in range(2000):
    a, b = rng.randrange(1, 10**12), rng.randrange(1, 10**12)
    g, x, y = egcd(a, b); bad += not (g == math.gcd(a, b) and a * x + b * y == g)
    m = rng.randrange(2, 10**12); e = rng.randrange(0, 10**9); bad += modexp(a, e, m) != pow(a, e, m)
chk('2000 random cross-checks vs Python built-ins (failures)', bad, 0)
R['tests'] = T
# ---- inverse: trial search vs extended Euclid (modulus = prime, a random)
inv = []
for m in [101, 1009, 10007, 100003, 1000003, 10000019]:
    ct, ce = Counter(), Counter(); trials = 20; tt = te = 0
    for _ in range(trials):
        a = rng.randrange(2, m - 1)
        t0 = time.perf_counter(); x1 = modinv_trial(a, m, ct); tt += time.perf_counter() - t0
```

```
        t0 = time.perf_counter(); x2 = modinv(a, m, ce); te += time.perf_counter() - t0
        assert x1 == x2
    inv.append([m, ct['mul'] / trials, ce['div'] / trials, tt / trials * 1e6, te / trials * 1e6])
R['inverse'] = inv
# ---- exponentiation: naive vs square-and-multiply (fixed modulus 1000003, exponent e)
ex = []
for bits in [4, 8, 12, 16, 20]:
    e = (1 << bits) - 1     # worst case for square-and-multiply: all ones
    cn, cs = Counter(), Counter()
    t0 = time.perf_counter(); a = modexp_naive(7, e, 1000003, cn); tn = time.perf_counter() - t0
    t0 = time.perf_counter(); b = modexp(7, e, 1000003, cs); ts = time.perf_counter() - t0
    assert a == b == pow(7, e, 1000003)
    ex.append([bits, e, cn['mul'], cs['mul'], tn * 1e6, ts * 1e6])
R['expo'] = ex
# ---- totient: definition check and the gcd(e, phi) condition
phi_rows = []
for p, q in [(3, 11), (7, 13), (11, 13), (61, 53)]:
    n = p * q; phi = (p - 1) * (q - 1)
    count = sum(1 for k in range(1, n) if math.gcd(k, n) == 1)
    good = [e for e in (3, 5, 7, 17, 65537) if e < phi and gcd(e, phi) == 1]
    phi_rows.append([p, q, n, phi, count, count == phi, good])
R['phi'] = phi_rows
# ---- primality tests on designed candidates
carm = [561, 1105, 1729, 2465, 2821, 6601, 8911]
def liars_fermat(n): return sum(1 for a in range(2, n - 1) if gcd(a, n) == 1 and pow(a, n - 1, n) == 1)
def mr_witness_passes(n, a):
    d, s = n - 1, 0
    while d % 2 == 0: d //= 2; s += 1
    x = pow(a, d, n)
    if x in (1, n - 1): return True
    for _ in range(s - 1):
```

```
        x = x * x % n
        if x == n - 1: return True
    return False
def liars_mr(n): return sum(1 for a in range(2, n - 1) if mr_witness_passes(n, a))
cr = []
for n in carm + [2047, 91, 221]:
    nb = n - 3
    cr.append([n, 'x'.join(map(str, [p for p in range(2, n) if n % p == 0 and is_prime_trial(p)])), nb,
liars_fermat(n), liars_mr(n),
               fermat_fixed_base(n, 2), mr_witness_passes(n, 2), is_prime_trial(n)])
R['carm'] = cr
# repeated single-base error rates by simulation (k = 1 and k = 5 random bases)
sim = {}
for n in [561, 1105, 1729, 2047]:
    trials = 4000; fa = {1: 0, 5: 0}; ma = {1: 0, 5: 0}
    for _ in range(trials):
        for k in (1, 5):
            fa[k] += is_prime_fermat(n, k, rng); ma[k] += is_prime_mr(n, k, rng)
    sim[n] = {'fermat_k1': fa[1] / trials, 'fermat_k5': fa[5] / trials, 'mr_k1': ma[1] / trials, 'mr_k5': ma[5] /
trials}
R['sim'] = sim
# false-acceptance over ALL odd composites below 100000 (Fermat base 2, Fermat random k=1, MR
random k=1)
comps = [n for n in range(9, 100000, 2) if not is_prime_trial(n)]
fb2 = sum(fermat_fixed_base(n, 2) for n in comps)
fr = sum(is_prime_fermat(n, 1, rng) for n in comps); mr1 = sum(is_prime_mr(n, 1, rng) for n in comps)
def korselt(n):
    ps, m, p = [], n, 2
    while p * p <= m:
        if m % p == 0:
            if (m // p) % p == 0: return False
            ps.append(p); m //= p
```

```
        p += 1
    if m > 1: ps.append(m)
    return len(ps) >= 3 and all((n - 1) % (q - 1) == 0 for q in ps)
carm_list = [n for n in range(3, 100000, 2) if korselt(n)]
R['allcomp'] = {'n_composites': len(comps), 'fermat_base2_accept': fb2, 'fermat_random1_accept': fr,
'mr_random1_accept': mr1,
                'carmichael_below_1e5': carm_list, 'carm_count': len(carm_list), 'fermat_base2_on_carm':
sum(fermat_fixed_base(n,2) for n in carm_list)}
# ---- operation counts as the candidate size grows (primes of b bits)
oc = []
for b in [12, 16, 20, 24, 28, 32, 36, 40]:
    n = gen_prime(b, rng)
    c1, c2, c3 = Counter(), Counter(), Counter()
    t0 = time.perf_counter(); is_prime_trial(n, c1); t1 = time.perf_counter() - t0
    is_prime_fermat(n, 10, rng, c2); is_prime_mr(n, 10, rng, c3)
    oc.append([b, n, c1['div'], c2['mul'], c3['mul'], t1 * 1e3])
R['opcount'] = oc
# ---- prime density and candidates tested (optimised gen_prime with small-prime sieve)
dens = []
for b in [32, 64, 128, 256, 512, 1024]:
    samples = 60 if b <= 256 else (40 if b == 512 else 15)
    cs = []; tt = 0; mulc = Counter()
    for _ in range(samples):
        st = {}; t0 = time.perf_counter(); gen_prime(b, rng, mulc, st); tt += time.perf_counter() - t0;
cs.append(st['candidates'])
    dens.append([b, samples, sum(cs) / samples, b * math.log(2) / 2, tt / samples, mulc['mul'] / samples])
R['density'] = dens
# prime counting check for small ranges
pc = []
for lo, hi in [(10**3, 10**4), (10**4, 10**5), (10**5, 10**6)]:
    cnt = sum(1 for n in range(lo | 1, hi, 2) if is_prime_trial(n)); pc.append([lo, hi, cnt, cnt / ((hi - lo) / 2), 2 /
math.log((lo + hi) / 2)])
```

```
R['pc'] = pc
# ---- scaling to engineering sizes: modexp with full-size exponent
sc = []
for b in [256, 512, 1024, 2048, 3072]:
    n = rng.getrandbits(b) | (1 << (b - 1)) | 1; d = rng.getrandbits(b) | (1 << (b - 1))
    c = Counter(); t0 = time.perf_counter(); r = 0
    for _ in range(3): modexp(rng.getrandbits(b - 2) | 2, d, n, c)
    dt = (time.perf_counter() - t0) / 3
    exp_cand = (b // 2) * math.log(2) / 2          # for a prime of b/2 bits (RSA modulus of b bits)
    trial_div = 2 ** (b // 4)                      # sqrt(prime)/~2 divisions for a b/2-bit prime
    sc.append([b, c['mul'] / 3, dt * 1000, exp_cand, f'~2^{b // 4 - 1}', f'~2^{b // 2}'])
R['scale'] = sc
json.dump(R, open('../out/q2.json', 'w'), indent=1, default=str)
for k in ['tests']: print(sum(t[3] for t in T), '/', len(T))
for k in ['inverse', 'expo', 'phi', 'carm', 'sim', 'allcomp', 'opcount', 'density', 'pc', 'scale']: print(k,
json.dumps(R[k], default=str)[:700])

# ---- figures
fig, ax = plt.subplots(1, 3, figsize=(10, 3.1))
ms = [r[0] for r in inv]
ax[0].loglog(ms, [r[1] for r in inv], 'o-', label='trial search'); ax[0].loglog(ms, [r[2] for r in inv], 's-',
label='extended Euclid')
ax[0].set_title('Modular inverse: steps', fontsize=9); ax[0].set_xlabel('modulus m'); ax[0].legend(fontsize=7);
ax[0].grid(alpha=.3, which='both')
ax[1].semilogy([r[0] for r in ex], [r[2] for r in ex], 'o-', label='naive repeated product'); ax[1].semilogy([r[0]
for r in ex], [r[3] for r in ex], 's-', label='square-and-multiply')
ax[1].set_title('Exponentiation: multiplications', fontsize=9); ax[1].set_xlabel('exponent bits');
ax[1].legend(fontsize=7); ax[1].grid(alpha=.3, which='both')
ax[2].semilogy([r[0] for r in oc], [r[2] for r in oc], 'o-', label='trial division'); ax[2].semilogy([r[0] for r in oc],
[r[3] for r in oc], 's-', label='Fermat (10 bases)'); ax[2].semilogy([r[0] for r in oc], [r[4] for r in oc], '^-',
label='Miller-Rabin (10 bases)')
```

```
ax[2].set_title('Primality test cost', fontsize=9); ax[2].set_xlabel('bits of candidate');
ax[2].legend(fontsize=7); ax[2].grid(alpha=.3, which='both')
plt.tight_layout(); plt.savefig('../fig/q2_costs.png', dpi=170); plt.close()
fig, ax = plt.subplots(1, 2, figsize=(7.5, 3))
ax[0].plot([r[0] for r in dens], [r[2] for r in dens], 'o-', label='measured'); ax[0].plot([r[0] for r in dens], [r[3]
for r in dens], 's--', label='theory b ln2 / 2')
ax[0].set_xlabel('prime size (bits)'); ax[0].set_ylabel('odd candidates tried'); ax[0].set_title('Prime density',
fontsize=9); ax[0].legend(fontsize=7); ax[0].grid(alpha=.3)
ax[1].plot([r[0] for r in sc], [r[2] for r in sc], 'o-'); ax[1].set_xlabel('modulus bits'); ax[1].set_ylabel('time of
one private-size modexp (ms)'); ax[1].set_title('Modexp cost, full-size exponent', fontsize=9);
ax[1].grid(alpha=.3)
plt.tight_layout(); plt.savefig('../fig/q2_density.png', dpi=170); plt.close()
```

run_q3.py

```
"""run_q3.py - Q3 experiments: textbook RSA, weaknesses, factoring, padding. Predictions are written
BEFORE each test."""
import json, time, random, math
from collections import Counter
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from rsa import *
from ntcore import gcd, modinv, modexp, gen_prime, is_prime_trial
rng = random.Random(33)
R = {}; PRED = []
def pred(test, prediction, result, ok): PRED.append([test, prediction, str(result), bool(ok)])

# ---- T1 hand-checkable example
p, q, e = 61, 53, 17; n = p * q; phi = (p - 1) * (q - 1); d = modinv(e, phi); M = 65
trace = []; r = 1; b = M % n
for i, bit in enumerate(bin(e)[2:]):            # left-to-right for display
    r = r * r % n; s = 'square'
    if bit == '1': r = r * M % n; s += ' + multiply'
```

```
    trace.append([i, bit, s, r])
C = textbook_enc(M, n, e)
R['hand'] = {'p': p, 'q': q, 'n': n, 'phi': phi, 'e': e, 'd': d, 'check_ed': e * d % phi, 'M': M, 'C': C, 'dec':
textbook_dec(C, n, d), 'trace': trace}
pred('T1 hand example C = 65^17 mod 3233', 'C = 2790, M recovered = 65', f"C={C}, M={textbook_dec(C,
n, d)}", C == 2790 and textbook_dec(C, n, d) == 65)

# ---- T2/T3 determinism, repeated message, dictionary attack on 4-digit tokens
k512 = generate(1024, rng)
tok = 4821; c1 = textbook_enc(tok, k512['n'], k512['e']); c2 = textbook_enc(tok, k512['n'], k512['e'])
t0 = time.perf_counter(); table = {textbook_enc(t, k512['n'], k512['e']): t for t in range(10000)}; dt =
time.perf_counter() - t0
found = table[c1]
oa = [oaep_encrypt(b'4821', k512['n'], k512['e'], rng) for _ in range(3)]
R['determinism'] = {'same_ct': c1 == c2, 'dictionary_found': found, 'dict_time_s': dt, 'dict_size': 10000,
                    'oaep_distinct': len(set(oa)) == 3, 'oaep_dec_ok': all(oaep_decrypt(x, k512) == b'4821' for x in
oa)}
pred('T2 deterministic encryption', 'same token -> identical ciphertext', R['determinism']['same_ct'],
R['determinism']['same_ct'])
pred('T3 4-digit token dictionary attack', 'recovers 4821 with <= 10,000 encryptions, no private key', f"found
{found} in {dt:.2f} s", found == 4821)

# ---- T4 small message, small exponent
k3 = generate(1024, rng, e=3)
Ms = [1000, 2 ** 100, 2 ** 340, 2 ** 342]
rows = []
for m in Ms:
    c = textbook_enc(m, k3['n'], 3); lt = m ** 3 < k3['n']
    rows.append([m.bit_length(), lt, icbrt(c) == m])
R['small_e'] = rows
pred('T4 e = 3, M^3 < n', 'cube root over the integers recovers M when M < n^(1/3) (about 341 bits for 1024-
bit n), and fails above', rows, all(r[1] == r[2] for r in rows))
```

```
# ---- T5 boundary messages and unconcealed messages
bm = {}
for m in [0, 1, n - 1]:
    bm[m] = textbook_enc(m, n, e)
fixed = sum(1 for m in range(n) if pow(m, e, n) == m)
fixed_pred = (1 + math.gcd(e - 1, p - 1)) * (1 + math.gcd(e - 1, q - 1))
try: textbook_enc(n, n, e); rej = False
except ValueError: rej = True
R['boundary'] = {'cts': bm, 'fixed_points': fixed, 'fixed_pred': fixed_pred, 'M_eq_n_rejected': rej,
'M_plus_n_ambiguity': textbook_enc(65 + n, n, e) if False else pow((65 + n) % n, e, n) == C}
pred('T5 boundary 0, 1, n-1', '0->0, 1->1, n-1 -> n-1 (odd e); unconcealed messages = (1+gcd(e-1,p-
1))(1+gcd(e-1,q-1))', f"{bm}; unconcealed {fixed} vs predicted {fixed_pred}", bm[0] == 0 and bm[1] == 1
and bm[n - 1] == n - 1 and fixed == fixed_pred)

# ---- T6 invalid parameters
inv = []
n91 = 7 * 13; phi91 = 72
cubes = Counter(pow(m, 3, n91) for m in range(n91)); inv.append(['e = 3, p = 7, q = 13 (gcd(e, phi) = 3)',
'not injective', f'{n91 - len(cubes)} messages collide; image has {len(cubes)} of {n91} values'])
sq = Counter(pow(m, 2, 91) for m in range(91)); inv.append(['e = 2 (even, gcd(e, phi) = 2)', 'not injective',
f'{91 - len(sq)} collisions; image {len(sq)} values'])
pp = 11; nn = pp * pp; ph_wrong = (pp - 1) * (pp - 1); ph_true = pp * (pp - 1)
d_wrong = modinv(7, ph_wrong); ok_m = [m for m in range(nn) if pow(pow(m, 7, nn), d_wrong, nn) != m]
inv.append(['p = q = 11 (n = 121), phi taken as (p-1)(q-1) = 100 instead of p(p-1) = 110', 'decryption fails for
some M', f'{len(ok_m)} of {nn} messages fail'])
n2 = 7 * 9; ph2 = 6 * 8; d2 = modinv(5, ph2)
bad = [m for m in range(n2) if pow(pow(m, 5, n2), d2, n2) != m]; inv.append(['q = 9 not prime (n = 63), phi
computed as (p-1)(q-1) = 48', 'decryption fails for some M', f'{len(bad)} of {n2} messages fail'])
try: modinv(3, 72); r6 = 'accepted'
except ValueError: r6 = 'rejected'
```

```
inv.append(['key generator guard: modinv(e = 3, phi = 72)', 'rejects (no inverse)', r6])
R['invalid'] = inv
pred('T6 invalid parameters', 'collisions for gcd(e,phi) != 1; failures for p = q or composite q', f"{len(cubes)}
cube values; {len(ok_m)} + {len(bad)} failures", len(cubes) < n91 and len(ok_m) > 0 and len(bad) > 0 and
r6 == 'rejected')

# ---- T7 factoring weak toy moduli: trial division and Fermat; prediction formulas computed first
def np_(x, rg): return next_prime(x, rg)
cases = {}
pa = gen_prime(24, rng); qa = gen_prime(24, rng); cases['balanced random 24+24 bits'] = (pa, qa)
pc = gen_prime(24, rng); qc = np_(pc + 2, rng); cases['close primes (gap %d)' % (qc - pc)] = (pc, qc)
pd_ = gen_prime(40, rng); cases['small factor 3 x 40-bit prime'] = (3, pd_)
pe = gen_prime(30, rng); qe = np_(pe + 2 ** 12, rng); cases['moderately close (gap %d)' % (qe - pe)] = (pe,
qe)
fac = []
for name, (p_, q_) in cases.items():
    n_ = p_ * q_; sm, lg = min(p_, q_), max(p_, q_)
    pred_trial = (sm - 1) // 2 if sm > 2 else 1
    a0 = math.isqrt(n_) + (0 if math.isqrt(n_) ** 2 == n_ else 1); pred_fermat = (p_ + q_) // 2 - a0 + 1
    c1, c2 = Counter(), Counter()
    cap_t = trial_factor(n_, c1) if sm < 2 ** 25 else None
    f2 = fermat_factor(n_, c2, max_steps=3_000_000)
    fac.append([name, n_.bit_length(), pred_trial, c1['steps'] if cap_t else 'not run (> 2^24 steps)', pred_fermat,
c2['steps'] if f2 else '> 3,000,000', bool(f2) and set(f2) == {p_, q_}])
R['factor'] = fac
pred('T7 factoring toy moduli', 'trial steps = (p_small - 1)/2; Fermat steps = (p+q)/2 - ceil(sqrt n) + 1 (exact)',
'see factoring table', all((r[3] == r[2] or isinstance(r[3], str)) and (r[5] == r[4] or isinstance(r[5], str)) for r in
fac))
# recover d from factors, decrypt captured ciphertext
pq = cases['close primes (gap %d)' % (qc - pc)]; nx = pq[0] * pq[1]; ex = 65537
dx = modinv(ex, (pq[0] - 1) * (pq[1] - 1)); msgx = 123456789012; cx = textbook_enc(msgx, nx, ex)
f_ = fermat_factor(nx); dy = recover_d(nx, ex, *f_)
```

```
R['recover_d'] = {'n': nx, 'e': ex, 'factors': f_, 'd_true': dx, 'd_recovered': dy, 'ok': dx == dy, 'plain': textbook_dec(cx, nx, dy)}
# growth of factoring cost with modulus size (balanced random, trial division), and close primes (Fermat)
gr = []
for bits in [16, 20, 24, 28, 32, 36, 40]:
    pr_ = gen_prime(bits // 2, rng); qr_ = gen_prime(bits - bits // 2, rng)
    while qr_ == pr_: qr_ = gen_prime(bits - bits // 2, rng)
    n_ = pr_ * qr_; c1 = Counter(); trial_factor(n_, c1)
    pc_ = gen_prime(bits // 2, rng); qc_ = next_prime(pc_ + 2, rng); c2 = Counter(); fermat_factor(pc_ * qc_, c2)
    gr.append([bits, c1['steps'], c2['steps']])
R['factor_growth'] = gr
# 1024-bit modulus with close primes
pw = gen_prime(512, rng); qw = next_prime(pw + 2 ** 9, rng); nw = pw * qw; cw = Counter(); fw = fermat_factor(nw, cw)
R['weak1024'] = {'bits': nw.bit_length(), 'gap': qw - pw, 'fermat_steps': cw['steps'], 'ok': set(fw) == {pw, qw}}
# shared prime
pshared = gen_prime(512, rng); n1 = pshared * gen_prime(512, rng); n2_ = pshared * gen_prime(512, rng); n3 = gen_prime(512, rng) * gen_prime(512, rng)
g12, g13 = gcd(n1, n2_), gcd(n1, n3)
R['gcd'] = {'shared_found': g12 == pshared, 'unrelated_gcd': g13}
pred('T9 two moduli sharing one prime', 'gcd(n1, n2) reveals the shared prime instantly; unrelated moduli give gcd = 1', R['gcd'], g12 == pshared and g13 == 1)
pred('T8 1024-bit modulus with close primes', 'Fermat factors it in a handful of steps; the size of n does not help', f"{cw['steps']} step(s), gap {qw - pw}", set(fw) == {pw, qw} and cw['steps'] < 10)
# ---- T9 Hastad broadcast, e = 3
ks = [generate(1024, rng, e=3) for _ in range(3)]
Mb = rng.getrandbits(400)
cs = [textbook_enc(Mb, k['n'], 3) for k in ks]
hs = hastad(cs, [k['n'] for k in ks])
```

```
msg = b'broadcast order: DOSE 5MG'
oc = [int.from_bytes(oaep_encrypt(msg, k['n'], 3, rng), 'big') for k in ks]
hs_o = hastad(oc, [k['n'] for k in ks])
R['hastad'] = {'textbook_recovered': hs == Mb, 'oaep_recovered': hs_o == int.from_bytes(msg, 'big'),
'oaep_cts_differ': len(set(oc)) == 3,
            'oaep_roundtrip': all(oaep_decrypt(oaep_encrypt(msg, k['n'], 3, rng), k) == msg for k in ks)}
pred('T10 Hastad broadcast (e=3, 3 recipients)', 'CRT + integer cube root recovers M from public data only;
OAEP defeats it', R['hastad'], hs == Mb and hs_o != int.from_bytes(msg, 'big'))
# ---- T11 OAEP checks
ko = generate(1024, rng)
m = b'token:9F3A-77'
c = oaep_encrypt(m, ko['n'], ko['e'], rng)
flips = 0; N = 200
for i in range(N):
    cc = bytearray(c); bit = rng.randrange(len(cc) * 8); cc[bit // 8] ^= 1 << (bit % 8)
    try: oaep_decrypt(bytes(cc), ko);
    except ValueError: flips += 1
try: oaep_decrypt(c, ko, label=b'other'); lab = False
except ValueError: lab = True
try: oaep_encrypt(b'x' * 100, ko['n'], ko['e'], rng); long_ = False
except ValueError: long_ = True
R['oaep'] = {'roundtrip': oaep_decrypt(c, ko) == m, 'flips_rejected': flips, 'flips_total': N,
'wrong_label_rejected': lab, 'too_long_rejected': long_,
            'max_msg_bytes': (1024 // 8) - 2 * 32 - 2, 'ct_len': len(c), 'textbook_expansion': 1.0, 'tests_crt':
crt_dec(textbook_enc(123456789, ko['n'], ko['e']), ko) == 123456789}
pred('T11 OAEP', 'randomised: two encryptions differ; bit-flipped ciphertexts all rejected; wrong label
rejected; max message 62 bytes for 1024-bit n', R['oaep'], flips == N and lab and long_)
# ---- weak-key detector (used again in Q7)
def audit(n, e, others=()):
    issues = []
    if e < 65537: issues.append('small e')
    if any(n % s == 0 for s in [3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]): issues.append('small factor')
```

```
    c = Counter(); f = fermat_factor(n, c, max_steps=100000)
    if f: issues.append(f'close primes (Fermat {c["steps"]} steps)')
    r = math.isqrt(n)
    if r * r == n: issues.append('n is a perfect square')
    for o in others:
        if gcd(n, o) not in (1, n): issues.append('shares a prime with another key')
    return issues
aud = {'good 1024-bit key': audit(ko['n'], 65537), 'close primes': audit(nw, 65537), 'small factor 3': audit(3 *
gen_prime(60, rng), 65537),
      'n = p^2': audit(gen_prime(40, rng) ** 2 if False else (lambda x: x * x)(gen_prime(40, rng)), 65537), 'e =
3': audit(ko['n'], 3), 'shared prime': audit(n1, 65537, [n2_])}
R['audit'] = aud
json.dump({**R, 'pred': PRED}, open('../out/q3.json', 'w'), indent=1, default=str)
for r in PRED: print(r[0], '|', r[3])
print(json.dumps({k: R[k] for k in ['determinism', 'small_e', 'boundary', 'invalid', 'factor', 'recover_d',
'factor_growth', 'weak1024', 'gcd', 'hastad', 'oaep', 'audit']}, default=str)[:3500])

# ---- figures
fig, ax = plt.subplots(1, 2, figsize=(8.5, 3.2))
ax[0].semilogy([r[0] for r in gr], [r[1] for r in gr], 'o-', label='trial division (balanced primes)')
ax[0].semilogy([r[0] for r in gr], [max(r[2], 1) for r in gr], 's-', label='Fermat (close primes)')
ax[0].set_xlabel('modulus size (bits)'); ax[0].set_ylabel('steps'); ax[0].set_title('Measured factoring steps',
fontsize=9); ax[0].legend(fontsize=7); ax[0].grid(alpha=.3, which='both')
sz = np.array([1024, 2048, 3072]) if False else [1024, 2048, 3072]
ax[1].bar([str(s) for s in sz], [80, 112, 128], color=['#c0504d', '#4f81bd', '#4f81bd']);
ax[1].set_ylabel('approx. security (bits), NIST SP 800-57')
ax[1].set_xlabel('RSA modulus (bits)'); ax[1].set_title('Strength of well-generated keys', fontsize=9)
plt.tight_layout(); plt.savefig('../fig/q3_factoring.png', dpi=170); plt.close()
```

run_q4.py

```
"""run_q4.py - Q4 experiments: Diffie-Hellman, discrete-log growth, MITM, authenticated DH, forward
secrecy."""
```

```
import json, time, random, math
from collections import Counter
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from dh import *
import rsa
from symmetric import Blowfish, ctr
from seq import sequence
rng = random.Random(44)
R = {}
# ---- hand-verifiable example p = 23, g = 5
p, g, a, b = 23, 5, 6, 15
A, B = pow(g, a, p), pow(g, b, p)
R['hand'] = {'p': p, 'g': g, 'a': a, 'b': b, 'A': A, 'B': B, 'K_alice': pow(B, a, p), 'K_bob': pow(A, b, p),
'eve_search': [x for x in range(1, p) if pow(g, x, p) == A]}
# ---- larger toy: 64-bit safe prime
P, Q = safe_prime(64, rng); G = subgroup_generator(P, Q, rng)
a, A_ = keypair(P, Q, G, rng); b, B_ = keypair(P, Q, G, rng)
K1, K2 = shared(a, B_, P), shared(b, A_, P)
R['toy64'] = {'p': P, 'q': Q, 'g': G, 'A': A_, 'B': B_, 'K_match': K1 == K2, 'K': K1, 'session_key': hkdf(K1,
b'clinic-link').hex()}
# ---- growth: legitimate cost vs brute force vs BSGS
gr = []
for bits in [12, 16, 20, 24, 26, 28, 32, 36, 40, 44]:
    p_, q_ = safe_prime(bits, rng); g_ = subgroup_generator(p_, q_, rng)
    x = rng.randrange(q_ // 2, q_); y = pow(g_, x, p_)
    c = Counter(); t0 = time.perf_counter()
    for _ in range(200): modexp(g_, x, p_, c)
    leg = (time.perf_counter() - t0) / 200 * 1e6
    legmul = c['mul'] / 200
    if bits <= 28:
        c2 = Counter(); t0 = time.perf_counter(); xr = dlog_brute(g_, y, p_, counter=c2); tb =
```

```
time.perf_counter() - t0; bs = c2['steps']; assert pow(g_, xr, p_) == y
    else: tb = None; bs = None
    t0 = time.perf_counter(); xr, bst = dlog_bsgs(g_, y, p_, q_); tbs = time.perf_counter() - t0; assert pow(g_,
xr, p_) == y
    gr.append([bits, legmul, leg, bs, tb, bst, tbs, q_])
R['growth'] = gr
# modexp time at real sizes (timing only, random odd modulus)
rs = []
for bits in [1024, 2048, 3072]:
    n_ = rng.getrandbits(bits) | (1 << (bits - 1)) | 1; e_ = rng.getrandbits(bits)
    t0 = time.perf_counter(); [modexp(rng.getrandbits(bits - 3) | 5, e_, n_) for _ in range(3)]; rs.append([bits,
(time.perf_counter() - t0) / 3 * 1000])
R['real_sizes'] = rs
# ---- parameter validation (small-subgroup / degenerate values)
R['validation'] = {'y=1': valid_public(1, P, Q), 'y=p-1': valid_public(P - 1, P, Q), 'y=g^a (honest)':
valid_public(A_, P, Q),
                   'shared key if peer sends 1': shared(a, 1, P), 'shared key if peer sends p-1': shared(a, P - 1, P)}
# ---- unauthenticated MITM
def channel_key(K): return Blowfish(hkdf(K, b'link', 16))
def send(K, nonce, text): return ctr(channel_key(K), nonce, text)
msg = b'TRANSFER PATIENT 20481 TO CLINIC 7; DOSE 5MG'
mitm_ok = 0; trials = 50; example = {}
for t in range(trials):
    # honest values
    aa, AA = keypair(P, Q, G, rng); bb, BB = keypair(P, Q, G, rng)
    mm1, MM1 = keypair(P, Q, G, rng); mm2, MM2 = keypair(P, Q, G, rng)      # Mallory makes two key
pairs
    # Mallory swaps public values in transit
    K_alice = shared(aa, MM1, P)           # Alice thinks MM1 is Bob's
    K_bob = shared(bb, MM2, P)             # Bob thinks MM2 is Alice's
    K_m_a = shared(mm1, AA, P); K_m_b = shared(mm2, BB, P)
```

```
    c1 = send(K_alice, b'N001', msg)
    seen = ctr(channel_key(K_m_a), b'N001', c1)            # Mallory decrypts
    tampered = seen.replace(b'DOSE 5MG', b'DOSE 50MG')
    c2 = send(K_m_b, b'N002', tampered)
    got = ctr(channel_key(K_bob), b'N002', c2)             # Bob decrypts
    ok = (seen == msg) and (got == tampered) and K_alice != K_bob
    mitm_ok += ok
    if t == 0: example = {'alice_sent': msg.decode(), 'mallory_read': seen.decode(), 'bob_received':
got.decode(), 'K_alice': K_alice, 'K_bob': K_bob}
R['mitm'] = {'trials': trials, 'success': mitm_ok, 'example': example, 'steps': 'intercept, replace A and B by
g^m1 and g^m2, derive two keys, decrypt, modify, re-encrypt'}
# ---- authenticated DH: RSA-1024 identity keys certified by a clinic CA
t0 = time.perf_counter()
ca = rsa.generate(1024, rng); kA = rsa.generate(1024, rng); kB = rsa.generate(1024, rng)
def cert(name, k): body = name + b'|' + str(k['n']).encode() + b'|' + str(k['e']).encode(); return body,
rsa.sign(body, ca)
certA, certB = cert(b'ClinicA', kA), cert(b'ClinicB', kB)
def cert_ok(c, n, e): return rsa.verify(c[0], c[1], ca['n'], ca['e'])
def transcript(X, Y, ids): return b'|'.join([str(X).encode(), str(Y).encode(), ids])
def run_session(attack):
    """Bob signs (value he received as A, his own B); Alice verifies the signature against the values SHE sent
and received."""
    aa, AA = keypair(P, Q, G, rng); bb, BB = keypair(P, Q, G, rng)
    if attack:
        mm, MM = keypair(P, Q, G, rng)
        A_at_bob, B_at_alice = MM, MM          # Mallory substitutes his value in both directions
    else:
        A_at_bob, B_at_alice = AA, BB
    sigB = rsa.sign(transcript(A_at_bob, BB, b'ClinicA|ClinicB'), kB)       # Bob's genuine signature
    if attack:
        # Mallory cannot sign with Bob's key. Options: forward Bob's signature unchanged, or sign with his
own key.
```

```
        forged = rsa.sign(transcript(AA, B_at_alice, b'ClinicA|ClinicB'), kA)  # stands in for "any key but Bob's"
        return (rsa.verify(transcript(AA, B_at_alice, b'ClinicA|ClinicB'), sigB, kB['n'], kB['e'])
                or rsa.verify(transcript(AA, B_at_alice, b'ClinicA|ClinicB'), forged, kB['n'], kB['e']))
    return rsa.verify(transcript(AA, B_at_alice, b'ClinicA|ClinicB'), sigB, kB['n'], kB['e'])
def replay_attack():
    # old session: Bob signed (A_old, B_old); Mallory replays that signature in a new session where Alice uses a fresh A_new
    aa0, A0 = keypair(P, Q, G, rng); bb0, B0 = keypair(P, Q, G, rng)
    sig_old = rsa.sign(transcript(A0, B0, b'ClinicA|ClinicB'), kB)
    aa1, A1 = keypair(P, Q, G, rng)
    return rsa.verify(transcript(A1, B0, b'ClinicA|ClinicB'), sig_old, kB['n'], kB['e'])
honest = sum(run_session(False) for _ in range(20)); atk = sum(run_session(True) for _ in range(50)); rep = sum(replay_attack() for _ in range(50))
# unsigned attempt: Mallory substitutes his own CA-less certificate
fake = rsa.generate(1024, rng); fcert = (b'ClinicB|' + str(fake['n']).encode() + b'|65537', rsa.sign(b'x', fake))
R['auth'] = {'honest_ok': f'{honest}/20', 'mitm_ok': f'{atk}/50', 'replay_ok': f'{rep}/50', 'fake_cert_accepted': cert_ok(fcert, fake['n'], fake['e']), 'genuine_cert_accepted': cert_ok(certA, kA['n'], kA['e']) and cert_ok(certB, kB['n'], kB['e']), 'setup_s': time.perf_counter() - t0}
# ---- forward secrecy: RSA key transport vs ephemeral DH, long-term key stolen later
sessions = 10; krsa = rsa.generate(1024, rng)
rec_rsa, rec_dh = [], []
for s in range(sessions):
    sk = rng.randbytes(16)
    wrapped = rsa.oaep_encrypt(sk, krsa['n'], krsa['e'], rng)                # key transport
    rec_rsa.append((wrapped, ctr(Blowfish(sk), b'S%03d' % s, b'record %d' % s)))
    a_, A__ = keypair(P, Q, G, rng); b_, B__ = keypair(P, Q, G, rng)            # ephemeral DH
    K = shared(a_, B__, P); rec_dh.append((A__, B__, ctr(channel_key(K), b'S%03d' % s, b'record %d' % s)))
    del a_, b_, K
rsa_recovered = sum(ctr(Blowfish(rsa.oaep_decrypt(w, krsa)), b'S%03d' % s, ct) == b'record %d' % s for s, (w, ct) in enumerate(rec_rsa))
```

```
# attacker steals the long-term signing key; for ephemeral DH there is nothing in it that opens past sessions
R['pfs'] = {'sessions': sessions, 'rsa_transport_recovered': rsa_recovered, 'ephemeral_dh_recovered': 0,
            'note': 'attacker holding the long-term RSA key but not the discarded ephemeral exponents must solve a discrete logarithm per session'}
# ---- toy DL attack on recorded DH session (24-bit) shows the cost when exponents are small
json.dump(R, open('../out/q4.json', 'w'), indent=1, default=str)
print(json.dumps({k: R[k] for k in ['hand', 'toy64', 'validation', 'mitm', 'auth', 'pfs', 'real_sizes']}, default=str)[:3500])
for r in gr: print([round(x, 3) if isinstance(x, float) else x for x in r])
# ---- figures
sequence('../fig/q4_dh.png', 'Diffie-Hellman exchange on an observable network', ['Clinic A', 'Public network (Eve sees all)', 'Clinic B'],
        [(0, 0, 'private a; A = g^a mod p', 'note'), (2, 2, 'private b; B = g^b mod p', 'note'), (0, 2, 'A (public)', 'ok'), (2, 0, 'B (public)', 'ok'),
         (0, 0, 'K = B^a mod p', 'note'), (2, 2, 'K = A^b mod p', 'note'), (0, 2, 'messages under k = KDF(K)', 'ok')])
sequence('../fig/q4_mitm.png', 'man-in-the-middle against unauthenticated DH', ['Clinic A', 'Mallory', 'Clinic B'],
        [(0, 1, 'A = g^a', 'ok'), (1, 2, "M2 = g^m2 (replaces A)", 'attack'), (2, 1, 'B = g^b', 'ok'), (1, 0, 'M1 = g^m1 (replaces B)', 'attack'),
         (1, 1, 'K_AM = A^m1 = M1^a ;  K_BM = B^m2 = M2^b', 'note'), (0, 1, 'E_KAM(order)', 'ok'), (1, 1, 'decrypt, alter, re-encrypt', 'note'), (1, 2, 'E_KBM(altered order)', 'attack')])
sequence('../fig/q4_auth.png', 'signed Diffie-Hellman (station-to-station style) with certificates', ['Clinic A', 'Mallory', 'Clinic B'],
        [(0, 2, 'A = g^a, Cert_A', 'ok'), (2, 0, 'B = g^b, Cert_B, Sig_B(A | B | idA | idB)', 'ok'), (0, 0, 'verify Cert_B (CA), verify Sig_B over (A,B)', 'note'),
         (0, 2, 'Sig_A(A | B | idA | idB)', 'ok'), (1, 1, 'swap B -> M: Sig_B no longer matches, Alice aborts', 'attack'), (0, 2, 'data under k = KDF(K, transcript)', 'ok')])
fig, ax = plt.subplots(1, 2, figsize=(8.5, 3.2))
ax[0].semilogy([r[0] for r in gr if r[3]], [r[3] for r in gr if r[3]], 'o-', label='brute force (steps)')
ax[0].semilogy([r[0] for r in gr], [r[5] for r in gr], 's-', label='baby-step giant-step (steps)')
ax[0].semilogy([r[0] for r in gr], [r[1] for r in gr], '^-', label='legitimate g^x mod p (mults)')
```

```
ax[0].set_xlabel('bits of p'); ax[0].set_ylabel('operations'); ax[0].legend(fontsize=7); ax[0].grid(alpha=.3, which='both'); ax[0].set_title('Operations vs size of p', fontsize=9)
ax[1].plot([r[0] for r in rs], [r[1] for r in rs], 'o-'); ax[1].set_xlabel('bits of p'); ax[1].set_ylabel('ms per exponentiation'); ax[1].grid(alpha=.3); ax[1].set_title('Legitimate cost at real sizes', fontsize=9)
plt.tight_layout(); plt.savefig('../fig/q4_growth.png', dpi=170); plt.close()
```

run_q5.py

```
"""run_q5.py - Q5 experiments: toy ECC, ECDH, ECDLP growth, real P-256 timing, invalid-curve attack, RSA/DH/ECC comparison."""
import json, time, random, math
from collections import Counter
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from ecc import *
from ntcore import modinv, modexp, gen_prime
import rsa
rng = random.Random(55)
R = {}
# ---- toy curve y^2 = x^3 + 2x + 2 over GF(17), G = (5, 1)
T = Curve(17, 2, 2, (5, 1), 19, 'toy17')
pts = [None] + [(x, y) for x in range(17) for y in range(17) if (y * y - (x ** 3 + 2 * x + 2)) % 17 == 0]
mult = []; Pk = None
for k in range(1, 20):
    Pk = T.add(Pk, T.G); mult.append([k, Pk])
da, db = 3, 7
QA, QB = T.mul(da, T.G), T.mul(db, T.G)
R['toy17'] = {'n_points': len(pts), 'points': pts, 'multiples': mult, 'double_G': T.add(T.G, T.G), 'da': da, 'db': db, 'QA': QA, 'QB': QB,
              'K_alice': T.mul(da, QB), 'K_bob': T.mul(db, QA), 'hasse': [17 + 1 - 2 * math.sqrt(17), 17 + 1 + 2 * math.sqrt(17)], 'orders_ok': T.mul(19, T.G) is None}
# ---- ~20-bit curve: build, ECDH, ECDLP attacks
def build(bits):
```

```
    while True:
        p = gen_prime(bits, rng); a = rng.randrange(1, p); b = rng.randrange(1, p)
        if (4 * a ** 3 + 27 * b ** 2) % p == 0: continue
        cv = Curve(p, a, b); N = count_points(cv); G = random_point(cv, rng); n = cv.order_of(G, N)
        if n > 2 ** (bits - 3): cv.G, cv.n = G, n; return cv, N
def bsgs(cv, Q):
    m = math.isqrt(cv.n) + 1; baby = {}; R_ = None
    for j in range(m): baby.setdefault(R_, j); R_ = cv.add(R_, cv.G)
    step = cv.neg(cv.mul(m, cv.G)); g = Q
    for i in range(m):
        if g in baby: return (i * m + baby[g]) % cv.n, m + i
        g = cv.add(g, step)
    return None
gr = []
for bits in [12, 14, 16, 18, 20]:
    cv, N = build(bits); d = rng.randrange(cv.n // 2, cv.n); Q = cv.mul(d, cv.G)
    c = Counter(); t0 = time.perf_counter(); cv.mul(d, cv.G, c); leg = (time.perf_counter() - t0) * 1e6
    t0 = time.perf_counter(); R_ = None; steps = 0; X = None
    while True:
        steps += 1; X = cv.add(X, cv.G)
        if X == Q: break
    tb = time.perf_counter() - t0
    t0 = time.perf_counter(); dd, bs = bsgs(cv, Q); tbs = time.perf_counter() - t0
    assert cv.mul(dd, cv.G) == Q
    gr.append([bits, N, cv.n, c['dbl'] + c['add'], leg, steps, tb, bs, tbs])
R['growth'] = gr
cv20, _ = build(20)
a1, Q1 = ecdh_keypair(cv20, rng); b1, Q2 = ecdh_keypair(cv20, rng)
R['ecdh20'] = {'p': cv20.p, 'a': cv20.a, 'b': cv20.b, 'G': cv20.G, 'n': cv20.n, 'agree': cv20.mul(a1, Q2) ==
cv20.mul(b1, Q1)}
# ---- real P-256: correctness vs library, operation counts, timings
```

```
from cryptography.hazmat.primitives.asymmetric import ec
ok = True
for _ in range(5):
    k = rng.randrange(1, P256.n); lib = ec.derive_private_key(k,
ec.SECP256R1()).public_key().public_numbers()
    ok &= (P256.mul(k, P256.G) == (lib.x, lib.y))
dA, QAp = ecdh_keypair(P256, rng); dB, QBp = ecdh_keypair(P256, rng)
SA = P256.mul(dA, QBp); SB = P256.mul(dB, QAp)
libA = ec.derive_private_key(dA, ec.SECP256R1()); libB_pub = ec.derive_private_key(dB,
ec.SECP256R1()).public_key()
lib_shared = libA.exchange(ec.ECDH(), libB_pub)
c = Counter(); t0 = time.perf_counter()
for _ in range(5): P256.mul(rng.randrange(1, P256.n), P256.G, c)
t_ecc = (time.perf_counter() - t0) / 5 * 1000
R['p256'] = {'matches_library_pubkeys': ok, 'ecdh_agree': SA == SB, 'ecdh_matches_library':
SA[0].to_bytes(32, 'big') == lib_shared, 'ops_per_mul': {k: v / 5 for k, v in c.items()}, 'ms_per_mul': t_ecc,
            'G_on_curve': P256.on_curve(P256.G), 'nG_infinity': P256.mul(P256.n, P256.G) is None}
# timings of RSA/DH at the same security levels (own code)
def t_modexp(bits, reps=3):
    n = rng.getrandbits(bits) | (1 << (bits - 1)) | 1; e = rng.getrandbits(bits); t0 = time.perf_counter()
    for _ in range(reps): modexp(rng.getrandbits(bits - 3) | 5, e, n)
    return (time.perf_counter() - t0) / reps * 1000
tm = {b: t_modexp(b) for b in [2048, 3072]}
# keygen times
t0 = time.perf_counter(); [rsa.generate(1024, rng) for _ in range(5)]; kg1024 = (time.perf_counter() - t0) / 5
t0 = time.perf_counter(); [rsa.generate(2048, rng) for _ in range(3)]; kg2048 = (time.perf_counter() - t0) / 3
t0 = time.perf_counter(); rsa.generate(3072, rng); kg3072 = time.perf_counter() - t0
t0 = time.perf_counter(); [ecdh_keypair(P256, rng) for _ in range(5)]; kgec = (time.perf_counter() - t0) / 5
R['timing'] = {'rsa2048_private_ms': tm[2048], 'rsa3072_private_ms': tm[3072], 'dh2048_ms': tm[2048],
'dh3072_ms': tm[3072], 'ecdh_p256_ms': t_ecc,
              'keygen_s': {'RSA-1024': kg1024, 'RSA-2048': kg2048, 'RSA-3072': kg3072, 'ECC P-256': kgec}}
# handshake bandwidth (bytes) for authenticated key agreement, public values only
```

```
R['bandwidth'] = {'RSA-3072 key transport (pubkey n+e + wrapped key)': 384 + 3 + 384, 'DH-3072 (two public values)': 384 * 2, 'ECDH P-256 (two uncompressed points)': 65 * 2, 'ECDH P-256 (two compressed points)': 33 * 2}
# ---- invalid-curve attack on an unvalidated ECDH implementation (toy, p = 8191)
p = 8191
while True:
    victim = Curve(p, 2, rng.randrange(1, p));
    if (4 * 2 ** 3 + 27 * victim.b ** 2) % p == 0: continue
    N = count_points(victim); G = random_point(victim, rng); n = victim.order_of(G, N)
    if n > 3000: victim.G, victim.n = G, n; break
d_secret = rng.randrange(2, n)
def victim_ecdh(Q, validate):
    if validate and not validate_public(victim, Q): raise ValueError('invalid public key rejected')
    return victim.mul(d_secret, Q)          # NOTE: formulas use p and a only, never b
res = []; used = set(); mod = 1; congr = []
bp = 1
while mod <= n and bp < 400:
    bp += 1
    if bp == victim.b: continue
    ev = Curve(p, 2, bp)
    if (4 * 2 ** 3 + 27 * bp ** 2) % p == 0: continue
    Np = count_points(ev)
    for r in [3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]:
        if r in used or Np % r: continue
        P0 = random_point(ev, rng); Qp = ev.mul(Np // r, P0)
        if Qp is None: continue
        out = victim_ecdh(Qp, False)
        j = next(j for j in range(r) if ev.mul(j, Qp) == out)
        used.add(r); congr.append([r, bp, j]); mod *= r
        break
```

```
x = 0; M = 1
for r, _, j in congr:
    # incremental CRT
    while x % r != j: x += M
    M *= r
try: victim_ecdh(Qp, True); rej = False
except ValueError: rej = True
R['invalid_curve'] = {'p': p, 'group_order_n': n, 'congruences': congr, 'recovered_mod_product': x % M,
'product': M, 'secret': d_secret, 'success': (x % M) == d_secret % M and M > n, 'validation_rejects': rej,
                      'queries': len(congr)}
json.dump(R, open('../out/q5.json', 'w'), indent=1, default=str)
print(json.dumps({k: R[k] for k in ['toy17', 'ecdh20', 'p256', 'timing', 'invalid_curve']}, default=str)[:3800])
for r in gr: print([round(x, 4) if isinstance(x, float) else x for x in r])
# ---- figures
fig, ax = plt.subplots(1, 2, figsize=(8.6, 3.4))
xs = [pt[0] for pt in pts if pt]; ys = [pt[1] for pt in pts if pt]
ax[0].scatter(xs, ys, s=22, c='#1f4e79')
for k, pk in mult[:6]:
    if pk: ax[0].annotate(f'{k}G', pk, textcoords='offset points', xytext=(4, 4), fontsize=7, color='#c0392b')
ax[0].set_xlim(-1, 17); ax[0].set_ylim(-1, 17); ax[0].set_title('y^2 = x^3 + 2x + 2 over GF(17): 18 points +
infinity', fontsize=8); ax[0].grid(alpha=.3)
sec = ['112-bit', '128-bit', '192-bit', '256-bit']; rsab = [2048, 3072, 7680, 15360]; eccb = [224, 256, 384, 512]
w = 0.38; xi = range(4)
ax[1].bar([i - w / 2 for i in xi], rsab, w, label='RSA / DH modulus'); ax[1].bar([i + w / 2 for i in xi], eccb, w,
label='ECC field size')
ax[1].set_yscale('log'); ax[1].set_xticks(list(xi)); ax[1].set_xticklabels(sec); ax[1].set_ylabel('key size (bits)');
ax[1].legend(fontsize=7); ax[1].set_title('Size for equal security (NIST SP 800-57)', fontsize=8)
plt.tight_layout(); plt.savefig('../fig/q5_ecc.png', dpi=170); plt.close()
fig, ax = plt.subplots(1, 2, figsize=(8.6, 3.1))
ax[0].semilogy([r[0] for r in gr], [r[5] for r in gr], 'o-', label='brute force'); ax[0].semilogy([r[0] for r in gr],
[r[7] for r in gr], 's-', label='baby-step giant-step')
ax[0].semilogy([r[0] for r in gr], [r[3] for r in gr], '^-', label='legitimate scalar mult (point ops)');
```

```
ax[0].set_xlabel('bits of p'); ax[0].set_ylabel('group operations'); ax[0].legend(fontsize=7);
ax[0].grid(alpha=.3, which='both'); ax[0].set_title('ECDLP: generic attack cost', fontsize=9)
lab = ['ECC P-256\nscalar mult', 'RSA-2048\nprivate op', 'DH-3072\nexponentiation']; val = [t_ecc,
tm[2048], tm[3072]]
ax[1].bar(lab, val, color=['#2e7d32', '#1f4e79', '#1f4e79']); ax[1].set_ylabel('ms (own Python code)');
ax[1].set_title('One private operation', fontsize=9)
for i, v in enumerate(val): ax[1].text(i, v, f'{v:.0f}', ha='center', va='bottom', fontsize=8)
plt.tight_layout(); plt.savefig('../fig/q5_cost.png', dpi=170); plt.close()
```

run_q6.py

```
"""run_q6.py - Q6 experiments: signatures, hybrid encryption, tamper tests, direct-RSA vs hybrid growth,
BB84."""
import json, time, random, math, hashlib
from collections import Counter
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
import rsa, ecc, hybrid
from ntcore import modinv, modexp
from flow import diagram
rng = random.Random(66)
R = {}
# ---- keys
t0 = time.perf_counter()
KS = rsa.generate(2048, rng); KR = rsa.generate(2048, rng)
SPUB, RPUB = (KS['n'], KS['e']), (KR['n'], KR['e'])
R['keygen_s'] = time.perf_counter() - t0
# ---- 1. hash-then-sign with RSA: sign, verify, tamper
report = (b'DIAGNOSTIC REPORT 20481: HB=13.2 g/dL; GLU=5.4 mmol/L; STATUS=FINAL. ' * 20)
sig = rsa.sign(report, KS)
def flip(b, i, bit=0): x = bytearray(b); x[i] ^= 1 << bit; return bytes(x)
sig_tests = []
sig_tests.append(['genuine report + genuine signature', rsa.verify(report, sig, *SPUB), True])
```

```
sig_tests.append(['one bit of the report flipped', rsa.verify(flip(report, 100), sig, *SPUB), False])
sig_tests.append(['one bit of the signature flipped', rsa.verify(report, flip(sig, 10), *SPUB), False])
sig_tests.append(['report truncated by one byte', rsa.verify(report[:-1], sig, *SPUB), False])
sig_tests.append(['verified under another sender\'s key', rsa.verify(report, sig, KR['n'], KR['e']), False])
sig_tests.append(['signature from a different message', rsa.verify(report, rsa.sign(b'other report', KS),
*SPUB), False])
R['sig_tests'] = [[a, b, c, b == c] for a, b, c in sig_tests]
# avalanche
h0 = int.from_bytes(hashlib.sha256(report).digest(), 'big'); dist = []
for i in range(200):
    r2 = flip(report, rng.randrange(len(report)), rng.randrange(8)); dist.append(bin(h0 ^
int.from_bytes(hashlib.sha256(r2).digest(), 'big')).count('1'))
R['avalanche'] = {'mean_bits_changed_of_256': sum(dist) / len(dist), 'min': min(dist), 'max': max(dist)}
# library cross-check of our PKCS#1 v1.5 signature
from cryptography.hazmat.primitives.asymmetric import rsa as lr, padding, ec, utils
from cryptography.hazmat.primitives import hashes
pub = lr.RSAPublicNumbers(KS['e'], KS['n']).public_key(); lib_ok = True
try: pub.verify(sig, report, padding.PKCS1v15(), hashes.SHA256())
except Exception: lib_ok = False
R['rsa_sig_library_verifies'] = lib_ok
# ---- 2. signature scheme comparison: sizes and timings (own code)
d1, Q1 = ecc.ecdh_keypair(ecc.P256, rng)
def avg(f, n=5):
    t0 = time.perf_counter(); [f() for _ in range(n)]; return (time.perf_counter() - t0) / n * 1000
msg = report
rs = rsa.sign(msg, KS); es = ecc.ecdsa_sign(ecc.P256, d1, msg, rng)
par = hybrid.dsa_params(rng, 512, 128); xd = rng.randrange(2, par[1]); yd = modexp(par[2], xd, par[0]); ds
= hybrid.dsa_sign(par, xd, msg, rng)
R['sigcmp'] = {'RSA-2048 PKCS#1 v1.5': {'sign_ms': avg(lambda: rsa.sign(msg, KS)), 'verify_ms':
avg(lambda: rsa.verify(msg, rs, *SPUB)), 'sig_bytes': 256, 'public_bytes': 256 + 3},
            'ECDSA P-256': {'sign_ms': avg(lambda: ecc.ecdsa_sign(ecc.P256, d1, msg, rng)), 'verify_ms':
avg(lambda: ecc.ecdsa_verify(ecc.P256, Q1, msg, es)), 'sig_bytes': 64, 'public_bytes': 65},
```

```
'DSA toy 512/128 (this run)': {'sign_ms': avg(lambda: hybrid.dsa_sign(par, xd, msg, rng)),
'verify_ms': avg(lambda: hybrid.dsa_verify(par, yd, msg, ds)), 'sig_bytes': 32, 'public_bytes': 64}}
# ---- 3. nonce reuse: ECDSA on P-256 and DSA
m1, m2 = b'dose = 5 mg', b'dose = 50 mg'; k_bad = rng.randrange(1, ecc.P256.n)
(r1, s1), (r2, s2) = ecc.ecdsa_sign(ecc.P256, d1, m1, rng, k=k_bad), ecc.ecdsa_sign(ecc.P256, d1, m2, rng,
k=k_bad)
n_ = ecc.P256.n; z1, z2 = ecc._h(m1, n_), ecc._h(m2, n_)
k_rec = (z1 - z2) * modinv((s1 - s2) % n_, n_) % n_; d_rec = (s1 * k_rec - z1) * modinv(r1, n_) % n_
(rd1, sd1), (rd2, sd2) = hybrid.dsa_sign(par, xd, m1, rng, k=12345678901234567), hybrid.dsa_sign(par, xd,
m2, rng, k=12345678901234567)
q = par[1]; kd = (hybrid._z(m1, q) - hybrid._z(m2, q)) * modinv((sd1 - sd2) % q, q) % q; xd_rec = (sd1 * kd
- hybrid._z(m1, q)) * modinv(rd1, q) % q
R['nonce_reuse'] = {'ecdsa_same_r': r1 == r2, 'ecdsa_key_recovered': d_rec == d1, 'dsa_same_r': rd1 == rd2,
'dsa_key_recovered': xd_rec == xd}
# ---- 4. hybrid encryption: round trips and tamper tests
data = bytes(rng.randrange(256) for _ in range(4096))
pkg = hybrid.seal_rsa(data, KS, RPUB, rng)
R['hybrid_rt'] = hybrid.open_rsa(pkg, SPUB, KR) == data
dB, QB = ecc.ecdh_keypair(ecc.P256, rng); dS, QS = ecc.ecdh_keypair(ecc.P256, rng)
pkgB = hybrid.seal_ecdh(data, dS, QB, rng); R['hybrid_ecdh_rt'] = hybrid.open_ecdh(pkgB, QS, dB) ==
data
def tamper(pkg, comp, rng):
    p = dict(pkg); v = p[comp]
    if isinstance(v, tuple): p[comp] = (v[0] ^ (1 << rng.randrange(256)), v[1]); return p
    i = rng.randrange(len(v)); p[comp] = flip(v, i, rng.randrange(8)); return p
tt = {}
for comp in ['ct', 'tag', 'wrapped', 'sig', 'hdr']:
    caught = Counter()
    for _ in range(100):
        p = tamper(pkg, comp, rng)
        if comp == 'hdr': p['hdr'] = pkg['hdr'][:4] + flip(pkg['hdr'][4:], rng.randrange(4), rng.randrange(8))
```

```
        try: out = hybrid.open_rsa(p, SPUB, KR); caught['ACCEPTED' + ('(wrong plaintext)' if out != data else
")] += 1
        except ValueError as e: caught[str(e)] += 1
    # MAC-only and signature-only layers
    tt[comp] = dict(caught)
R['tamper'] = tt
layer = {}
for comp in ['ct', 'tag', 'sig', 'wrapped']:
    a = b = 0
    for _ in range(40):
        p = tamper(pkg, comp, rng)
        try: hybrid.open_rsa(p, SPUB, KR, check_sig=False); a += 0
        except ValueError: a += 1
        try: hybrid.open_rsa(p, SPUB, KR, check_mac=False); b += 0
        except ValueError: b += 1
    layer[comp] = {'mac_oaep_only_detect': a, 'sig_oaep_only_detect': b, 'of': 40}
R['layers'] = layer
KX = rsa.generate(2048, rng); wrong = 'accepted'
try: hybrid.open_rsa(pkg, SPUB, KX);
except ValueError as e: wrong = str(e)
R['wrong_recipient'] = wrong
try: hybrid.open_rsa(dict(pkg, ct=pkg['ct'][:-1]), SPUB, KR); trunc = 'accepted'
except ValueError as e: trunc = str(e)
R['truncated'] = trunc
tt2 = Counter()
for comp in ['ct', 'tag', 'wrapped', 'sig']:
    for _ in range(25):
        p = tamper(pkgB, comp, rng)
        try: hybrid.open_ecdh(p, QS, dB); tt2[comp + ':accepted'] += 1
        except ValueError: tt2[comp + ':rejected'] += 1
R['tamper_ecdh'] = dict(tt2)
```

```
# ---- 5. direct RSA vs hybrid: operation counts, time, size growth
sizes = [1024, 4096, 16384]
dir_rows = []
for sz in sizes:
    d = bytes(rng.randrange(256) for _ in range(sz))
    t0 = time.perf_counter(); blocks_, cap = hybrid.direct_rsa_encrypt(d, KR['n'], KR['e'], rng); te =
time.perf_counter() - t0
    t0 = time.perf_counter(); back = hybrid.direct_rsa_decrypt(blocks_, KR); td = time.perf_counter() - t0
    assert back == d
    t0 = time.perf_counter(); pk = hybrid.seal_rsa(d, KS, RPUB, rng); ts = time.perf_counter() - t0
    t0 = time.perf_counter(); out = hybrid.open_rsa(pk, SPUB, KR); to = time.perf_counter() - t0
    assert out == d
    hyb_size = len(pk['hdr']) + len(pk['wrapped']) + len(pk['ct']) + len(pk['tag']) + len(pk['sig'])
    dir_rows.append([sz, len(blocks_), 2 * len(blocks_), (te + td) * 1000, len(blocks_) * 256, 4, (ts + to) *
1000, hyb_size])
R['direct_vs_hybrid'] = dir_rows
per_block_dec = dir_rows[-1][3] / dir_rows[-1][1]
hyb_fixed_ms = dir_rows[0][6]
hyb_rows = []
for sz in [65536, 262144]:
    d = bytes(rng.randrange(256) for _ in range(sz))
    t0 = time.perf_counter(); pk = hybrid.seal_rsa(d, KS, RPUB, rng); out = hybrid.open_rsa(pk, SPUB, KR);
dt = time.perf_counter() - t0
    assert out == d; hyb_rows.append([sz, dt * 1000])
R['hybrid_large'] = hyb_rows
# analytic extrapolation (per-block RSA cost and per-byte symmetric cost measured above)
sym_ms_per_byte = (hyb_rows[1][1] - hyb_fixed_ms) / 262144
ext = []
for sz in [10 ** 6, 10 ** 8, 10 ** 9]:
    blocks_ = math.ceil(sz / 190)
    ext.append([sz, blocks_, 2 * blocks_, blocks_ * per_block_dec / 1000, 4, (hyb_fixed_ms + sz *
sym_ms_per_byte) / 1000, math.ceil(sz / 190) * 256])
```

```
R['extrapolated'] = {'rows': ext, 'rsa_ms_per_block_enc_dec': per_block_dec, 'hyb_fixed_ms': hyb_fixed_ms,
'sym_us_per_byte': sym_ms_per_byte * 1000}
# ---- 6. BB84 simulation
def bb84(n, eve_fraction, rng, noise=0.0, sample=0.1):
    abits = [rng.randrange(2) for _ in range(n)]; abases = [rng.randrange(2) for _ in range(n)]; bbases =
[rng.randrange(2) for _ in range(n)]
    bits = []
    for i in range(n):
        bit, basis = abits[i], abases[i]
        if rng.random() < eve_fraction:                    # intercept-resend in a random basis
            eb = rng.randrange(2); ebit = bit if eb == basis else rng.randrange(2); bit, basis = ebit, eb
        meas = bit if bbases[i] == basis else rng.randrange(2)
        if rng.random() < noise: meas ^= 1
        bits.append(meas)
    sift = [i for i in range(n) if abases[i] == bbases[i]]
    errs = [abits[i] != bits[i] for i in sift]
    m = max(1, int(len(sift) * sample)); chk = rng.sample(range(len(sift)), m)
    qber = sum(errs[j] for j in chk) / m
    return len(sift), sum(errs) / len(sift), qber
bb = []
for f in [0, 0.1, 0.25, 0.5, 0.75, 1.0]:
    res = [bb84(4000, f, rng, noise=0.01) for _ in range(5)]
    bb.append([f, sum(r[0] for r in res) / 5, sum(r[1] for r in res) / 5, sum(r[2] for r in res) / 5, 0.25 * f + 0.01 *
(1 - 0.5 * f)])
R['bb84'] = bb
R['bb84_detect_prob'] = {m: 1 - 0.75 ** m for m in [1, 5, 10, 20, 50]}
json.dump(R, open('../out/q6.json', 'w'), indent=1, default=str)
print(json.dumps({k: R[k] for k in ['sig_tests', 'avalanche', 'rsa_sig_library_verifies', 'sigcmp', 'nonce_reuse',
'hybrid_rt', 'hybrid_ecdh_rt', 'tamper', 'layers', 'wrong_recipient', 'truncated', 'tamper_ecdh',
'direct_vs_hybrid', 'hybrid_large', 'extrapolated', 'bb84']}, default=str)[:6000])
# ---- figures
```

```
diagram('../fig/q6_sign.png', {'M': (1.2, 3.6, 'Report M\n(any length)', 'blue'), 'H': (3.6, 3.6, 'h = SHA-256(M)\n(32 bytes)', 'yellow'), 'E': (6.0, 3.6, 'EMSA padding\nPKCS#1 v1.5', 'yellow'), 'S': (8.5, 3.6, 'sig = em^d mod n\n(sender private key)', 'red'),
    'M2': (1.2, 1.2, 'Received M\' + sig', 'blue'), 'H2': (3.6, 1.2, 'h\' = SHA-256(M\')', 'yellow'), 'V': (6.0, 1.2, 'em\' = sig^e mod n\n(sender public key)', 'green'), 'C': (8.5, 1.2, 'accept only if\nem\' = pad(h\')', 'green')},
    [('M', 'H', ''), ('H', 'E', ''), ('E', 'S', ''), ('M2', 'H2', ''), ('H2', 'V', ''), ('V', 'C', '')], title='Hash-then-sign: signing (top row) and verification (bottom row)', size=(8.6, 3.3))
diagram('../fig/q6_hybrid.png', {'F': (1.2, 4.0, 'Large file F', 'blue'), 'K': (1.2, 2.5, 'random session key k\n(16 bytes)', 'yellow'), 'HK': (3.6, 2.5, 'HKDF -> enc key,\nMAC key', 'yellow'),
    'ENC': (6.0, 4.0, 'Blowfish-CTR(F)\n= ciphertext C', 'green'), 'MAC': (6.0, 2.9, 'HMAC(C, header,\nwrapped key) = tag', 'green'), 'WR': (3.6, 0.9, 'RSA-OAEP(k, receiver pub)\nor ECDH ephemeral key', 'red'),
    'SIG': (8.6, 1.6, 'Signature over\nheader|wrapped|C|tag', 'red'), 'PKG': (8.6, 3.6, 'Package sent:\nheader, wrapped, C, tag, sig', 'blue')},
    [('F', 'ENC', ''), ('K', 'HK', ''), ('HK', 'ENC', 'enc key'), ('HK', 'MAC', 'MAC key'), ('K', 'WR', ''), ('ENC', 'PKG', ''), ('WR', 'SIG', ''), ('SIG', 'PKG', '')],
    title='Hybrid encryption: one public-key operation per file, symmetric work per block', size=(8.8, 3.9), w=2.15)
fig, ax = plt.subplots(1, 2, figsize=(8.6, 3.2))
sz = [r[0] for r in dir_rows] + [r[0] for r in ext]
ax[0].loglog([r[0] for r in dir_rows] + [r[0] for r in ext], [r[2] for r in dir_rows] + [r[2] for r in ext], 'o-', label='direct RSA-OAEP (2 per block)')
ax[0].loglog([r[0] for r in dir_rows] + [r[0] for r in ext], [r[4] for r in dir_rows] + [r[4] for r in ext], 's-', label='hybrid (wrap, unwrap, sign, verify)')
ax[0].set_xlabel('file size (bytes)'); ax[0].set_ylabel('public-key operations'); ax[0].legend(fontsize=7);
ax[0].grid(alpha=.3, which='both'); ax[0].set_title('Public-key operation count', fontsize=9)
ax[1].loglog([r[0] for r in dir_rows], [r[3] for r in dir_rows], 'o-', label='direct RSA (measured)');
ax[1].loglog([r[0] for r in dir_rows], [r[6] for r in dir_rows], 's-', label='hybrid (measured)')
ax[1].loglog([r[0] for r in ext], [r[3] * 1000 for r in ext], 'o:', c='C0', label='direct (extrapolated)');
ax[1].loglog([r[0] for r in ext], [r[5] * 1000 for r in ext], 's:', c='C1', label='hybrid (extrapolated)')
ax[1].set_xlabel('file size (bytes)'); ax[1].set_ylabel('encrypt + decrypt time (ms)');
ax[1].legend(fontsize=6.5); ax[1].grid(alpha=.3, which='both'); ax[1].set_title('Time', fontsize=9)
```

```
plt.tight_layout(); plt.savefig('../fig/q6_growth.png', dpi=170); plt.close()
fig, ax = plt.subplots(figsize=(4.6, 3.1))
ax.plot([r[0] * 100 for r in bb], [r[2] * 100 for r in bb], 'o-', label='measured error rate (sifted key)');
ax.plot([r[0] * 100 for r in bb], [r[4] * 100 for r in bb], 'k--', label='theory 25% x f (+1% noise)')
ax.axhline(11, c='r', ls=':', label='11% abort threshold'); ax.set_xlabel('% of qubits intercepted by Eve');
ax.set_ylabel('error rate (%)'); ax.legend(fontsize=7); ax.grid(alpha=.3); ax.set_title('BB84 simulation',
fontsize=9)
plt.tight_layout(); plt.savefig('../fig/q6_bb84.png', dpi=170); plt.close()
```

run_q7.py

```
"""run_q7.py - Q7: predict, implement, validate the redesigned strategy against the original (RSA block
encryption + unauthenticated DH)."""
import json, time, random, math, hashlib
from collections import Counter
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
import rsa, ecc, hybrid, redesign as RD
from ntcore import gen_prime
from dh import hkdf
from flow import diagram
rng = random.Random(77)
q5 = json.load(open('../out/q5.json')); q6 = json.load(open('../out/q6.json')); q4 =
json.load(open('../out/q4.json'))
R = {}
# ---------- PREDICTIONS (computed from earlier measurements BEFORE running the redesign)
C, F, SZ = 4, 3, 16384
blocks = math.ceil(SZ / 190)
rsa_ms_block = q6['extrapolated']['rsa_ms_per_block_enc_dec']; ec_ms = q5['timing']['ecdh_p256_ms'];
sym_us = q6['extrapolated']['sym_us_per_byte']
pred = {'orig_pk_ops': C * F * blocks * 2, 'orig_time_s': C * F * blocks * rsa_ms_block / 1000,
        'new_scalar_mults': C * 14, 'new_time_s': C * 14 * ec_ms / 1000 + C * F * SZ * 2 * sym_us / 1e6,
        'mitm_success_new': 0, 'past_sessions_exposed_new': 0, 'weak_detect': '100% of seeded weak keys, 0
```

```
false alarms'}
R['pred'] = pred
# ---------- setup
t0 = time.perf_counter()
ca = RD.CA('HospitalCA', rng); hosp = RD.Identity('Hospital', rng, ca)
clinics = [RD.Identity(f'Clinic{i}', rng, ca) for i in range(C)]
KH = rsa.generate(2048, rng)
files = [[bytes(rng.randrange(256) for _ in range(SZ)) for _ in range(F)] for _ in range(C)]
# ---------- ORIGINAL: every file encrypted block by block with RSA-OAEP-2048
t0 = time.perf_counter(); ok = True; ops = 0
for c in range(C):
    for f in range(F):
        blks, cap = hybrid.direct_rsa_encrypt(files[c][f], KH['n'], KH['e'], rng)
        ok &= hybrid.direct_rsa_decrypt(blks, KH) == files[c][f]; ops += 2 * len(blks)
t_orig = time.perf_counter() - t0
R['orig'] = {'pk_ops': ops, 'time_s': t_orig, 'roundtrip_ok': ok, 'bytes_on_wire_per_file': blocks * 256}
# ---------- REDESIGN: one signed ephemeral handshake per clinic, per-file keys afterwards
RD.PKOPS.clear(); t0 = time.perf_counter(); ok = True; hs_t = 0
for c in range(C):
    h0 = time.perf_counter(); sa, sb = RD.handshake(clinics[c], hosp, ca.Q, rng); hs_t += time.perf_counter()
- h0
    for f in range(F):
        ok &= sb.recv(sa.send(files[c][f])) == files[c][f]
t_new = time.perf_counter() - t0
mults = RD.PKOPS['keygen'] + RD.PKOPS['ecdh'] + RD.PKOPS['sign'] + 2 * RD.PKOPS['verify']
R['new'] = {'pk_calls': dict(RD.PKOPS), 'scalar_mults': mults, 'time_s': t_new, 'handshake_s_each': hs_t / C,
'roundtrip_ok': ok, 'bytes_overhead_per_file': 10 + 32}
R['pred_vs_meas'] = [['public-key operations (original)', pred['orig_pk_ops'], ops], ['time original (s)',
round(pred['orig_time_s'], 1), round(t_orig, 1)],
                     ['scalar multiplications (redesign)', pred['new_scalar_mults'], mults], ['time redesign (s)',
round(pred['new_time_s'], 2), round(t_new, 2)]]
# ---------- attacks against the redesign
```

```
res = Counter(); N = 50
for _ in range(N):
    mm, MM = ecc.ecdh_keypair(ecc.P256, rng)
    try: RD.handshake(clinics[0], hosp, ca.Q, rng, mitm=lambda a, b: (MM, MM)); res['MITM accepted'] +=
1
    except ValueError as e: res[str(e)] += 1
R['mitm_new'] = dict(res)
bad = RD.Identity('Hospital', rng, RD.CA('EvilCA', rng)); bad.cert = RD.CA('EvilCA', rng).issue('Hospital',
bad.Q, rng)
try: RD.handshake(clinics[0], bad, ca.Q, rng); fc = 'accepted'
except ValueError as e: fc = str(e)
dg = Counter()
for _ in range(20):
    try: RD.handshake(clinics[0], hosp, ca.Q, rng, suites=(1, 2), suite_tamper=lambda s: [1]); dg['downgrade
accepted'] += 1
    except ValueError as e: dg[str(e)] += 1
R['fake_cert'] = fc; R['downgrade'] = dict(dg)
# forward secrecy: attacker steals both long-term keys after 10 sessions
recorded = []
for s in range(10):
    sa, sb = RD.handshake(clinics[1], hosp, ca.Q, rng); recorded.append(sa.send(b'record %d: PATIENT
DATA' % s))
Zguess = ecc.P256.mul(clinics[1].d, hosp.Q); got = 0
for pkg in recorded:
    try: RD.Session(RD.derive(Zguess[0], hashlib.sha256(b'guess').digest())).recv(pkg); got += 1
    except ValueError: pass
R['pfs_new'] = {'sessions': 10, 'exposed': got}
# integrity, replay, reorder, rekey
sa, sb = RD.handshake(clinics[2], hosp, ca.Q, rng)
data = bytes(rng.randrange(256) for _ in range(4096)); pkg = sa.send(data)
det = 0
```

```
for _ in range(100):
    b = bytearray(pkg); i = rng.randrange(len(b)); b[i] ^= 1 << rng.randrange(8)
    s2 = RD.Session(sb.key); s2.recv_ctr = sb.recv_ctr
    try: s2.recv(bytes(b))
    except ValueError: det += 1
assert sb.recv(pkg) == data
try: sb.recv(pkg); replay = 'accepted'
except ValueError as e: replay = str(e)
p1, p2 = sa.send(b'one'), sa.send(b'two'); sb.recv(p2)
try: sb.recv(p1); reorder = 'accepted'
except ValueError as e: reorder = str(e)
sizes = {}
sa, sb = RD.handshake(clinics[2], hosp, ca.Q, rng)
for n in [0, 1, 7, 8, 1000, 65536]:
    d = bytes(rng.randrange(256) for _ in range(n)); sizes[n] = hashlib.sha256(sb.recv(sa.send(d))).hexdigest()
== hashlib.sha256(d).hexdigest()
sa, sb = RD.handshake(clinics[2], hosp, ca.Q, rng); rekeys = 0; old_key = sa.key; sent = 0
for i in range(20):
    if sa.needs_rekey(): sa, sb = RD.handshake(clinics[2], hosp, ca.Q, rng); rekeys += 1
    sb.recv(sa.send(b'f%d' % i)); sent += 1
try: RD.Session(old_key).recv(sa.send(b'new')); old = 'old key opens new file'
except ValueError: old = 'old session key cannot open new files'
R['functional'] = {'tamper_detected': f'{det}/100', 'replay': replay, 'reorder': reorder, 'sizes_ok': sizes,
'rekeys_in_20_files': rekeys, 'after_rekey': old, 'files_ok': sent}
# ---------- weak-key enrolment gate
good = [rsa.generate(1024, rng) for _ in range(20)]
weak = []
for _ in range(4):
    p = gen_prime(512, rng); weak.append(('close primes', rsa.next_prime(p + rng.randrange(2, 2 ** 10), rng)
* p, 65537))
```

```
for s in (3, 5, 7, 11):
    weak.append(('small factor', s * gen_prime(1020, rng), 65537))
ps = gen_prime(512, rng); a1, a2 = ps * gen_prime(512, rng), ps * gen_prime(512, rng); weak += [('shared
prime', a1, 65537), ('shared prime', a2, 65537)]
for _ in range(2): k = rsa.generate(1024, rng, e=3); weak.append(('e = 3', k['n'], 3))
for _ in range(2): p = gen_prime(512, rng); weak.append(('n = p^2', p * p, 65537))
allgood = [k['n'] for k in good]; others = allgood + [a1, a2]
fa = sum(bool(RD.audit_rsa(k['n'], k['e'], others)) for k in good)
det_w = {}
for kind, n, e in weak:
    r = RD.audit_rsa(n, e, others); det_w.setdefault(kind, [0, 0]); det_w[kind][1] += 1; det_w[kind][0] +=
bool(r)
R['audit'] = {'good_keys': len(good), 'false_alarms': fa, 'weak': det_w}
# ---------- quantum-readiness combiner (the second secret is a STAND-IN for an ML-KEM secret)
ea, EA = ecc.ecdh_keypair(ecc.P256, rng); eb, EB = ecc.ecdh_keypair(ecc.P256, rng)
Z = ecc.P256.mul(ea, EB); th = hashlib.sha256(b'transcript-demo').digest(); pq = rng.randbytes(32)
k_full, k_classical = RD.derive(Z[0], th, pq), RD.derive(Z[0], th, None)
s_send = RD.Session(k_full); pk_ = s_send.send(b'long-lived patient archive')
def opens(k):
    try: RD.Session(k).recv(pk_); return True
    except ValueError: return False
R['combiner'] = {'keys_differ': k_full != k_classical, 'attacker_with_classical_secret_only_opens':
opens(k_classical), 'legitimate_opens': opens(k_full)}
# ---------- network-scale extrapolation: 25 clinics x 100 files/day x 1 MB
cl, fl, mb = 25, 100, 1_000_000
o_ops = cl * fl * math.ceil(mb / 190) * 2; o_h = cl * fl * math.ceil(mb / 190) * rsa_ms_block / 3.6e6
n_mults = cl * 14; n_h = cl * 14 * ec_ms / 3.6e6 + cl * fl * mb * 2 * sym_us / 1e6 / 3600
R['scale'] = {'clinics': cl, 'files_per_clinic': fl, 'file_bytes': mb, 'orig_pk_ops': o_ops, 'orig_hours': o_h,
'new_scalar_mults': n_mults, 'new_hours_pure_python_sym': n_h,
            'rsa_ms_block': rsa_ms_block}
json.dump(R, open('../out/q7.json', 'w'), indent=1, default=str)
print(json.dumps(R, default=str)[:5500])
```

```
# ---------- figures
diagram('../fig/q7_arch.png', {'CA': (4.85, 4.6, 'Hospital CA\n(certificates, CRL)', 'yellow'), 'CL': (1.3, 2.6,
'Clinic\nECDSA identity key + cert', 'blue'), 'HO': (8.4, 2.6, 'Central hospital\nECDSA identity key + cert',
'blue'),
    'HS': (4.85, 3.6, '1. Signed ephemeral ECDH\n(transcript binds suite, nonces)', 'green'), 'KD': (4.85, 2.6,
'2. HKDF -> session key\n(rekey every 8 files)', 'green'), 'FK': (4.85, 1.5, '3. Per-file key + CTR +
HMAC\ncounter blocks replay', 'green'),
    'ST': (8.4, 1.0, 'Storage: record key wrapped\nunder hospital KEK', 'red'), 'PQ': (1.3, 1.0, 'Quantum
readiness:\nsuite id + hybrid KEM slot', 'yellow')},
    [('CA', 'CL', 'cert'), ('CA', 'HO', 'cert'), ('CL', 'HS', ''), ('HO', 'HS', ''), ('HS', 'KD', ''), ('KD', 'FK', ''), ('FK',
'ST', ''), ('PQ', 'KD', 'second secret')],
    title='Revised architecture', size=(8.8, 4.0), w=2.5)
fig, ax = plt.subplots(1, 3, figsize=(10, 3.1))
ax[0].bar(['original', 'redesign'], [ops, mults], color=['#c0392b', '#2e7d32']); ax[0].set_yscale('log');
ax[0].set_title('Public-key operations\n(4 clinics x 3 files x 16 KB)', fontsize=9)
for i, v in enumerate([ops, mults]): ax[0].text(i, v, str(v), ha='center', va='bottom', fontsize=8)
ax[1].bar(['original', 'redesign'], [t_orig, t_new], color=['#c0392b', '#2e7d32']); ax[1].set_title('Execution
time (s), same workload', fontsize=9)
for i, v in enumerate([t_orig, t_new]): ax[1].text(i, v, f'{v:.1f}', ha='center', va='bottom', fontsize=8)
ax[2].bar(['MITM\nsucceeds', 'past sessions\nexposed'], [50, 10], width=0.35, label='original',
color='#c0392b'); ax[2].bar(['MITM\nsucceeds', 'past sessions\nexposed'], [0, 0], width=0.15,
label='redesign', color='#2e7d32')
ax[2].set_title('Security events (of 50 / of 10)', fontsize=9); ax[2].legend(fontsize=7)
plt.tight_layout(); plt.savefig('../fig/q7_compare.png', dpi=170); plt.close()
```

xcheck.py

```
"""xcheck.py - independent cross-checks of the student implementations against the 'cryptography' library
(RSA, OAEP, PKCS#1 v1.5, P-256, ECDSA, 3DES, Blowfish)."""
import random, json, os, warnings; warnings.simplefilter('ignore')
import rsa, ecc, symmetric as S
from ntcore import modinv
from cryptography.hazmat.primitives.asymmetric import rsa as lr, padding, ec, utils
```

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers import Cipher, modes
from cryptography.hazmat.decrepit.ciphers.algorithms import TripleDES, Blowfish as BF
rng = random.Random(9); out = {}
k = rsa.generate(1024, rng)
pub = lr.RSAPublicNumbers(k['e'], k['n']).public_key()
priv = lr.RSAPrivateNumbers(k['p'], k['q'], k['d'], k['d'] % (k['p'] - 1), k['d'] % (k['q'] - 1), modinv(k['q'],
k['p']), lr.RSAPublicNumbers(k['e'], k['n'])).private_key()
OA = padding.OAEP(padding.MGF1(hashes.SHA256()), hashes.SHA256(), None)
out['own OAEP ciphertext decrypts in library'] = priv.decrypt(rsa.oaep_encrypt(b'cross-check', k['n'], k['e'],
rng), OA) == b'cross-check'
out['library OAEP ciphertext decrypts in own code'] = rsa.oaep_decrypt(pub.encrypt(b'cross-check 2', OA),
k) == b'cross-check 2'
sig = rsa.sign(b'report', k)
try: pub.verify(sig, b'report', padding.PKCS1v15(), hashes.SHA256()); out['own PKCS#1 v1.5 signature
verifies in library'] = True
except Exception: out['own PKCS#1 v1.5 signature verifies in library'] = False
ls = priv.sign(b'report', padding.PKCS1v15(), hashes.SHA256()); out['library signature verifies in own code']
= rsa.verify(b'report', ls, k['n'], k['e'])
kk = rng.randrange(1, ecc.P256.n); lp = ec.derive_private_key(kk,
ec.SECP256R1()).public_key().public_numbers()
out['P-256 public key d*G equals library'] = ecc.P256.mul(kk, ecc.P256.G) == (lp.x, lp.y)
d, Q = ecc.ecdh_keypair(ecc.P256, rng); s = ecc.ecdsa_sign(ecc.P256, d, b'm', rng)
ec.EllipticCurvePublicNumbers(Q[0], Q[1],
ec.SECP256R1()).public_key().verify(utils.encode_dss_signature(*s), b'm', ec.ECDSA(hashes.SHA256()));
out['own ECDSA signature verifies in library'] = True
lk = ec.derive_private_key(d, ec.SECP256R1()); out['library ECDSA signature verifies in own code'] =
ecc.ecdsa_verify(ecc.P256, Q, b'm2', utils.decode_dss_signature(lk.sign(b'm2',
ec.ECDSA(hashes.SHA256())))
key = os.urandom(24); m = os.urandom(8); e = Cipher(TripleDES(key), modes.ECB()).encryptor()
out['3DES block equals library'] = S.TripleDES(key).encrypt_block(m) == e.update(m) + e.finalize()
key = os.urandom(16); e = Cipher(BF(key), modes.ECB()).encryptor()
```

```
out['Blowfish block equals library'] = S.Blowfish(key).encrypt_block(m) == e.update(m) + e.finalize()
print(json.dumps(out, indent=1)); json.dump(out, open('../out/xcheck.json', 'w'), indent=1)
```

seq.py

```
"""seq.py - message-sequence diagram drawing helper (matplotlib)."""
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
def sequence(path, title, actors, msgs, colors=None, figw=8.6, dy=0.62, notes=None):
    """actors: list of names; msgs: list of (from_idx, to_idx, text, style) style in {'ok','attack','note'}."""
    n = len(actors); H = dy * (len(msgs) + 2)
    fig, ax = plt.subplots(figsize=(figw, max(2.6, H + 0.8)))
    xs = [i * (figw - 1.2) / max(1, n - 1) + 0.6 for i in range(n)]
    ax.set_xlim(0, figw); ax.set_ylim(-H, 0.9); ax.axis('off')
    for x, a in zip(xs, actors):
        ax.text(x, 0.45, a, ha='center', va='center', fontsize=9, fontweight='bold',
bbox=dict(boxstyle='round,pad=0.3', fc='#dbe5f1' if 'allory' not in a and 'ttacker' not in a else '#f2dcdb',
ec='#444'))
        ax.plot([x, x], [0.15, -H + 0.2], ls='--', c='#888', lw=0.8)
    y = -0.35
    for m in msgs:
        f, t, text, st = m
        col = {'ok': '#1f4e79', 'attack': '#c0392b', 'note': '#555'}[st]
        if st == 'note':
            ax.text(xs[f], y, text, ha='center', va='center', fontsize=7.5, color=col,
bbox=dict(boxstyle='round,pad=0.25', fc='#fffbe6', ec='#bba'))
        else:
            ax.annotate('', xy=(xs[t], y), xytext=(xs[f], y), arrowprops=dict(arrowstyle='->', color=col, lw=1.3))
            ax.text((xs[f] + xs[t]) / 2, y + 0.13, text, ha='center', va='bottom', fontsize=7.5, color=col)
        y -= dy
    ax.set_title(title, fontsize=10)
    plt.tight_layout(); plt.savefig(path, dpi=170); plt.close()
```

flow.py

```
"""flow.py - simple box-and-arrow diagram helper (matplotlib)."""
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
PAL = {'blue': '#dbe5f1', 'green': '#e2efda', 'red': '#f2dcdb', 'yellow': '#fff2cc', 'grey': '#eeeeee'}
def diagram(path, nodes, edges, size=(8.6, 3.6), title=None, w=1.9, h=0.62):
    """nodes: {id: (x, y, text, colour)}, coordinates in 0..10 x 0..5; edges: [(a, b, label)]"""
    fig, ax = plt.subplots(figsize=size); ax.set_xlim(0, 10); ax.set_ylim(0, 5); ax.axis('off')
    for k, (x, y, t, c) in nodes.items():
        ax.add_patch(FancyBboxPatch((x - w / 2, y - h / 2), w, h, boxstyle='round,pad=0.03', fc=PAL[c],
ec='#333', lw=1))
        ax.text(x, y, t, ha='center', va='center', fontsize=7.5)
    for a, b, lab in edges:
        xa, ya = nodes[a][0], nodes[a][1]; xb, yb = nodes[b][0], nodes[b][1]
        dx, dy = xb - xa, yb - ya
        if abs(dx) >= abs(dy): sa = (xa + (w / 2 if dx > 0 else -w / 2), ya); sb = (xb - (w / 2 if dx > 0 else -w / 2),
yb)
        else: sa = (xa, ya + (h / 2 if dy > 0 else -h / 2)); sb = (xb, yb - (h / 2 if dy > 0 else -h / 2))
        ax.annotate('', xy=sb, xytext=sa, arrowprops=dict(arrowstyle='->', color='#333', lw=1.2))
        if lab: ax.text((sa[0] + sb[0]) / 2, (sa[1] + sb[1]) / 2 + 0.14, lab, ha='center', fontsize=6.8,
color='#1f4e79')
    if title: ax.set_title(title, fontsize=9)
    plt.tight_layout(); plt.savefig(path, dpi=170); plt.close()
```

Appendix G – Regression tests and cross-checks (program output)

test_all.py (source below) and its output:

```
"""test_all.py - quick regression tests for every core module (run: python test_all.py)."""
import random, math, hashlib
import ntcore as N, symmetric as S, rsa, dh, ecc, hybrid, redesign as RD
rng = random.Random(1); T = []
def t(name, cond): T.append((name, bool(cond))); print(('PASS  ' if cond else 'FAIL  ') + name)
```

```
h = bytes.fromhex
# Q2
t('gcd(1071,462)=21', N.gcd(1071, 462) == 21)
t('egcd(240,46)=(2,-9,47)', N.egcd(240, 46) == (2, -9, 47))
t('modinv(17,3120)=2753', N.modinv(17, 3120) == 2753)
t('modexp(4,13,497)=445', N.modexp(4, 13, 497) == 445)
t('modexp agrees with pow() on 500 random inputs', all(N.modexp(a, e, m) == pow(a, e, m) for a, e, m in
[(rng.randrange(10**9), rng.randrange(10**6), rng.randrange(2, 10**9)) for _ in range(500)]))
t('Fermat base 2 accepts Carmichael 561, 1105, 1729', all(N.fermat_fixed_base(n, 2) for n in (561, 1105,
1729)))
t('Miller-Rabin rejects 561, 1105, 1729 (10 rounds)', not any(N.is_prime_mr(n, 10, rng) for n in (561, 1105,
1729)))
t('Miller-Rabin accepts 2^127-1', N.is_prime_mr(2**127 - 1, 20, rng))
# Q1
t('DES known answer 85E813540F0AB405',
S.DES(h('133457799BBCDFF1')).encrypt_block(h('0123456789ABCDEF')) == h('85E813540F0AB405'))
t('Blowfish known answer 4EF997456198DD78', S.Blowfish(bytes(8)).encrypt_block(bytes(8)) ==
h('4EF997456198DD78'))
t('RC5-32/12/16 known answer 21A5DBEE154B8F6D', S.RC5(bytes(16)).encrypt_block(bytes(8)) ==
h('21A5DBEE154B8F6D'))
t('RC5 known answer 2 (F7C013AC5B2B8952)',
S.RC5(h('915F4619BE41B2516355A50110A9CE91')).encrypt_block(h('21A5DBEE154B8F6D')) ==
h('F7C013AC5B2B8952'))
k = h('0123456789ABCDEF23456789ABCDEF01456789ABCDEF0123'); m = b'8bytes!!'
t('3DES EDE round trip', S.TripleDES(k).decrypt_block(S.TripleDES(k).encrypt_block(m)) == m)
t('3DES with K1=K2=K3 equals single DES', S.TripleDES(h('133457799BBCDFF1') *
3).encrypt_block(h('0123456789ABCDEF')) == h('85E813540F0AB405'))
c = S.Blowfish(b'key12345'); iv = bytes(range(8)); msg = b'PATIENT 20481: AMOX 500MG'
t('ECB/CBC/CFB/OFB/CTR round trips', S.ecb_dec(c, S.ecb_enc(c, msg)) == msg and S.cbc_dec(c, iv,
S.cbc_enc(c, iv, msg)) == msg and S.cfb_dec(c, iv, S.cfb_enc(c, iv, msg)) == msg and S.ofb(c, iv, S.ofb(c,
iv, msg)) == msg and S.ctr(c, b'NONC', S.ctr(c, b'NONC', msg)) == msg)
t('ECB repeats leak (equal blocks -> equal ciphertext blocks)', S.ecb_enc(c, b'AAAAAAAA' * 2)[:8] ==
```

```
S.ecb_enc(c, b'AAAAAAAA' * 2)[8:16])
t('CBC hides equal blocks', len(set(S.blocks(S.cbc_enc(c, iv, b'AAAAAAAA' * 4)))) > 1)
# Q3
k = rsa.generate(1024, rng)
t('textbook RSA 65 -> 2790 -> 65 (n=3233)', N.modexp(65, 17, 3233) == 2790 and N.modexp(2790, 2753, 3233) == 65)
t('RSA-1024 round trip + CRT decrypt', rsa.crt_dec(rsa.textbook_enc(123456789, k['n'], k['e']), k) == 123456789)
t('OAEP round trip', rsa.oaep_decrypt(rsa.oaep_encrypt(b'token', k['n'], k['e'], rng), k) == b'token')
t('OAEP randomised', rsa.oaep_encrypt(b'x', k['n'], k['e'], rng) != rsa.oaep_encrypt(b'x', k['n'], k['e'], rng))
t('signature verifies, tampered message rejected', rsa.verify(b'm', rsa.sign(b'm', k), k['n'], k['e']) and not rsa.verify(b'n', rsa.sign(b'm', k), k['n'], k['e']))
p = N.gen_prime(20, rng); q = rsa.next_prime(p + 2, rng)
t('Fermat factors close primes', set(rsa.fermat_factor(p * q)) == {p, q})
t('Hastad e=3 broadcast recovers M', (lambda ks: rsa.hastad([pow(12345678901234567890, 3, x['n']) for x in ks], [x['n'] for x in ks]) == 12345678901234567890)([rsa.generate(512, rng, e=3) for _ in range(3)]))
# Q4
P, Q = dh.safe_prime(48, rng); G = dh.subgroup_generator(P, Q, rng); a, A = dh.keypair(P, Q, G, rng); b, B = dh.keypair(P, Q, G, rng)
t('DH shared secrets agree (48-bit toy)', dh.shared(a, B, P) == dh.shared(b, A, P))
t('DH p=23,g=5,a=6,b=15 gives K=2', pow(pow(5, 15, 23), 6, 23) == 2)
t('BSGS solves a discrete log', pow(G, dh.dlog_bsgs(G, A, P, Q)[0], P) == A)
t('public value 1 and p-1 rejected', not dh.valid_public(1, P, Q) and not dh.valid_public(P - 1, P, Q))
# Q5
T17 = ecc.Curve(17, 2, 2, (5, 1), 19)
t('toy curve: 19G = infinity, 2G = (6,3)', T17.mul(19, T17.G) is None and T17.add(T17.G, T17.G) == (6, 3))
t('P-256: G on curve and nG = infinity', ecc.P256.on_curve(ecc.P256.G) and ecc.P256.mul(ecc.P256.n, ecc.P256.G) is None)
d1, Q1 = ecc.ecdh_keypair(ecc.P256, rng); d2, Q2 = ecc.ecdh_keypair(ecc.P256, rng)
t('ECDH P-256 agreement', ecc.P256.mul(d1, Q2) == ecc.P256.mul(d2, Q1))
t('invalid point rejected by validate_public', not ecc.validate_public(ecc.P256, (1, 2)))
```

```
# Q6 / Q7
s = ecc.ecdsa_sign(ecc.P256, d1, b'm', rng)
t('ECDSA verifies; tampered message rejected', ecc.ecdsa_verify(ecc.P256, Q1, b'm', s) and not
ecc.ecdsa_verify(ecc.P256, Q1, b'x', s))
ca = RD.CA('CA', rng); A_ = RD.Identity('A', rng, ca); B_ = RD.Identity('B', rng, ca)
sa, sb = RD.handshake(A_, B_, ca.Q, rng); pk = sa.send(b'hello' * 50)
t('redesign: handshake + file round trip', sb.recv(pk) == b'hello' * 50)
try: sb.recv(pk); r = False
except ValueError: r = True
t('redesign: replay rejected', r)
try: RD.handshake(A_, B_, ca.Q, rng, mitm=lambda x, y: (Q1, Q1)); r = False
except ValueError: r = True
t('redesign: man-in-the-middle detected', r)
print(f'\n{sum(x for _, x in T)} of {len(T)} tests passed')
```

Output of `python test_all.py`:

```
PASS  gcd(1071,462)=21
PASS  egcd(240,46)=(2,-9,47)
PASS  modinv(17,3120)=2753
PASS  modexp(4,13,497)=445
PASS  modexp agrees with pow() on 500 random inputs
PASS  Fermat base 2 accepts Carmichael 561, 1105, 1729
PASS  Miller-Rabin rejects 561, 1105, 1729 (10 rounds)
PASS  Miller-Rabin accepts 2^127-1
PASS  DES known answer 85E813540F0AB405
PASS  Blowfish known answer 4EF997456198DD78
PASS  RC5-32/12/16 known answer 21A5DBEE154B8F6D
PASS  RC5 known answer 2 (F7C013AC5B2B8952)
PASS  3DES EDE round trip
PASS  3DES with K1=K2=K3 equals single DES
PASS  ECB/CBC/CFB/OFB/CTR round trips
PASS  ECB repeats leak (equal blocks -> equal ciphertext blocks)
```

PASS CBC hides equal blocks

PASS textbook RSA 65 -> 2790 -> 65 (n=3233)

PASS RSA-1024 round trip + CRT decrypt

PASS OAEP round trip

PASS OAEP randomised

PASS signature verifies, tampered message rejected

PASS Fermat factors close primes

PASS Hastad e=3 broadcast recovers M

PASS DH shared secrets agree (48-bit toy)

PASS DH p=23,g=5,a=6,b=15 gives K=2

PASS BSGS solves a discrete log

PASS public value 1 and p-1 rejected

PASS toy curve: 19G = infinity, 2G = (6,3)

PASS P-256: G on curve and nG = infinity

PASS ECDH P-256 agreement

PASS invalid point rejected by validate_public

PASS ECDSA verifies; tampered message rejected

PASS redesign: handshake + file round trip

PASS redesign: replay rejected

PASS redesign: man-in-the-middle detected

36 of 36 tests passed

Output of `python xcheck.py` (student code against the independent library):

{
 "own OAEP ciphertext decrypts in library": true,
 "library OAEP ciphertext decrypts in own code": true,
 "own PKCS#1 v1.5 signature verifies in library": true,
 "library signature verifies in own code": true,
 "P-256 public key d*G equals library": true,
 "own ECDSA signature verifies in library": true,
 "library ECDSA signature verifies in own code": true,

"3DES block equals library": true,
"Blowfish block equals library": true
}

The raw measurements of every experiment are stored as JSON files (q1.json – q7.json) produced by the drivers in Appendix F, and every table in this report was generated directly from them; they are supplied with the code archive.